

SERVICE BOTS • MILITARY BOTS • AUTONOMOUS BOTS

SERVO

FOR THE ROBOT INNOVATOR

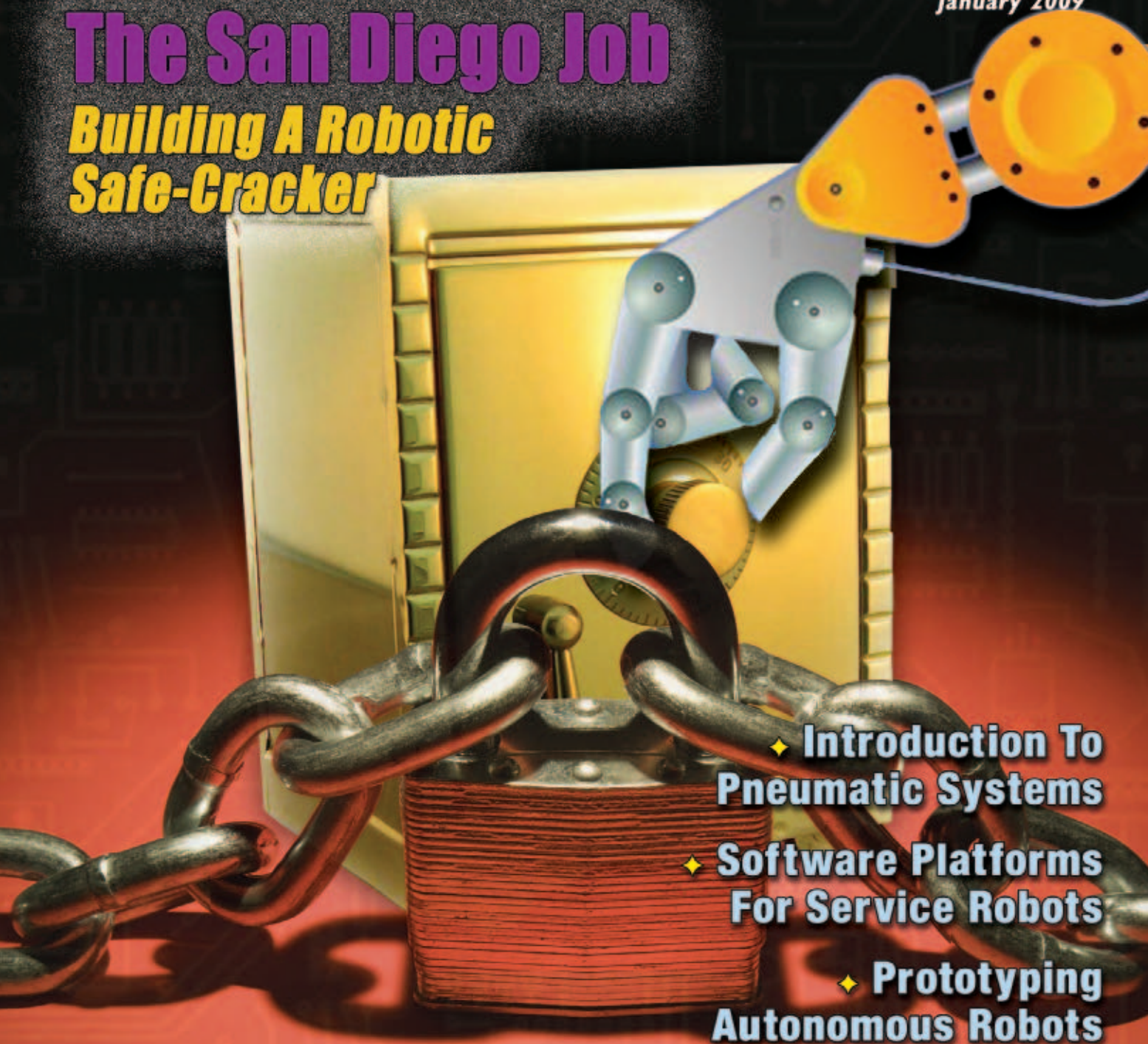
www.servomagazine.com

MAGAZINE

January 2009

The San Diego Job

*Building A Robotic
Safe-Cracker*



♦ Introduction To
Pneumatic Systems

♦ Software Platforms
For Service Robots

♦ Prototyping
Autonomous Robots

- a) Science
 - b) Technology
 - c) Engineering
 - d) Mathematics
 - e) ~~All of the above~~
- ROBOTICS**

Classroom Lab Kit
bundles start at \$699



Studies prove what we at VEX® already knew... Students love Robotics.

Given a choice, middle and high school students prefer robotics among science courses. And our Classroom Lab Kits make it easier than ever to bring VEX Robotics to your school while making your budget go farther. With standards-based curriculum available from Intelitek, Carnegie-Mellon and Autodesk, VEX is quickly becoming the robotics platform of choice among schools internationally. We now offer local, regional, national and international competitions for students to test their skill and express themselves. Visit RobotEvents.com for event information. Only one choice is clear when considering an educational robotics platform – the VEX Robotics Design System.

Think. Create. Build. Amaze. **Vex.**

VEX
ROBOTICS DESIGN SYSTEM



A product of Innovation First.

Copyright 2009. Innovation First, Inc.

VISIT WWW.VEXROBOTICS.COM

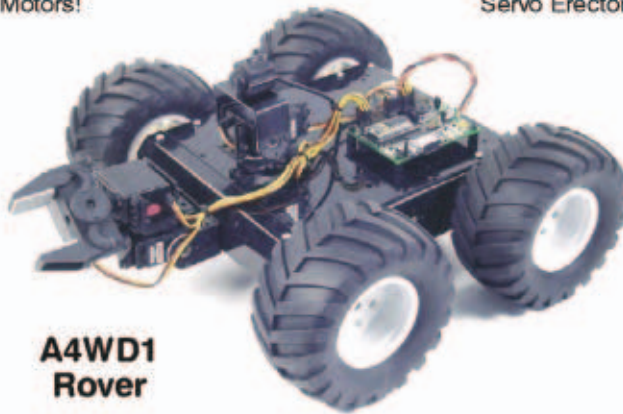


The Lynxmotion Servo Erector Set
Imagine It... Build It... Control It!

Featured Robot

Rugged Chassis!
 Strong Motors!

Compatible with
 Servo Erector Set!



**A4WD1
 Rover**

The Lynxmotion Aluminum 4WD1
 is a robust and expandable chassis for
 your RC or autonomous robot project.

Chassis Length: 8.75"
 Chassis Width: 7.0"
 Chassis Height: 3.5"



Biped Nick



Biped Pete



Biped Scout



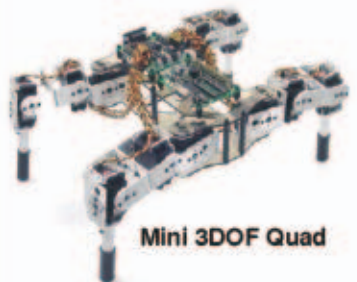
Biped 209



Walking Stick



Mini 2DOF Hex



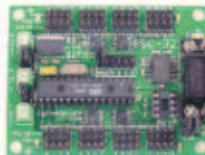
Mini 3DOF Quad

**With our popular Servo Erector Set you can easily
 build and control the robot of your dreams!**

Our interchangeable aluminum brackets, hubs,
 and tubing make the ultimate in precision
 mechanical assemblies possible. The images here
 are just a sample of what can be built. The Bot
 Board II and SSC-32 provide powerful control
 options. Our Visual Sequencer program provides
 powerful PC, Basic Atom, or BS2 based control.



Bot Board II - \$24.95
 Carrier for Atom / Pro, BS2, etc.
 Servo and Logic power inputs.
 5vdc 250mA LDO Regulator.
 Buffered Speaker.
 Sony PS2 game controller port.
 3 Push button / LED interface.



SSC-32 - \$39.95
 32 Channel Servo Controller.
 Speed, Timed, or Group moves.
 Servo and Logic power inputs.
 5vdc 250mA LDO Regulator.
 TTL or RS-232 Serial Comms.
 No better SSC value anywhere!

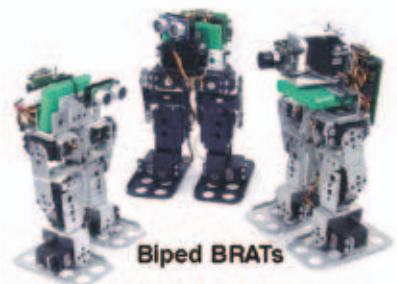
We also carry motors, wheels, hubs, batteries, chargers,
 servos, sensors, RC radios, pillow blocks, hardware, etc!



Visit our huge website to see our complete line
 of Aluminum and Lexan based robot kits,
 electronics, and mechanical components.



CH3-R Hexapod



Biped BRATs



SES Arm



Images represent a fraction of what can be made! www.lynxmotion.com SES now has 62 unique aluminum components!

SERVO

MAGAZINE

Departments

- 06** Mind/Iron
- 18** New Products
- 19** Events Calendar
- 37** Robotics Showcase
- 64** SERVO Webstore
- 81** Robo-Links
- 81** Advertiser's Index



PAGE 08



Columns

- 08** **Robytes**
by Jeff Eckert
Stimulating Robot Tidbits
- 10** **GeerHead**
by David Geer
Open Robot Platform is Student's Delight!
- 14** **Ask Mr. Roboto**
by Dennis Clark
Your Problems Solved Here
- 68** **Twin Tweaks**
by Bryce and Evan Woolley
The San Diego Job
- 74** **Robotics Resources**
by Gordon McComb
Kits for Learning About Electronics
- 78** **Then and Now**
by Tom Carroll
Government and Military Robots



PAGE 10

SERVO Magazine (ISSN 1546-0592/CDN Pub Agree#40702530) is published monthly for \$24.95 per year by T & L Publications, Inc., 430 Princland Court, Corona, CA 92879. PERIODICALS POSTAGE PAID AT CORONA, CA AND AT ADDITIONAL ENTRY MAILING OFFICES. POSTMASTER: Send address changes to **SERVO Magazine, P.O. Box 15277, North Hollywood, CA 91615** or Station A, P.O. Box 54, Windsor ON N9A 6J5; cpcreturns@servomagazine.com



PAGE 50



Features & Projects

30

Build The Ultimate Robot

by Michael Simpson
Part 4: Firebot.

38

Prototyping Autonomous Robots

by John Blankenship and
Samuel Mishal

Designing and building autonomous robots is an involved process. However, with a simulation program specifically set up for this kind of prototyping, things get a lot easier.

43

BLDC Hardware

by Fred Eady

Assemble a microcontroller-based BLDC motor driver using motor control building blocks that can be scaled and applied to other BLDC designs.

50

Software Platforms for Service Robots

by Michael Somby

This article reviews several of the software products available on the market today for service robotics applications.

61

An Introduction to Pneumatic Systems

by David A. Ward

Give your robots extra muscle by incorporating pneumatics into your designs.

The Combat Zone... Features

- 20 PARTS IS PARTS:
A123 Battery Packs
- 22 MANUFACTURING:
Moving on With Your Robot

Events

- 24 Results and Upcoming Competitions
- 24 Event Report: Roaming Robots Winter Tour
- 26 Event Report: Combots Texas Cup at Maker Faire Austin

Robot Profile

- 28 Devil's Plunger

Mind / Iron



by Bryan Bergeron, Editor

INNOVATING ON A BUDGET

Based on numerous emails from readers, one of the frustrations of learning about the latest developments in robotics is the cost involved in keeping up with the leading edge of innovation. Replicating, for example — the latest micro-flying robot — can cost thousands of dollars and weeks to months of effort — assuming you can find or fabricate the parts. So what's the solution to innovating upon the latest robotic discovery without breaking the bank? I've found several strategies invaluable in pursuing innovation on a budget, especially if you're working alone.

The first and most important strategy is to focus. Define exactly what technology or aspect of robotics that you're interested in exploring and developing. Take one of the new remote-controlled micro-air vehicles with onboard video. Realize that it's impractical to build one from scratch and focus on one or perhaps even two aspects of the technology.

You could focus on micro-miniature robotics platforms, ultra-lightweight, high-capacity batteries, ultra-miniature video capture, or more efficient, insect-like flight, among other areas.

With your focus defined, you'll be more likely to actually achieve your goal. You'll also avoid the common pitfall of getting bogged down in a dead-end because of unexpected costs or component limitations.

The second strategy is to approximate — and not get lost in trying to exactly replicate — the target technology. Even if your focus is insect-sized flying robots, you needn't constrain yourself to a life-sized wingspan. Start with a more reasonable platform with a 12" wingspan that will allow you to use larger, heavier, and much more affordable motors, batteries, and onboard control electronics. By the time you finish a 12" flying robot, you'll no doubt have made several personal discoveries and have a much better idea of what's really involved in going for a scaled down version of the robot.

The third cost- and sanity-saving strategy is to repurpose when possible. Building any robot from the ground up using a piece-by-piece approach is expensive. Assuming you can locate vendors that will ship in single item quantities, you'll spend a fortune in postage and handling fees.

For example, suppose you want to manipulate the control surfaces of your flying robot with one of the new micro-miniature quartz motors. You could spend \$800 or more for a "developers kit" from DigiKey (www.digikey.com), or go to eBay and pick up a second-hand camera that uses one of the motors to control an automatic, retractable lens cover. It's amazing what you can find in the way of repurposable robot parts in a \$75, second-hand digital camera.

Case in point: Several readers wrote in requesting information on how they could replicate the three

Published Monthly By
T & L Publications, Inc.
 430 Princeland Ct., Corona, CA 92879-1300
(951) 371-8497
 FAX **(951) 371-3052**
 Webstore Only **1-800-783-4624**
www.servomagazine.com

Subscriptions
 Toll Free **1-877-525-2539**
 Outside US **1-818-487-4545**
 P.O. Box 15277, N. Hollywood, CA 91615

PUBLISHER
 Larry Lemieux
publisher@servomagazine.com

**ASSOCIATE PUBLISHER/
 VP OF SALES/MARKETING**
 Robin Lemieux
display@servomagazine.com

EDITOR
 Bryan Bergeron
techedit-servo@yahoo.com

TECHNICAL EDITOR
 Dan Danknick
dan@teamdelta.com

CONTRIBUTING EDITORS
 Jeff Eckert
 Gordon McComb
 Dennis Clark
 Fred Eady
 Michael Somby
 Samuel Mishal
 Donald Sung
 Wendy Maxham
 Evan Woolley
 Tom Carroll
 David Geer
 R. Steven Rainwater
 Kevin Berry
 John Blankenship
 Michael Simpson
 John Frizell
 Bryce Woolley

CIRCULATION DIRECTOR
 Tracy Kerley
subscribe@servomagazine.com

**MARKETING COORDINATOR
 WEBSTORE**
 Brian Kirkpatrick
sales@servomagazine.com

WEB CONTENT
 Michael Kaudze
website@servomagazine.com

PRODUCTION/GRAPHICS
 Shannon Lemieux
 AJ Cohen

ADMINISTRATIVE ASSISTANT
 Debbie Stauffacher

Copyright 2009 by
T & L Publications, Inc.
 All Rights Reserved

All advertising is subject to publisher's approval. We are not responsible for mistakes, misprints, or typographical errors. *SERVO Magazine* assumes no responsibility for the availability or condition of advertised items or for the honesty of the advertiser. The publisher makes no claims for the legality of any item advertised in *SERVO*. This is the sole responsibility of the advertiser. Advertisers and their agencies agree to indemnify and protect the publisher from any and all claims, action, or expense arising from advertising placed in *SERVO*. Please send all editorial correspondence, UPS, overnight mail, and artwork to: **430 Princeland Court, Corona, CA 92879.**

Mind/Iron Continued →

gram, 10 cm wingspan Delfly Micro, developed by engineers at Delft University of Technology in the Netherlands (home.tudelft.nl). A few minutes on the web revealed the micro-air vehicle uses a one gram battery to stay aloft for three minutes; it has a top speed of five meters/second; and the 0.5 gram video camera is designed for a surgical endoscope.

Let's assume your focus is how to handle live video from a micro-air vehicle. You'll need a suitable camera. However, micro-miniature endoscope cameras are expensive — expect to pay \$5,000 or more for single quantity cameras. A quick search on eBay revealed what looked like a suitable camera/endoscope assembly for \$250. Not inexpensive, and a risky purchase without a schematic, but much cheaper than \$5,000 for a new camera with documentation.

Again, if you accept an approximation — a slightly heavier, mass-produced camera sold for surveillance work — you'll save hundreds. You might find it sufficiently challenging — and rewarding — to extract one of the ubiquitous \$50 mini-surveillance video cameras from its plastic housing and mount it in a lighter, more compact structure.

Furthermore, unless you want to bother with the platform and control infrastructure, consider purchasing a ready-made plane, helicopter, or other flying vehicle from Tower Hobbies (www.towerhobbies.com) or even RadioShack. Foam and plastic, remote-controlled planes,

LINEAR SERVOS

→ Radio Control
→ Industrial
→ Robotics

Order online at
www.firgelli.com



Firgelli

In the November issue of SERVO on page 70, the website info was inadvertently left off the Firgelli ad. We apologize for any inconvenience.

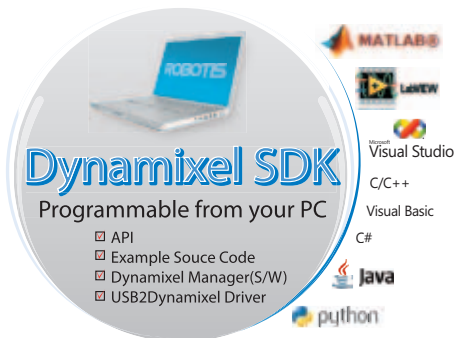
birds, and helicopters can be had for less than \$100 — far less than you'd spend creating a flying platform from scratch.

On the other hand, if your focus is creating a more efficient flying platform, then by all means take out the balsa wood and Styrofoam. Repurpose the camera and control electronics from some other craft.

Most importantly, when you've succeeded and developed your robot or enabling technology, document your journey and share it with your fellow readers. **SV**

Why DYNAMIXEL?

High-performance networked actuators for robots



AX-12+



- Engineering plastic gear
- Control network: TTL
- Gear Ratio : 254
- Position Sensor : POT

	at 7V	at 9.6V
Holding torque(Kgf.cm)	12.5	17.1
Speed(sec/60deg)	0.269	0.196
Weight(g)	53.5	

RX-28 / DX-117



- Full metallic gear
- Control network: RS-485
- Gear Ratio : 193
- Position Sensor : POT

	RX-28		DX-117	
	at 12V	at 16V	at 12V	at 16V
Holding torque(Kgf.cm)	28.3	37.7	28.3	37.7
Speed(sec/60deg)	0.167	0.126	0.167	0.126
Weight(g)	72		67	

RX-64



- Full metallic gear
- Control network: RS-485
- Gear Ratio : 200
- Position Sensor : POT

	at 15V	at 18V
Holding torque(Kgf.cm)	64.4	77.2
Speed(sec/60deg)	0.188	0.157
Weight(g)	125	

EX-106



- Full metallic gear
- Control network: RS-485
- Gear Ratio : 164
- Position Sensor : Encoder

	at 14.8V	at 18.5V
Holding torque(Kgf.cm)	84	106
Speed(sec/60deg)	0.182	0.143
Weight(g)	155	

ROBOTIS

Visit www.robotis.com for more information on parts and accessories.
For business partnership contact us at contactus2@robotis.com

www.robotis.com



Heavy-Duty Hauler



With a gross weight estimated at close to 700 tons, Caterpillar's haul trucks will qualify as the world's heaviest autonomous vehicles. Photo courtesy of Caterpillar, Inc.

In many nooks and crannies of the bot industry, miniaturization is the name of the game. Not so at Caterpillar, Inc. (www.cat.com), which is working with Carnegie Mellon's Robotics Institute (www.ri.cmu.edu) to develop autonomous versions of haul trucks used in mining operations. Among them is the 793D haul truck, which can handle payloads of 240+ tons. Aimed at increased productivity and zero-injury safety levels, the behemoths will be fitted with an array of gadgets to keep them functioning, including GPS receivers, laser range finders, visual sensors, and so on.

According to a spokesman, "In an autonomous operation, a truck can be programmed to consistently back under the shovel to within 25 cm (10 in) all day, every day, so the shovel operator does not have to waste valuable time trying to chase the truck due to poor spotting; an autonomous truck cycle time will be consistent throughout the shift — beginning, end, and in the middle; an operator gets tired at the end of a shift, while an autonomous machine operates at the same efficiency 24/7." By 2010, several are scheduled to be operating in some mine sites

owned by BHP Billiton, which happens to be the world's largest mining company.

Cleanliness Is Next To Botliness



KUKA's KR 15 SL handles meat, pharmaceuticals, etc. Photo courtesy of KUKA Robotics.

You may usually relate industrial robots to grease, paint, and other unsavory glop, but KUKA offers a range of machines for clean rooms, medical applications, food handling, etc. For example, the newly introduced KR 15 SL is "the only articulated robot made exclusively of stainless steel on all surfaces," making it suitable for fields with stringent requirements for hygiene, sterility, and the absence of particle contamination. With an ingress protection (IP) rating of 67 (dust tight, sealed against water intrusion from heavy seas or powerful jets), it is great for handling food, pharmaceuticals, and other medical products. But if you need a bot to operate in deep freeze environments, you may prefer the KR 180-2 PA Arctic unit which functions in temperatures as low as -22°F (-30°C) without heaters or other related accessories. You can see the

whole family of four-, five-, and six-axis bots at www.kuka.com.

Honda Has No Wheels



Stair climbing with Honda's experimental walking assist device. Photo courtesy of Honda.

Also in a sense designed to move meat around is the second-generation experimental walking assist device, introduced in November by Honda Motor Co. This is a far cry from a Gold Wing, consisting more of a wedgie with legs. It's aimed at people who can still walk but not very well; e.g., those who suffer from Parkinson's, stroke, or a six-pack of Milwaukee's Best.

According to Honda, "The new walking assist device with the body weight support system reduces the load on leg muscles and joints (in the hip, knees, and ankles) by supporting a portion of the person's body weight. The device has a simple structure consisting of seat, frame, and shoes, and the user can put it on

by simply wearing the shoes and lifting the seat into position.”

In addition, it incorporates a mechanism that directs forces toward the user’s center of gravity and varies the assist force to match the leg’s movements. Specs include 6.5 kg weight, including shoes and batteries, and a two-hour operating time driven by twin motors and a lithium-ion battery. Testing continues at Honda’s Saitama factory.

Bot IDs And Destroys IEDs



Hon. Warren Snowdon unveils Australia's latest anti-IED device — SPIKER. Photo courtesy of Australian Dept. of Defense.

Yes, it looks like G. Gordon Liddy fondling a WWII-vintage scanning electron microscope. But it’s really Warren Snowdon, MP, Australia’s current Minister for Defense Science and Personnel, introducing a new bot designed to counter improvised explosive devices. Dubbed the Special Purpose IED Killer Experimental Robot

(SPIKER), the remote-controlled tracked vehicle employs an array of devices — including explosives of its own — to dispose of IEDs. Fitted with any of several weapon systems, it can climb stairs and perform long-range communications.

Perhaps the most remarkable feature is the reported \$15,000 price tag, which is about a tenth that of other bots that do the same job. For details, visit www.dsto.defence.gov.au.

Flower Power

Technical specs are strangely nonexistent, but the robot research lab at Korea’s Chonnam National University (web.jnu.ac.kr/en) claims to have developed a robotic plant that is in many ways preferable to the real thing. What we do know is that it is about 4.25 ft (130 cm) tall, 1.3 ft (40 cm) in dia., and made up of a pot, a stem, and five buds.

Through some unspecified mechanism, it emits oxygen, moisture, and smelly stuff into the room, much like



A robotic plant developed by Chonnam National University. Photo courtesy of Yonhap News Agency.

a real flower. In addition, it bends toward anyone who comes within 40 cm of it and its buds bloom. The flower also blooms and shakes if you speak in a loud voice. Finally, it will dance if you play some music.

According to project leader Park Jong-oh, “It’s a fresh attempt to introduce the concept of plants, rather than humans or animals, to robot making.” Maybe so, but it’s only a matter of time before someone comes up with some robotic aphids. **SV**

HVW Sensors > The Force

How else do you think that little floaty robot was able to kick Luke’s ass on the Falcon?

IR, motion, temperature sensors - we stock what you need to give your project the edge against wimpy moisture farmers.



1-866-276-2687
www.hvwtech.com

HVW Technologies is a division of Solidus Inc.

PS- We all know blowing up the Death Star was just a fluke. If the targeting sensors had been on, it could’ve been done with just one torpedo.





GEER HEAD

by David Geer

Contact the author at geercom@alltel.net

Open Robot Platform is Student's Delight!

Robotician and PhD candidate Abe Howell created the Open Robot platform so it could provide broad applications across robotics education from kindergarten through the university level.

Open Robot is, not surprisingly, completely open source.

Teachers and students may modify and distribute all the robot's documentation, software, printed circuit board designs, and mechanical design files as long as credit to the inventor is maintained.

Howell is eager to see an open source community grow up around the Open Robot, adding hardware, software, sensors, and "mechanical end-effectors."

Its Construction

The Open Robot uses multiple

on-board sensors in an array which gather input from surrounding environments. Its Sharp GP2D120 infrared sensors enable the robot to sense the presence, distance, and relative location of obstacles. "The distance to an object is calculated in inches, centimeters, or other standard units of length," Howell explains.

These sensors teach students to calibrate the robots. By taking eight-bit obstacle detection readings at incremental distances between the sensor and object, students may plot the sensor's response. From this, they create regression lines and functions to arrive at detection distances from

objects when future readings are taken, Howell continues.

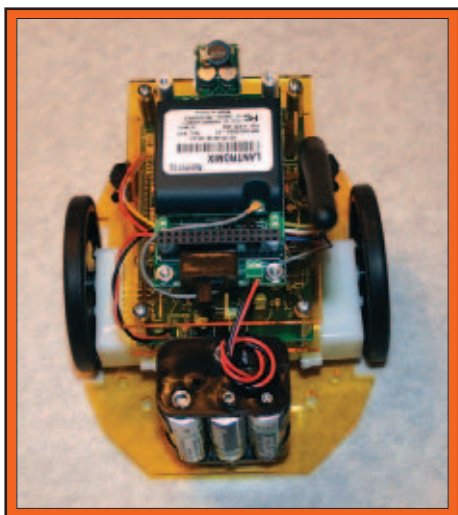
The robot senses light, as well as objects and surfaces. The robot's two forward CdS light sensors enable it to follow or avoid ambient light, depending on intensity.

The Open Robot also comes with capability for Radio Frequency Identification (RFID) tag reader applications and experiments. A chip module and antenna are integrated into the robot to read and write data to and from passive RFID tags (which the user must purchase separately).

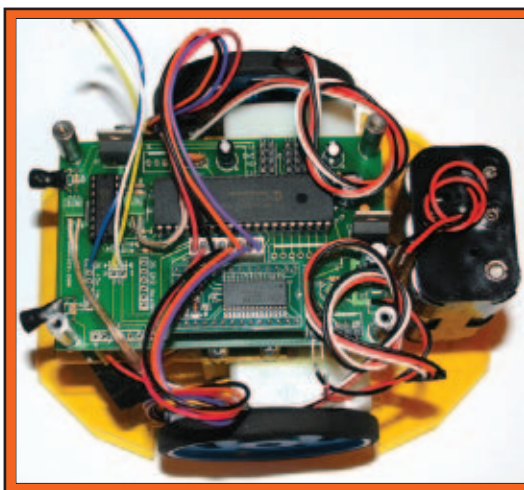
The tags may be positioned on the floor in the Open Robot's environment, and its antenna will read them when the robot passes over. The robot also uses in-range RFID tag discovery to determine when tags are in range and when to stop.

These RFID tags are great for doing maze navigation experiments. "Simply program each tag with a special code, then place them throughout the maze," says Howell. By creating artificial intelligence for the robot, students enable it to navigate the maze using the robot's built-in RFID reader. Since RFID tags have no power source, they receive power when an antenna comes within

Back of Open Robot with battery pack.



Open Robot controller board center features a PIC18F4520 microcontroller.



range and interrogates the tag.

Roboticians, teachers, and students operate Open Robot via wireless remote control. Options include a MatchPort b/g module for 802.11 wireless or an XBee module (sold separately) based on ZigBee wireless technology.

With 802.11, students control the robot from a laptop or desktop machine equipped with an 802.11 PC card. XBee enables wireless control through a serial port protocol.

Open Robot offers a free TCP Functions Class library for those who want to write their own programs that are compatible with the MatchPort connectivity module. The library works with Microsoft Visual Basic or C# Express. Programmers don't have to know TCP socket programming to program with the library. Open Robot also comes with a free ZigBee Functions Class library for programming to that module.

Open Robot is capable of driving with preset wheel velocities. To do this, the robot works with Nubotics wheel encoders. The model WW02 encoders provide feedback that helps the robot maintain wheel velocity despite the floor environment. The encoders also enable the robot to travel a preset distance before stopping or to turn in specific directions by preset degrees.

Adding a camera as an additional sensor allows Open Robot to perform vision processing or take pictures as it travels. The robot is compatible with the SRV-1 Blackfin camera board produced by Surveyor Corp. The board — a 500 MHz Blackfin Digital Signal Processor (DSP), includes a 1.3 megapixel camera. The Open Robot's 32 pin connector makes it possible to easily stack the camera and DSP directly on top of the robot.

Those who plan to purchase and use the SRV-1 should select the MatchPort b/g because it has the extra bandwidth the camera will need to send images back to the user's computer.

Open Robot Highlights

The Open Robot was designed for use in multiple robotics experiments and learning initiatives across all education levels. Here are some of the key elements that make the robot fit for all ages:

- 1) The software platform is 100 percent open source. Anyone across academia can develop and contribute to the platform.
- 2) All electrical and mechanical design files promote open robot modifications. Files include a 3D model of the whole robot, all PCB schematics and ExpressPCB designs, and C# and Java based programs and libraries.
- 3) A parts list is freely available for anyone who wants to build the robot from scratch or modify it to their own needs.
- 4) The base robot is inexpensive, flexible, and modular. Students add sensors, infrared, RFID module and antenna, compass, GPS, and sonar as required by specific experiments.
- 5) SRV-1 camera and board available for stills and computer vision technology.
- 6) ZigBee or 802.11 wireless available to suit multi-robot swarms or camera-based experiments, respectively.

Controller Board

The robot's brain consists of a PIC18F4520 controller board. This board includes all the hardware necessary to mobilize the robot, read from its sensors, and communicate with the MatchPort module. The controller runs firmware created to continuously read the robot's infrared and light sensors, as well as battery voltage. The firmware updates the data from these devices so that the data is always current.

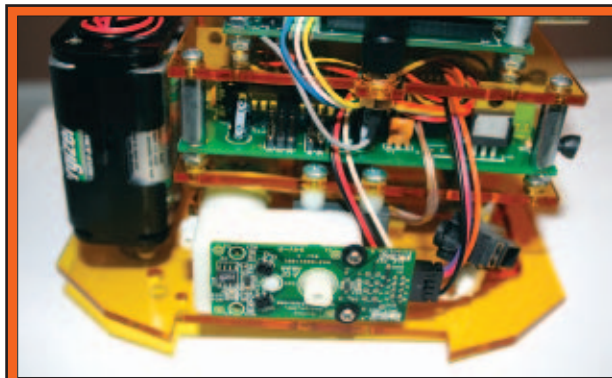
The firmware uses a special serial interrupt service routine to ingest output data from the MatchPort and to complete commands immediately upon receipt.

The PIC controller's boot loader program uploads custom firmware and firmware updates via wireless. Once the

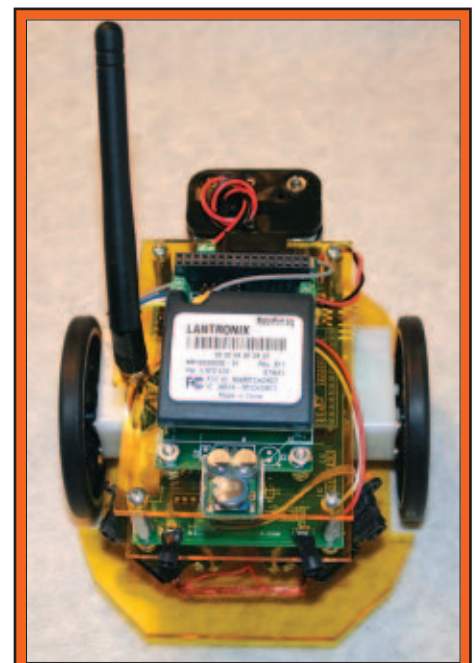
firmware is uploaded, the controller reboots and begins to run the upgrade immediately. "User created firmware must include the CCS C boot loader files or the new firmware will overwrite the existing boot loader," comments Howell.

In this case, the controller would need to be reprogrammed to include the boot loader which able roboticists can do for themselves, or they can ship the controller back to the maker

Close-up of Open Robot wheel encoder, bottom center.



Open Robot front with antenna.



Swarm Robot Fun!

Open Robots may be programmed to work as a team for experiments in swarm robot behavior. In an experiment designed for multiple robots, students program RFID tags with virtual food and arrange them across the floor. The goal is to see which robot survives the longest by getting the most food available.

Students may specify rules that control the number of equally sized pieces of food that a robot can eat from a single tag, or how many pieces of food a robot can retain at one time. Additional rules include parameters for the frequency of meals and the amount of food necessary for survival.



Open Robot with SRV-1 camera and board mounted.

Resources

Open Robot
www.abotix.com/open_robot.htm

Abe Howell's Robotics
www.abotix.com/abotix_about.htm

the PCH — also works but is a little harder to get started with. The educational version of the PCWH compiler only works with the PIC18F4520 controller. However, students and teachers receive the educational version of the compiler for about \$50. CCS also makes an in circuit programmer/debugger that may prove useful.

Conclusion

Open Robot is a great new option in platforms for academic institutions seeking to start or expand offerings in robotics education at any level. **SV**

at their own expense.

The Open Robot provides the option to use the original firmware that comes with it or to create custom firmware using a CCS C PIC compiler.

"I recommend using the PCWH compiler (about \$500) because it has a Windows IDE and several other nice features," says Howell.

The command line only version —



Item #975
\$99.95

The Pololu 3pi robot is a high-performance, compact mobile platform featuring:

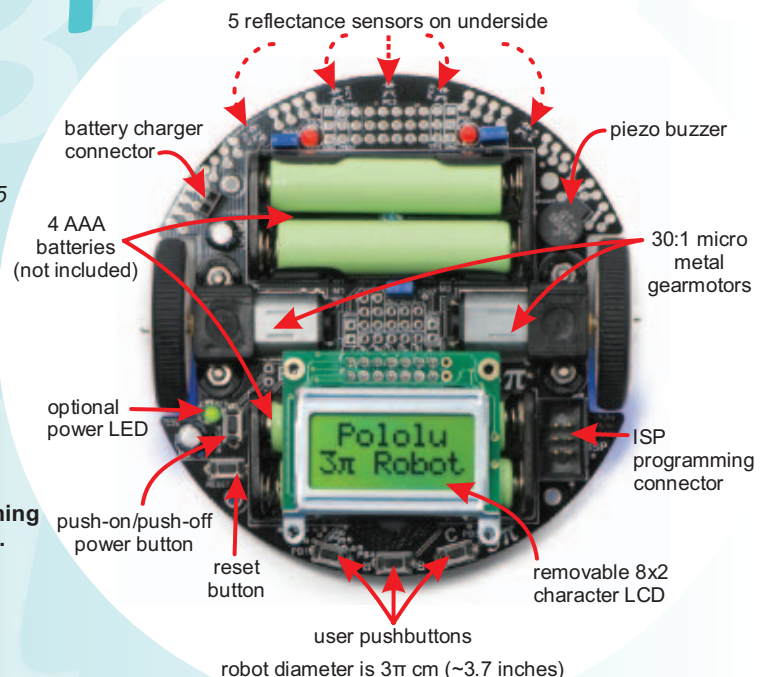
- * Two metal gearmotors
- * Five reflectance sensors
- * 8x2 character LCD
- * Three user pushbuttons
- * Buzzer and LEDs
- * High-traction silicone tires
- * Speeds exceeding 3 ft/sec using innovative constant-voltage motor supply

All peripherals are connected to an ATmega168 microcontroller running at 20 MHz, with free C-programming tools, libraries, and support for the Arduino environment.

Pololu
Robotics & Electronics

6000 S. Eastern Ave. 12D, Las Vegas, NV 89119

3pi Robot



Find out more at www.pololu.com/3pi or by calling 1-877-7-POLOLU.

CrustCrawler

Robotics

"Build Smarter."

AX12 Smart Arm!

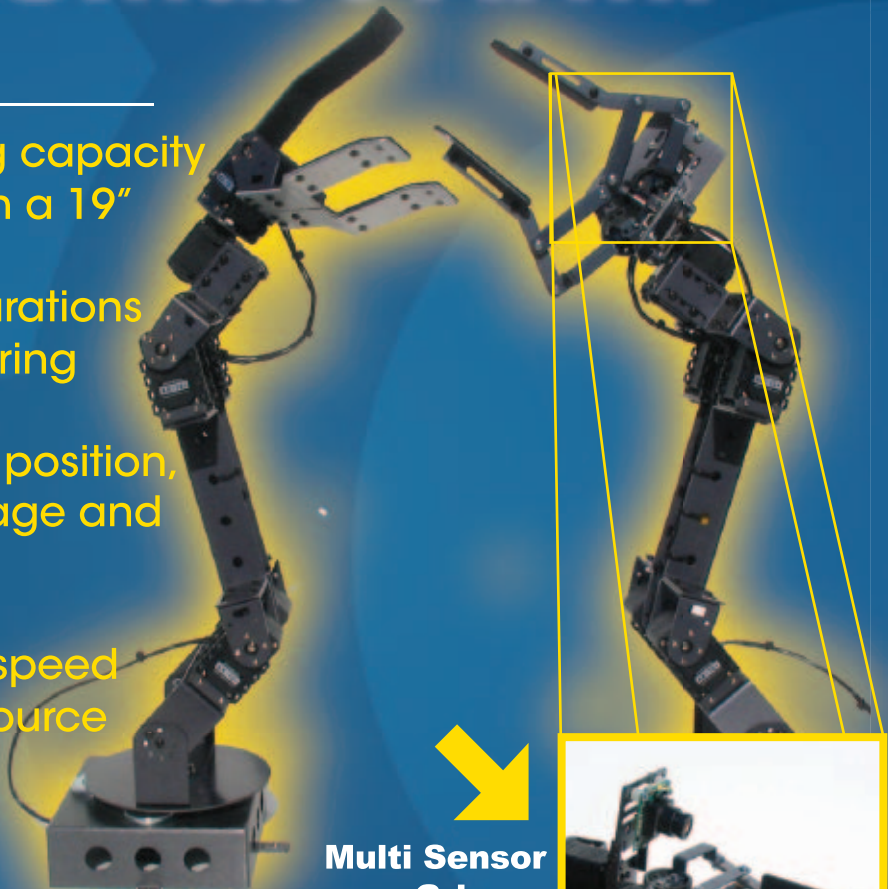
Key Features:

- ✓ 1.8+ pound lifting capacity
- ✓ 21.42" length with a 19" reach
- ✓ 3 gripper configurations
- ✓ Smooth, ball bearing turntable
- ✓ Full feedback on position, speed, load, voltage and temperature
- ✓ Ultra fast 1mbps communication speed
- ✓ Extensive open source libraries and APIs

**NEW from
Crust Crawler Interactive!**

Testdrive the Smart robotic arm over the web with CrustCrawler's new Telerobotics application!

Visit us at www.crustcrawler.com for details!



Multi Sensor Gripper

Engineered for cameras, pressure sensors, Ultrasonics, and IR detection.



Optional "2 in 1" Gripper



Great for grabbing large objects!



Great for grabbing objects of all sizes!!

See our web site for pictures, videos and more!
www.CrustCrawler.com

Or call us at:
480-577-5557



Our resident expert on all things robotic is merely an email away.
roboto@servomagazine.com

Tap into the sum of *all human knowledge* and get your questions answered here! From software algorithms to material selection, Mr. Roboto strives to meet you where you are — and what more would you expect from a complex service droid?

ASK MR. ROBOTO

by
Dennis Clark

Q. Here's one for you: I'm told that the PS3 wireless controllers use Bluetooth to talk to the PS3 console. How can I interface one with my own Bluetooth transceiver and use it as a robot controller?

— Joe in Fort Collins

A. Interesting question, and quite a bit to look up to answer, as well! The Bluetooth in the PS3 controller (which is called the "SixAxis" controller) is a slave, not a master device, so this means that it wants to pair with a computer which has the master software on it. If all you are wanting is to get the basic joystick operation from the device, you would use the USB connection on it which will work with any computer that uses a USB HID (Human Interface Device) driver for that kind of thing. If you are using a microcontroller that has USBOTG (On The Go) software in it, then you can connect right up, but only with the cable. If you want the Bluetooth capability, you can leave out most of our hobby robotics brains. MOST, but not all.

There are always those intrepid folks out there that cannot leave well enough alone and sally forth into the

unknown, or even intentionally obfuscate. Interestingly, the reason that this controller is called the SixAxis controller is that it has six axis accelerometers and a gyro in it! This data is only available over the Bluetooth link, so now we have a reason to hack the system. And, in fact, I found at least one place that did this and published full specs on how to get a Linux system to pair with the PS3 SixAxis and get all that accelerometer and gyro data. Here's the URL:

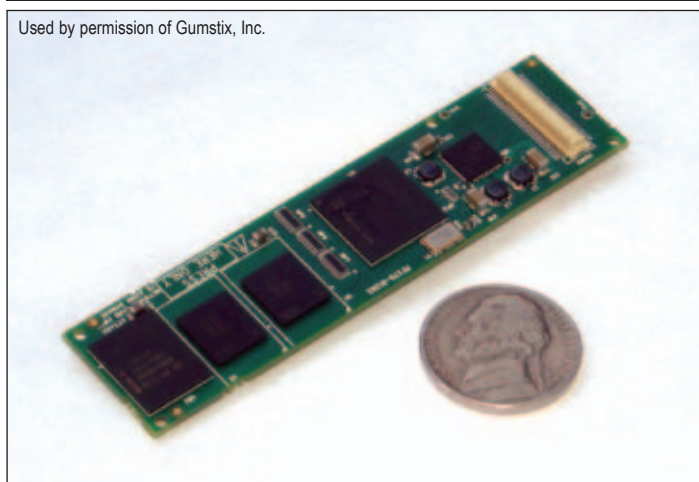
www.pabr.org/sixlinux/sixlinux.en.html.

They have a couple of very nice videos that show the full use of all the accelerometers using hobby servos. There are some complications apparently that come from the PS3 Bluetooth controller not being precisely like other Bluetooth devices in the way that it gets the accelerometer data out. They have special hacks made to deal with this. It isn't obvious in the first video but this is a fully embedded project that isn't using "big iron" to control these servos. The Linux kernel with the full Bluetooth master code is running on a Gumstix processor board which is quite small, in fact, they are more or less the size of a pack of stick gum, so they can easily fit into smaller robots. The Gumstix processor boards are generally under \$200; you can them at **<http://gumstix.com/>**.

The Gumstix boards (see Figure 1) run embedded Linux with full open source available and are very capable devices for their size. The Verdex Pro XM4-bt (one with Bluetooth) has 64 MB of RAM, 16 MB of Flash, and a microSD slot for mass storage. All this runs on a 400 MHz XScale processor. For a Linux box, that isn't all that fast but for an embedded processor, it is. It still runs at power levels that we can tolerate with our batteries in a robot and the board weighs in at about 8g (1/4 oz).

The Gumstix processor boards are programmed using the OpenEmbedded build system which uses gcc and runs primarily on Linux distributions. So, you'll need to have a Linux system as your primary build machine. This build environment requires you to check out your source code to build images; to do this, you'll need to have Subversion on your machine. Subversion is a source code control and

Figure 1. A Gumstix processor board.



repository system. A serial connection and Kermit (a terminal emulator program) is all you need to program a board.

Since you've seen a lot of big words and serious programmer "geek speak" in this last paragraph, you should know that unless you are already a Linux expert, you will have a fairly steep learning curve with this. Those of you that are pretty savvy about UNIX type systems or have at least dealt with building on a Linux/UNIX system before, will have very few problems. The Gumstix folks have a very active developer community that can help you over the bumps. Check out the website and follow the links to get an idea of what you are getting into.

Q. I would like to build a lawnmower robot and have some specific questions about the drive system. First, about my design: I am thinking of using brushless DC motors with planetary or worm gear reducers to drive 12" wheels on each side, balanced by a trailing caster wheel.

You mentioned PID control in the November column and I think I should use that to aid in navigating the optimal cutting path. What sort of battery energy penalty will it incur? What is the preferred motor controller configuration (and do MOSFET drives still rule?)

I've heard rumors about super, ultra-capacitor networks as alternate/adjunct high efficiency (power to weigh) power sources, and would like your take on this as well.

— Harry (via email)

A. That is really a wide-open question, so I'll give it my best shot to answer your specifics and/or point you to places that can.

First off, from my perspective as a long-time lawn mower, mowing lawns is not a precision activity and has a great tendency for overlap. Your lawn really isn't big enough to make GPS a useful solution since GPS has a resolution too low for your application. Sub-meter accuracy will usually only occur when the GPS is stationary for some specified amount of time — which your robot will not be — or augmented with a known correction signal.

You'll probably want to have some series of landmarks, beacons, or even buried wire with a known signal carried on it to mark your boundaries. For heading, I would use a compass. You are outside in clear space which is perfect for compass use.

Let's look at your first part. You must have some big brushless motors to drive a mower around but if properly sized, they should work fine. Large wheels will be what you want when wandering in a grassy area of unknown height. Only experimentation will tell if they are tall enough. In general, you want your wheel radius to be greater than your grass height or you will "high center" and stop moving. I'm also assuming that you want behavior based actions dependent upon grass height so that you can save power by not spinning the blade when the grass doesn't need mowing. My idea for detecting grass height is to use some slotted interruption-based IR detection system. You

can set the detector at the grass height the mower is mowing; if it is lower than that, then the grass won't enter the slot. Since nothing will be detected, then you won't have to spin up the cutting blade. I would use several detectors so that you can detect pretty much anything in front of the mower (as you drive over it that is.)

Now for second part of your questions. PID motor control is a good way to go in general because it varies the applied power to yield a precise control of your drive speed. Without PID, you could set your motor power down low to go slowly, but you might not have enough power delivered to the motors for some given grass height where your mower won't move. PID allows you to precisely set a speed — even a slow one — that your robot will go. However, while PID will allow you to set a speed and actually go that speed, it does not guarantee any kind of turn accuracy or repeatability.

Since you will no doubt be using some kind of rotational sensor on each wheel (or motor), you can get precise turns by figuring your radius from the center of your robot and calculating the differential "tics" that need to be seen at each motor to make that turn. There are advanced ways to do this mathematically, but I would do it empirically by "driving" the robot.

Make several different kinds of turns and see what your encoders give you for each wheel. You will then know exactly what to expect for various kinds of turns. Note that wheel slippage, tire diameter differences, and a variety of real-world imperfections will cause these turns to be somewhat less than perfectly repeatable. This is where I would use a compass. If you make your turn and you are basically using a plow pattern, then you know exactly what heading you should be going in and can actively correct for a bad corner. You may slip such that you aren't in the "aisle" you expect to be however, so I recommend some overlap in your coverage to compensate for it. I would also put an encoder on the trailing caster to allow you to detect when you are stalled. Your wheels may be moving, but your robot may not be. If you put an encoder on the turn angle of the trailing wheel, you can also get an approximate turn angle for your robot. This might be useful to confirm that you are actually cornering.

Now to the last part. We don't know just how heavy your beast is going to be, so I can't really offer confirmation of the power that you will require. You most likely aren't going to be able to specify your motors, so you probably will be using what you have in your junk box to create your mower. However, you can get some idea of the power that you will need if you know your worst-case angle from the horizontal that your robot will need to drive up.

Here is a way that you can figure the power that you will need for your mower to move in its expected environment. There are still unknowns, however, like the coefficient of friction and the amount of incline. The discussion that follows is based upon information in Chapter 2 of my book *Building Robot Drive Trains* (with Michael Owings, published by McGraw-Hill, 2002) which, in turn, is derived from information found in *Mobile Robots*:

Inspiration to Implementation, 1st edition by Joseph L. Jones and Anita M. Flynn, published by A.K. Peters, Ltd., 1993.

When we work with mobile robotics, we must take into account the many forces acting upon our robot. Simplified, those forces are mostly gravity and friction. Gravity is the force caused by the mass of our robot and the attraction of our planet working together to keep our robot in place. Friction is the resistance to movement (not caused by pure inertia) and is influenced by gravity. For our robot to move, we must overcome gravity (in the form of inertia) and also overcome friction (getting our wheels to turn.) These two forces are related, but not the same. How are they related? Look at Figure 2. This diagram describes these two forces and how they affect the power needed to move our robot.

In Figure 2, we want to know how much force we need to apply to make our robot move forward and up some maximum incline (ramp). The force we need is shown in the following formula:

$$F_{app} = F_f + F_w$$

where F_{app} is the required force to move our robot, F_f is the frictional force resisting that movement, and F_w is the weight causing us to roll downhill. This force is the torque required to move the robot and is measured in Newton-meters. The F_w (weight) component is 0 on a flat surface because it can't fall down anymore. Here is the formula for F_w :

$$F_w = mg \sin\theta$$

where mg is the force of gravity, so m is the mass and g is the acceleration due to gravity (9.8 m/s^2) and $\sin\theta$ is the angle of incline from the vertical (given in radians). $\sin(0)$ (straight down) is 0.00, which means that on level ground, this part of the formula doesn't need to be considered.

At any other angle, this adds to the force required to move an object. To prove this to yourself, try pushing your car on level ground — no problem, right? Now try to push it up a hill. It just got harder didn't it? This is F_w at work.

Frictional force is always with us, no matter where we

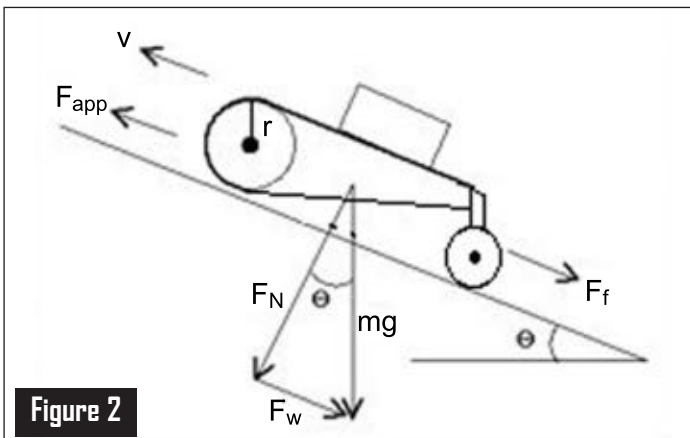


Figure 2

are (flat or hills). Frictional force, or F_f is affected by the size of the angle from the horizontal and by how much friction there is between the moving wheel and the surface on which it is moving, as shown by this formula:

$$F_f = \mu mg \cos\theta$$

where μ is the coefficient of friction (in our case, the *static* coefficient of friction), mg is the force of gravity, and $\cos\theta$ is the angle from the horizontal (again given in radians). A large μ means a large frictional resistance; think of trying to push a brick on a rough surface. A small value of μ is like a greased surface; things slide easily. The coefficient of friction, μ can be found in a number of books. It is usually estimated because so many things can change it.

Now let's make these formulae work for us. To figure out the power we need to move our robot, we have to use one more formula — the formula for power which is:

$$P = F_{app}v$$

Power is force times velocity. We now know what power we need to have to move our robot over the terrain and slopes we predict it will need to navigate. We also need to know what speed our motors need to rotate, so our final formula gets that rotational velocity. It is:

$$\omega = v/r$$

Rotational velocity ω is our velocity divided by the radius of the wheel spinning. This gives us how fast the motors need to turn to get our desired velocity (v).

Remember to keep your units consistent when using these formulae; nothing messes up a project worse than using feet in one place and meters in another! Since you are using two motors, divide your required power by two. They are sharing the load and only need to be half as large as the total required power, however, a little margin built in never hurts. This formula gives us the power needed for our robot measured in watts.

Finally, if at any time F_w is greater than F_f , or gravity is pushing down the slope more than the friction you have available to climb the slope, your robot will be slipping and all the power in the world will not help you. Increasing your traction — which is increasing your μ — will improve your hill climbing capability but it takes more power to move with a higher μ .

The only thing left to do is to figure out if your motors are powerful enough. Surplus motors rarely give us their precise specifications so you'll have to find this out yourself. Gordon McComb of *Robot Builders Bonanza* fame told me once he uses the "heft" method. He looks at his robot, picks up a motor and feels how heavy it is, and judges by that if it will move the robot. (You just need to have a calibrated arm to make that method work.) You can find

the approximate power of your motors by measuring the velocity of the output shaft with a wheel encoder. The output torque can be approximated by putting a 1 cm pulley or disk on your output shaft, and powering the motor at various speeds with known weights attached to a string that is attached to the outside edge of the pulley or wheel. You can use something other than 1 cm and scale the output torque to that. The torque is directly scalable in a 1:1 fashion by that length.

Since you are working with brushless motors, then you will be restricted to using a brushless motor driver that is commercially available unless you already know how to drive such motors. If, however, you consider switching to a brushed-type DC motor then you can probably build your own PWM-based controller. Yes, MOSFET driver bridges are indeed still currently the best drivers that you could use in this application. If you don't want to spend that much effort, you can pretty effectively use a relay/MOSFET DC motor driver which is cheaper and easier to implement as shown in my November column. There are commercially available, high current brushed motor drivers for sale in several places, as well. I don't have the third edition of *Robot Builders Bonanza*, so I don't know what prices are quoted there to compare today's prices to. I think that you should be under 50 amps for your application, so look around (check out <http://www.kellycontroller.com> — Ed.)

And now finally, for your question about supercaps. I've been reading about these and there are a couple of approaches that look promising, but nothing in a useful size that is commercially available for your robot today. Considering your application, I would recommend lead acid or nickel metal hydride (NiMH) as having the best energy density and durability.

That wraps up another Mr. Roboto session. As usual, if you have any questions about things robotic, please drop me a line at roboto@servomagazine.com and I'll be happy to work on it! Until next time, keep on building those robots! **SV**

Experience a Faster Way to Search for Components!

Jameco Electronics' new catalog and enhanced Jameco.com website are two tools that are designed to work together to give electronic professionals faster access to the hottest components in the industry.

Color coded references throughout the catalog assist you in analyzing a wide range of brand choices (from franchise sourced to house brands to factory overruns) offering you more pricing options than you'll see anywhere else.

Web codes allow you to quickly jump from catalog to website to view additional specifications, application notes, photos and more products. You'll find exactly what you're looking for without having to wade through hundreds of thousands of products on a complicated website or wielding a twenty pound catalog.

With a flip of the page or a click of the mouse, you have all the tools you need at your fingertips. Reach for Jameco first and order your catalog today.



Order your
web-enabled
catalog today!

JAMECO
ELECTRONICS

Call 1-800-831-4242 or Visit www.Jameco.com/Speed for the ride of your life!

NEW PRODUCTS

EDUCATIONAL KITS

A New Starter Kit

SchmartBoard has announced the availability of the new "Solder By Numbers(SBN) Starter Kit" — a starter kit for people who are new to electronics. The SBN Starter Kit includes:

- An Elenco 40 watt electronic soldering station
- Assorted sized tips for the soldering iron
- Water soluble solder flux in pen form
- Safety goggles
- A lighted magnifying glass
- Two screwdrivers
- Wire cutters
- Wire strippers
- Needle-nose pliers
- SchmartSolder
- A Solder By Numbers project with SchmartBoard and Components (LED paperweight)

The Starter Kit costs \$99 and its release coincides with the launch of www.solderbynumbers.com.

"It takes two things to build an electronic circuit: the ability to solder the components and the ability to take an electronic schematic and transfer the schematic to a real circuit board," said Neal Greenberg, SchmartBoard VP of Marketing. "SchmartBoard has made these issues easy which opens up electronics to a wider audience than previously possible."

With SchmartBoard prototyping technology, anyone can solder any type of SMT electronic component.

- SchmartSolder makes it possible for anyone to hand solder through-hole components.
- Solder By Numbers tells you where to solder them onto the SchmartBoards to create your circuits.

"Solder By Numbers is to circuit building what 'Paint By Numbers' is to art," said Greenberg. "We plan to have hundreds of SBN circuits on the SolderByNumbers website available soon."

For further information, please contact:

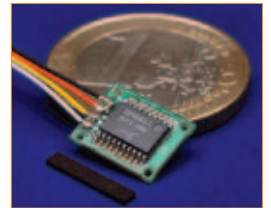
SchmartBoard

Website: www.schmartboard.com/index.asp?page=starter_kit

SENSORS

The TRACKER

New Scale Technologies has announced its TRACKER position sensor: a high-resolution magnetic sensor with integrated, on-chip digital encoder. Its small size and 2 μ m resolution make this magnetic encoder a robust and cost-effective alternative to miniature optical encoders. It can be used as a linear encoder or off-axis rotary encoder.



The TRK-1T02 TRACKER position sensor measures 8.5 x 11.5 x 1.61 mm. Total height with magnet and air gap is approximately 3 mm — up to 50% thinner than comparable optical encoders with glass scale. Unlike optical encoders, the TRACKER does not require a light source and is insensitive to external light, making it well suited for optical imaging applications such as miniature cameras. It has additional advantages of insensitivity to shock, vibration, and high-particulate environments, as well as less stringent mounting alignment requirements.

The on-chip encoder provides direct digital output using standard I²C protocol, eliminating the need for external pulse counters. A microprocessor can query multiple TRACKER position sensors and read position information directly.

With no light source, the TRACKER has lower power draw. Because this incremental encoder features a built-in zero reference, power consumption can be further reduced by saving position information to Flash memory and turning off the power to the encoder when the system is not moving; the encoder does not need to be reset when power is restored. Automatic gain control and offset compensation make it less sensitive to external magnetic fields. The TRACKER's small size makes it ideal for creating tiny, integrated closed-loop motion systems using micro motors such as New Scale's SQUIGGLE motor.

The TRK-1T02-E TRACKER evaluation pack includes a TRK-1T02 position sensor, linear magnetic strip, MC-31MB interface card and software to facilitate evaluation. The USB interface returns direct position information to a PC.

For further information, please contact:

New Scale Technologies

Website: www.newscaletech.com/Tracker_overview.html

EVENTS CALENDAR



Send updates, new listings, corrections, complaints, and suggestions to: steve@ncc.com or FAX 972-404-0269

Know of any robot competitions I've missed? Is your local school or robot group planning a contest? Send an email to steve@ncc.com and tell me about it. Be sure to include the date and location of your contest. If you have a website with contest info, send along the URL as well, so we can tell everyone else about it.

For last-minute updates and changes, you can always find the most recent version of the Robot Competition FAQ at Robots.net: <http://robots.net/rcfaq.html>

— R. Steven Rainwater

January

20-22 Singapore Robotic Games

Singapore Science Center, Republic of Singapore

The Singapore Robotic Games is another big multi-event competition. Some of the events planned are autonomous Sumo, RC Sumo, robot soccer, wall climbing, pole balancing, underwater robots, legged robot marathon, legged robot obstacle race, and others.

<http://guppy.mpe.nus.edu.sg/srg/>

21-24 Kurukshetra

Guindy, Chennai, India

In addition to traditional events like line following, the competition includes events with imaginative titles such as "The Road Not Taken."

www.kurukshetra.org.in/

24-26 Techfest

Indian Institute of Technology, Bombay, India

Autonomous bots compete in Micromouse and three other events called Pixel, Full Throttle: Grand Prix, and Blitzkrieg.

www.techfest.org/

29 Robotix

IIT Khargpur, West Bengal, India

Now at nine years old, this is supposed to be the largest robot competition in India. Expect to see over 1,000 teams competing and over 2,500 spectators this year.

www.robotix.in/

February

12-15 Pragyan

National Institute of Technology, Trichy, India

Another Micromouse competition. Don't miss it!

www.pragyan.org/

15-19 APEC Micromouse Contest

Washington, DC

Micromouse maze running. Cash prizes and trophies.

<http://www.apec-conf.org/>

March

13-14 AMD Jerry Sanders Creative Design Contest

University of Illinois at Urbana-Champaign, IL

JSDC is a college design contest centered around building a robot to compete in a two day engineering challenge.

<http://dc.cen.uiuc.edu/>

21 RobotChallenge

Vienna, Austria

Parallel Slalom, Slalom Enhanced, Standard Sumo, Mini Sumo, Micro Sumo. Autonomous.

www.robotchallenge.at/

21-22 Manitoba Robot Games (9th Annual)

Tec Voc High School, Winnipeg, Manitoba, Canada

Mini-Sumo Japanese style; Sumo Western style; Mini-Tractor Pull: robot pulls weighted sled in minimal time; Super Scramble: walk, wheel, or track, first over mixed terrain; Line Follower: four layouts, flat and inclined; Robo-Critters: contest for kids with supplied robot-kit. Autonomous and Remote-Control.

www.scmb.mb.ca/

21-22 Roboticon

University of Guelph, Ontario, Canada

Robotic Soccer for LEGO robots. Autonomous.

www.collegeroyal.uoguelph.ca/

COMBAT ZONE

Featured This Month:

Features

20 *PARTS IS PARTS:
A123 Battery Packs*
by Donald (DongHyun) Sung

22 *MANUFACTURING:
Moving on With Your
Robot* by John Frizell

Events

24 *Oct/Nov 2008 Results and
Jan/Feb 2008 Upcoming
Events*

24 *EVENT REPORT:
Roaming Robots Winter Tour*
by John Frizell

26 *EVENT REPORT:
Combots Texas Cup at
Maker Faire Austin*
by Wendy Maxham, Team PlumbCrazy

ROBOT PROFILE – Top Ranked Robot This Month:

28 *Devil's Plunger* by Kevin Berry

PARTS IS PARTS A123 Battery Packs

● by Donald (DongHyun) Sung

Batteries are one of — if not the most — important parts of a combat robot. They are what run the robots and bring them to life. Besides their importance in providing power, they are also one of the most dangerous parts of a robot. For this reason, it is important to have high quality batteries. Their quality also determines their available current output, as well as their life. Caring for batteries is crucial because of their potential danger if not maintained properly.

There are three commonly used battery types: Nickel Metal Hydride (NiMH), Nickel Cadmium (NiCd), and Lithium Polymer (LiPo). NiMH and NiCd batteries have been used in robots for a long time, and are still frequently used in the larger weight classes. As LiPo has become more affordable, their popularity has grown

in insect-weight classes for their light weight but high power output. For the same reasons, they are being used by a growing number of sub-light and heavier robots. Although LiPo batteries can produce a lot of power in a lightweight package, it does come with some drawbacks. All batteries may be dangerous if handled improperly, but LiPo batteries are regarded as one of the more dangerous types because of their tendency to burst into flames if improperly charged or handled.

In addition to the three types of batteries mentioned previously, the Lithium Iron Phosphate (LiFePO₄) batteries are becoming popular in many R/C hobbies including combat robots. These are commonly known as M1 or A123 batteries. A123s and LiPo batteries both fall under the Lithium-Ion category and are

comparable in their light weight and high power output, but they're also different in many ways. First, the A123s come in a can similar to that of NiMH and NiCd batteries as opposed to the soft rectangular packaging of the LiPos, which allows them to be made into customized packs. Although they are in cans, they are slightly larger than the standard sub-C size at approximately 1" diameter and 2.6" long.

Second, the can shell of the A123 batteries provides better durability and is claimed to be much safer than LiPos. Third, the A123s come in a 2,300 mAh capacity rating and have a 3.3V nominal instead of the 3.7V of LiPos which means different chargers or charging adapters (www.sikelectronics.com/lipodapter/index.htm) are necessary. Lastly, A123 batteries are more affordable than LiPo batteries.

When I designed my hobby-weight robot, Torrent, I made the decision to experiment with this new A123 technology. I chose to use A123s because the BotsIQ event no longer allowed the use of LiPo batteries. I decided against NiMH and NiCd batteries because they were simply too heavy and large. Also, the capacity rating of NiMH or NiCd packs would need to be larger to match the current output of the A123s which have 30C (30 times the cell Ah rating) continuous and 50C burst output ratings. Due to their advantages in power output, light weight, and affordability, I quickly decided to go with A123s.

As for obtaining A123 cells, they range from about \$14 to \$20 per cell, depending on the vendor and whether you get individual cells or packs. If you are not looking to use all 10 cells that come in a cordless DeWalt pack, a recommended vendor is BattlePacks (www.battlepacks.com). They have loose cells for sale if you are looking to do your own soldering, but for those who are not as willing, they have the option for custom packs at an additional fee.

After researching online, I found that the cheapest source of A123s was eBay. I

bought a DC9360 DeWalt battery pack which contained 10 A123 cells. After I received the pack, I followed the tutorial at www.sikelectronics.com/DeWalt/index.htm to make five two-cell packs with leads for Deans connectors. Making the packs was easy because I kept the stock tabs on the batteries, so all that needed to be done was soldering on the wire for the Deans Ultra connectors and balancing tabs.

When charging these batteries, I did not pay attention to balancing because from my experience, there was no significant difference in performance. Balancing the A123 cells is a frequently debated topic. There are those who argue that balancing is necessary for getting the top performance and extending the life of these cells, and others argue that their durability makes balancing unnecessary and that individual cell voltages don't vary much in packs.

At competitions, I was able to see these packs prove their durability and high current output. In my robot, Torrent, I used an Axi2826 motor to spin a 3.5 lb weapon and the spin-up was always quick. During matches, these packs would easily last three minutes through rigorous driving and numerous weapon spin-ups. Even during the five minute rumbles, I could tell that the weapon and drive were all working near full power throughout the entire duration. None of the packs failed during the four competitions I have attended so far, and as I previously mentioned, the fast charging ability of these A123 batteries always ensured that I entered the match with freshly peaked batteries.

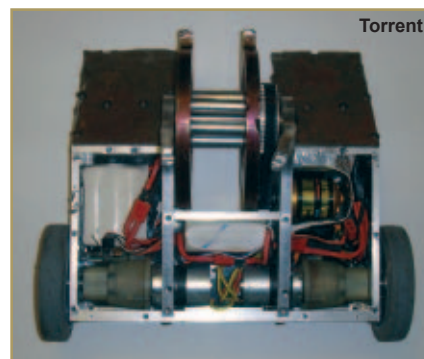
After using these batteries, I



concluded that I prefer the A123s over LiPo batteries, although in the near future I hope to see LiFePO4 batteries come in various sizes. Overall, these batteries provide more flexibility in designing a robot, as well as provide more than enough power for three or five minute matches for my hobbyweight. Also, I can leave them at home feeling a little safer about potential fire hazards.

The only downside to these cells is the fact that they only come in one size, so they are not ideal for use in antweights and beetleweights simply because they are too big. They are perfect for all weight classes from hobbyweights to middleweights, however, and possibly up to the super heavyweights. For cell data, check out www.battlepack.com/ANR26650M1.pdf. **SV**

Donald Sung likes to build combat robots, play Connect 4, eat Starbursts, and read car tuner magazines. He is in the process of building a new version of Torrent.



MANUFACTURING: Moving On With Your Robot

● by John Frizell

So, you've got yourself some battery powered drills and have started to convert them into gearmotors to drive a robot. (There are lots of good articles about how to do this on the web, as well as one in *SERVO* November '06.) But what do you do next? How do you mount it in your bot, attach a wheel, and power it up?

The tough plastic case of a cordless drill is designed to hold its motor and gearbox rigid as you drill through things so it makes a good mount. And it's free. Photo 1 shows a 9.6 volt cordless drill mounted to a piece of plywood with three pieces of 4 mm all-thread rod. (All-thread rod was used because the 4 mm bolts available weren't long enough.)

This sort of mounting wastes a

bit of volume around the motor but doesn't make the unit any longer. The rods are arranged in a triangle outside the body of the gearmotor. You could put two short bolts under the gearmotor through the bottom of the plastic case, and the third to one side — it works but it won't be as rigid.

You also could mount the case directly onto the base of your robot but having the unit on a plywood plate makes it easier to get the gearmotor in and out without having to take the case apart every time. If you haven't already taken off the adjustable torque mechanism, don't bother. Just turn it to its highest setting and leave it. If it slips in combat, it will probably be protecting your motors from burning out or overloading your speed controller.

The shaft that once held the chuck has a 3/8" UNF thread. The "bolt people" are gradually standardizing on one thread — metric — but this has a long way to go and hasn't reached the cordless drill industry yet. The best way to deal with this is to buy a pair of wheels that screw

directly on to the thread. You can try Team Rolling Thunder www.teamrollingthunder.com/Kitbots/Parts/body_parts.html or email jamesncooper@btconnect.com in the UK (Photo 2).

Big Wheels Will Make For A Faster Bot; Small Wheels Will Give It More Pushing Power

If you want to make wheels, the easiest way is to buy cheap plastic ones and tap the existing axle hole for the 3/8" thread. Clamp the wheel in a vise and go through it carefully with an 8.5 mm drill. Try drilling half way from each side — if you don't have a drill press, you will be amazed at how much the drill can wander. Then, wind your 3/8" UNF tap through (see Photo 3), remembering to make half a turn backwards for each one forward.

Cheap wheels are not designed for being driven and so the rim will often separate from the body of the wheel. Put a screw through to stop that from happening. Put lots of Loctite on the threads and screw the wheel into place. Make at least

one spare one — these wheels don't tend to last too long at events as Photo 4 illustrates.

For a more durable wheel, buy a 3/8" UNF bolt and some nuts at an auto supply store. Cut some steel strap (30 mm by 4 mm is good) to a length just a bit shorter than the diameter of



PHOTO 1. A 9.6 volt gearmotor ready to take a wheel.

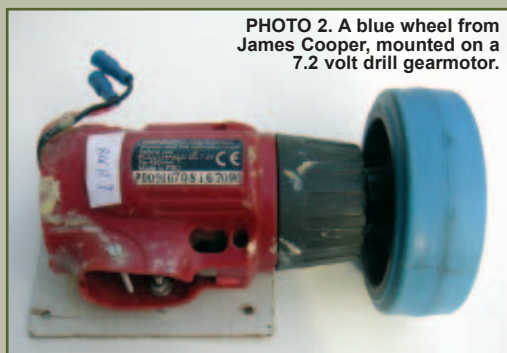


PHOTO 2. A blue wheel from James Cooper, mounted on a 7.2 volt drill gearmotor.



PHOTO 3. Tapping a plastic wheel.



PHOTO 4. A plastic wheel after six fights.



PHOTO 5. Steel strap drilled to take nut and fixing bolts.



PHOTO 6. Ready for welding. The outer nut gets welded on.

your wheel. Find the center line of your strap and drill a 10 mm hole in the center and a 5 mm hole a inch or so from each end (Photo 5). Put a nut on your 3/8" bolt, slip the bolt through the 10 mm hole, and screw on a second nut to hold it all in place (Photo 6).

If you can weld or braze, then go ahead and weld one nut on. If not, pay someone who can. A hint: If you show up at a machine shop or a garage with loose bits of metal to weld, they may not be interested. You will do better if you show up with a tightly assembled piece which makes life easy for them.

When the piece has cooled, unscrew the unwelded nut (you did remind them to only weld one, didn't you?) and the bolt, leaving behind the strap with the nut welded in place (Photo 7). Cut a piece of plywood or plastic to the desired wheel diameter, cut a strip of old bicycle tire and fasten it to the rim of your wheel; then center the nut on the wheel, drill a shallow hole in the wheel to accommodate the end of the gearmotor shaft if necessary, drill through the 5 mm holes and through the wheel, and bolt it all

together. An assembled wheel is shown in Photo 8.

I'd Just Buy Wheels If I Were You

Whatever wheels you use, screw the left hand thread machine screw that held on the chuck back into its hole in the gearmotor shaft with a split washer to hold the wheel on. If it comes off, then apply Loctite to the gearmotor shaft and tighten it all up again.

It's time for a test. Charge one of the batteries that came with the drills. Put connectors on the motor wires and glue the wires to the plastic drill casing so that a pull on the wire won't tear it off the motor. Fasten the units down to a scrap of plywood. Connect the two motors in parallel (join red from one motor and black from the other to positive; same to negative) and provide a generous length of wire to attach your test harness to the battery.

Photo 9 shows what it should look like. Connect the battery and your proto robot will take off across the floor. If it goes backwards, reverse the battery polarity. If it

spins in circles, join the red from each motor to positive instead of red and black.

Run Robot, Run

Once you have your assembly racing across the floor, you will know you have a working drive train for your robot.

Your motors will easily take 1.5 times their rated voltage — probably double — and this really improves performance. Speed goes up in proportion to the overvoltage and power goes up as its square. So, a 12 volt drill running at 24 volts goes twice as fast, draws twice as much current, and develops four times as much power as it did at 12 volts.

The batteries that came with the drill will get you going and you can put them in series for a higher voltage, but cheap cordless drill batteries are not very good quality and at some point you will have to move to better batteries. Have a look at what is available and leave room in your bot for them so that when you do move on from the drill batteries, you will have space to mount the new ones. **SV**



PHOTO 7. A completed attachment strap, ready to be bolted to the wheel.



PHOTO 8. A plywood wheel with bike tire rim.

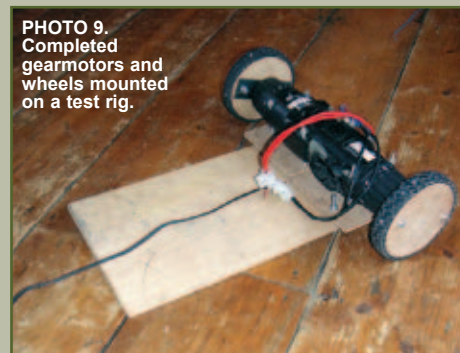


PHOTO 9. Completed gearmotors and wheels mounted on a test rig.

EVENTS

Results and Upcoming Events for October 2008 to February 2009

Results for October 13 to November 9

Mecha-Mayhem 2008 was presented by the Chicago Robotic Combat Association in Rosemont, IL on October 16th. Twenty-two bots were registered.



The ComBots Maker Cup was presented by ComBots in Austin, TX on October 18th-19th. Twenty-one bots attended,



plus a bunch of people.

2008 Halloween Robot Terror was presented by California Insect Bots in Gilroy, CA, on October 25th. Eleven bots were registered.



Nottingham 2008 was presented by Roaming Robots in Nottingham, England, on October 25th.

DRG FCC Raptors & Feathers was presented by Dutch Robot Games in Leeuwarden,



Netherlands, November 6th-9th.

Upcoming Events for January to February

Destination Devastation — a Robo Wars event — will be presented by the Queensland Robotics Sports Club in Oakleigh, Melbourne, Australia on January 19th-20th.



Motorama 2009 will be presented by North East Robotics Club in Harrisburg, PA, on February 20th. **SV**



EVENT REPORT: Roaming Robots Winter Tour

● by John Frizell

The Roaming Robots winter tour made a late but exuberant start in Portsmouth. As usual, there were three weekend events over October and November, and three cities with each well advertised event attracting a paying crowd for two or three shows. Production values were high with lots of sound and light effects.

This weekend saw the introduction of a new arena with three sides fenced in with metal mesh extending up 1.5 meters from the kickplate to stop the easy flips out of the arena which have brought a quick end to too many fights. The fourth side has the usual low wall but instead of a drop,

there is a floor at the arena floor level and ejected robots can drive back in through a gate if they can make it within a 30 second time limit. The gate stayed open for the first show because there wasn't time to get it working.

The arena assembly started at 8 am that morning, but the new

build was a challenge even for the experienced crew of Roaming Robots. By showtime at 5 pm it was still not complete. A long, long line of parents and kids snaked back through the halls of the Mountbatten center, almost to the front door. By 5:30, however, they were taking their seats and by 5:45 the show was underway. It did not disappoint.

Whiteboard fights — which are just for fun — feature three or four heavyweights armed with flippers or the occasional axe. Anyone who wants to take part just chalks their robot name on a board at transmitter control. First come, first served. Featherweight whiteboards consist of as many machines as can fit in the arena — usually 10 to 15. Feathers tended to be mostly pushers with a few CO₂ powered flippers. (Spinners are rare in the UK because an arena big enough to host a decent show and strong enough to safely contain them is too heavy to transport and erect for a weekend event.)

The Saturday show was all whiteboard but, despite the competition due to start the next day, fights were at full intensity and the gymnasium was filled with crashing and the cheers of a delighted crowd. The pits — which were in darkness because of the lighting effects — began to develop little pools of light as roboters worked by desk lamp or electric torch to do running repairs to their machines. Almost all the 19 heavyweights were old Robot Wars veterans, dating back to the end of the series in 2003 or before — very few new heavies have been built since. But old machines continue to be bought up (often on eBay), reconditioned, and put back into service. At least

Skink, a powerful featherweight flipper, being prepared for battle.



Getting some repairs done in a break, when the lights are on.



four of the heavies were old Robot Wars machines being operated by new owners and a recent purchase was seen in the pits although it was not yet ready to fight. Almost all the feathers have been built more recently, and there is a new wave of building underway to take advantage of the increase in weight limit from 12 kg to 13.6 kg to bring UK feathers in line with international standards.

The competition started at noon the next day with 13 heavyweights and 21 feathers taking part, all trying to accumulate enough points to make it through to the finals in two month's time. Heavies fought one on one and received one point for entering the arena, two points for winning by a judge's decision, or three points for winning by a knockout. Featherweights flooded the arena with 10 or more in each competition fight. The top four received points with four for the best, three for the runner-up, and so on.

Judging a fast moving swarm of robots wasn't easy but it was helped by the number that wound

up down the pit, propped up against a wall, or simply stopped working — there were only four or five left at the end of a five minute bout.

Notable among the heavies were Maelstrom, a double-sided flipper which could flip when inverted; Thor, a powerful axe; Hydra, which had both a flipper and an axe; and Puck, a truck tire surrounding a powerful drive unit which was invertible and almost indestructible, but had no weapon at all.

Feathers included the powerful flippers Mini-Mighty Mouse and Skink, and Pain in the Asp, a 12 kg six-wheeled machine with two horsepower of drive.

Seventeen out of the 19 heavies had CO₂ powered weapons and there was a constant stream of roboters carrying frost encrusted CO₂ bottles heading outside to the refilling station — a cylinder and a set of scales in the back of a truck.

Between the competition fights, there was still time for a few whiteboards for those who were just there for fun, those who had been knocked out of the competition,

Repairing a speed controller.



The new arena, between shows.





Kan Opener getting ready for a competition fight.



Closing up the machine before a fight.



Maelstrom and Hydra pose for the crowd.

and a few competitors who wanted more fights.

Also in the arena was a house robot, Major Damage, a 960 kg monster running on a tracked chassis from a miniature JCB digger and armed with a hydraulic claw and hammer. Nobody messed with it. It was well behaved, occasionally using its claw to haul robots out of the pit, as well as performing its

usual function of separating machines that had jammed together.

The last fight of the weekend saw five heavies going after each other in the arena, amid clouds of CO₂ and the bright lingering sparks characteristic of high speed collisions between 100 kg machines with steel or titanium armor. As roboteers loaded their machines into cars and trailers, some were already

discussing plans for a trip to Nottingham later in the month to try and accumulate more points to get them into the finals in Maidstone at the end of the November. About a dozen heavies and the same number of feathers will make it and one of each will emerge as winners. **SV**

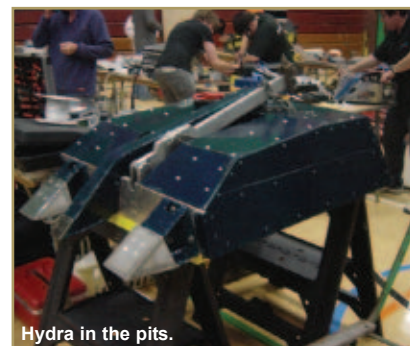
Go to www.roamingrobots.co.uk for more details.



Team Batter's machine after a bit of a battering.



Ian and Weird Alice.



Hydra in the pits.

EVENT REPORT: Comb^ots Texas Cup at Maker Faire Austin

● by Wendy Maxham, Team PlumbCrazy

Saturday October 18, 2008, 10:30 am and a 10 year old boy leans over the railing near the pits. "When does the fighting start?" he yells down to me. "Noon," I shout

back, "but you'll probably want to get here a little earlier because the good seats should go fast."

And go they did! The best seats (the bleachers at the front of the

arena) filled up about a half hour before each scheduled show time. People stood two to three deep on the sides of the arena

Texas Cup trophy awarded to Pipe Wenck.





Mitch and FW champion Pyromancer.



Texas Heat lights up LW champion The Big B.



Appreciative crowd at Maker Faire.
Photo courtesy of RoboGames.

as hundreds gathered to watch the mechanical mayhem. The Combots Texas Cup competition shared an indoor hockey/rodeo arena with First Robotics, ArcAttack (a group that makes music with signing Tesla Coils), and a Science in the Movies demonstration. Around the upper levels of the arena were other robotic projects, including several working R2D2 units.

From a builder's standpoint, the event was fairly low key. With 21 robots in five weight classes fighting round robin format, we knew who we were going to fight and which show we'd be fighting in. And with two hours between each show, there was lots of time to make repairs between fights and even an opportunity to wander around the Maker Faire a bit.

For the audience, it was robot combat at its best. As Anton Olsen put it in his Geekdad blog "what is most likely my favorite event at the Faire, the RoboGames, a.k.a., Combots. Most of the matches were spectacular, especially the super heavyweights. Lots of bent and broken metal, shattered lexan, and smoking speed controllers make for great entertainment."

The variety of robots was remarkable with flame throwers, hammers, flippers, lifters, rammers, and spinners throwing sparks and other robots all over the arena. Fighting was fierce between the

competitors, but as always the camaraderie in the pits is what makes this sport so great. And event organizer Dave Calkins threw something new into the mix — trading cards! The kids in the crowd went wild for them — treating the builders like rock stars by asking us for our autographs!

It wasn't all about fighting and destruction, however. Before the competition each day, Dave Calkins presented a "How to Build a Combat Robot" session. He went over the basics of what it takes to build a combat robot, then opened it up for questions from the audience. Several builders joined the Q&A sessions, answering audience questions like "How much does it cost to build a combat robot?"; "Can I put a gun on my robot?"; "How big can I make my robot?"; and "Can I have a flying robot?"

Saturday night the flame throwing robots demonstrated their heat by toasting marshmallows for the audience.

Sunday afternoon featured an unscheduled event as a domed tent from the camping area took flight and crashed into power lines, knocking out power for half of the Faire. The Combots crew quickly rerouted power to get the arena lights working, and fighting continued in the semi-darkness. Louder members of the Combots crew kept the audience updated on

who was fighting and who won each match during the outage.

Even with the minor power mishap, the Maker Faire was an excellent venue for the Combots Texas Cup. With unofficial attendance estimates somewhere between 20,000–40,000, there were plenty of appreciative folks of all ages to enjoy the event.

After seven shows featuring 33 fights over the two day Faire, Texas Cup trophies were awarded to the following robots (win-loss record):

30-pound

Pyromancer, Killerbotics (1-0)

60-pound

The Big B, Late Night Racing (4-0)

120-pound

Pipe Wench, Team PlumbCrazy (3-0)

220-pound

Gerald, Team Moon (4-0)

340-pound

The Judge, Mechanicus (3-1) **SV**

LINKS

- www.TPCbots.com
- www.Combots.net
- www.makerfaire.com
- <http://blog.wired.com/geekdad/2008/10/maker-faire-a-1.html>



HW champion Gerald gives Original Sin an uppercut.



SHW champion The Judge finds Vladiator guilty.



MW champion Pipe Wench helps Ice Cube take flight.

ROBOT PROFILE

TOP RANKED ROBOT THIS MONTH

● by Kevin Berry

Top Ranked Combat Bots					
History Score			Ranking		
Weight Class	Bot	Win/Loss	Weight Class	Bot	Win/Loss
150 grams	VD	26/7	150 gram s	Micro Drive	7/2
1 pound	Dark Pounder	44/5	1 pound	Dark Pounder	13/1
1 kg	Roadbug	27/10	1 kg	Roadbug	7/4
3 pounds	3pd	48/21	3 pounds	Yeti	8/0
6 pounds	G.I.R.	17/2	6 pounds	G.I.R.	8/0
12 pounds	Solaris	42/12	12 pounds	Tourinho	13/1
15 pounds	Humdinger 2	29/2	15 pounds	Humdinger 2	29/2
30 pounds	Helios	31/6	30 pounds	Pyromancer	6/0
30 (sport)	Bounty Hunter	9/1	30 (sport)	Upheaval	8/4
60 pounds	Wedge of Doom	43/5	60 pounds	K2	14/2
120 pounds	Devil's Plunger	53/15	120 pounds	Touro	23/3
220 pounds	BioHazard	35/5	220 pounds	Original Sin	15/6
340 pounds	SHOVELHEAD	39/15	340 pounds	Ziggy	8/2
390 pounds	MidEvil	28/9	390 pounds	MidEvil	3/0
History Score is calculated by performance at all events known to BotRank			Current Ranking is calculated by performance at all known events, using data from the last 18 months		
Rankings as of November 8, 2008					

Devil's Plunger – Currently Enjoying Retirement



Historical Ranking: #1 (Note: Last Middleweight fight November 2005)
Class: 120 lb Middleweight
Team: Team PlumbCrazy
Builder(s): Matt & Wendy Maxham
Location: Sacramento, CA

BotRank Data	Total Fights	Wins	Losses
Lifetime History Events	72	55	17

Devil's Plunger has competed at Steel Conflict 2, Steel Conflict 4, Triangle Series Nationals, RobOlympics 2004, Southwest Division Championships, NPC Charity Open, 2004 RFL Nationals, WBX Premier, RoboGames 2005, Battle Beach 3, Robotic Revolution, 2005 RFL Nationals, RoboGames 2006, Game Developers Conference, and the 2005 February Fun Fest. Details are:

- **Overall configuration:** Invertible floppy-nosed wedge/rammer.
- **Frame:** 3" x 4" aluminum tubing with 3/16" wall.
- **Drive:** S28-400 Magmotor on Apex Dynamics AB60 gear box.
- **Wheels:** Four wheel drive

w/Carefree tires on custom aluminum hubs.

- **Drive ESC:** IFI Thor 883 (two per drive motor).
- **Drive batteries:** Four 3,000 mAh nicad Battlepacks.
- **Weapon type:** 120 lb splitting maul — "The robot IS the weapon."
- **Weapon power:** n/a.
- **Weapon motor:** n/a.
- **Weapon controller:** n/a.
- **Armor:** 0.065" titanium top and bottom; 0.138" titanium wedge.
- **Radio system:** Spectrum DX6

with BR6000 receiver.

- **Future plans:** Rebuild stronger, faster, and with flames.
- **Design philosophy:** Take the HW drive train and put it into a middleweight for a powerful, reliable robot that can ram and slam the opponen.
- **Builder's bragging opportunity:** Devil's Plunger is the only MW to knock out the Combots Champion HW robot Last Rites when it fought as an HW at RoboGames 2006. **SV**

Photo and information are courtesy of Daniel Longmire. All fight statistics are courtesy of BotRank (www.botranks.com) as of November 8, 2008. Event attendance data is courtesy of BotRank and The Builder's Database (www.buildersdb.com) as of November 8, 2008.



RobotShop

is mutating



BUILD THE ULTIMATE ROBOT

Part 4: Firebot

by Michael Simpson

In the previous articles in this series, we worked on both the Firebot and the Megabot. In this article, we are going to finish up the Firebot shown in Figure 1. Before I get into some of the construction techniques, I want to go over some of the components we will be using in this portion of the project.

Take a look at Schematic 1. While I will make every attempt to describe the actual connections in the Firebot, you may find it helpful to use the schematic for reference.

The basic building blocks consist of the following:

- Ryobi 18V Li-Ion battery pack
- Kronos Robotics DiosPro and Workboard
- Vex radio
- CUWIN3500 CE-based controller
- Firebot base (assembly completed in previous articles)

I will be going into detail on each one of the components as we assemble our Firebot. Let's start by

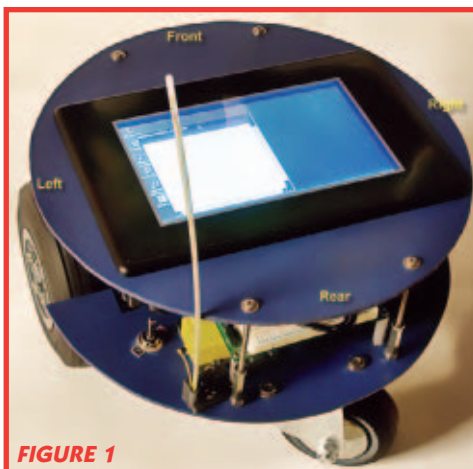


FIGURE 1

looking at the power source.

FIREBOT POWER SOURCE

I looked at many different battery options when I started this project. One thing I decided early on was that I would be using some sort of Li-Ion battery. Li-Ion batteries give you the best power-to-weight ratio of any battery technology that is readily available to the public. For the Firebot, I needed a 12V-18V battery, but it had to be small enough to be contained within the base structure.

Last month when I was talking about tools, I placed great emphasis on the portable Ryobi One+ system shown in Figure 2. I chose this system not only for its very reasonably priced tools, but also for its very efficient power system. After some extensive testing, the 18V Li-Ion batteries have proven themselves as an excellent power source for robot projects.

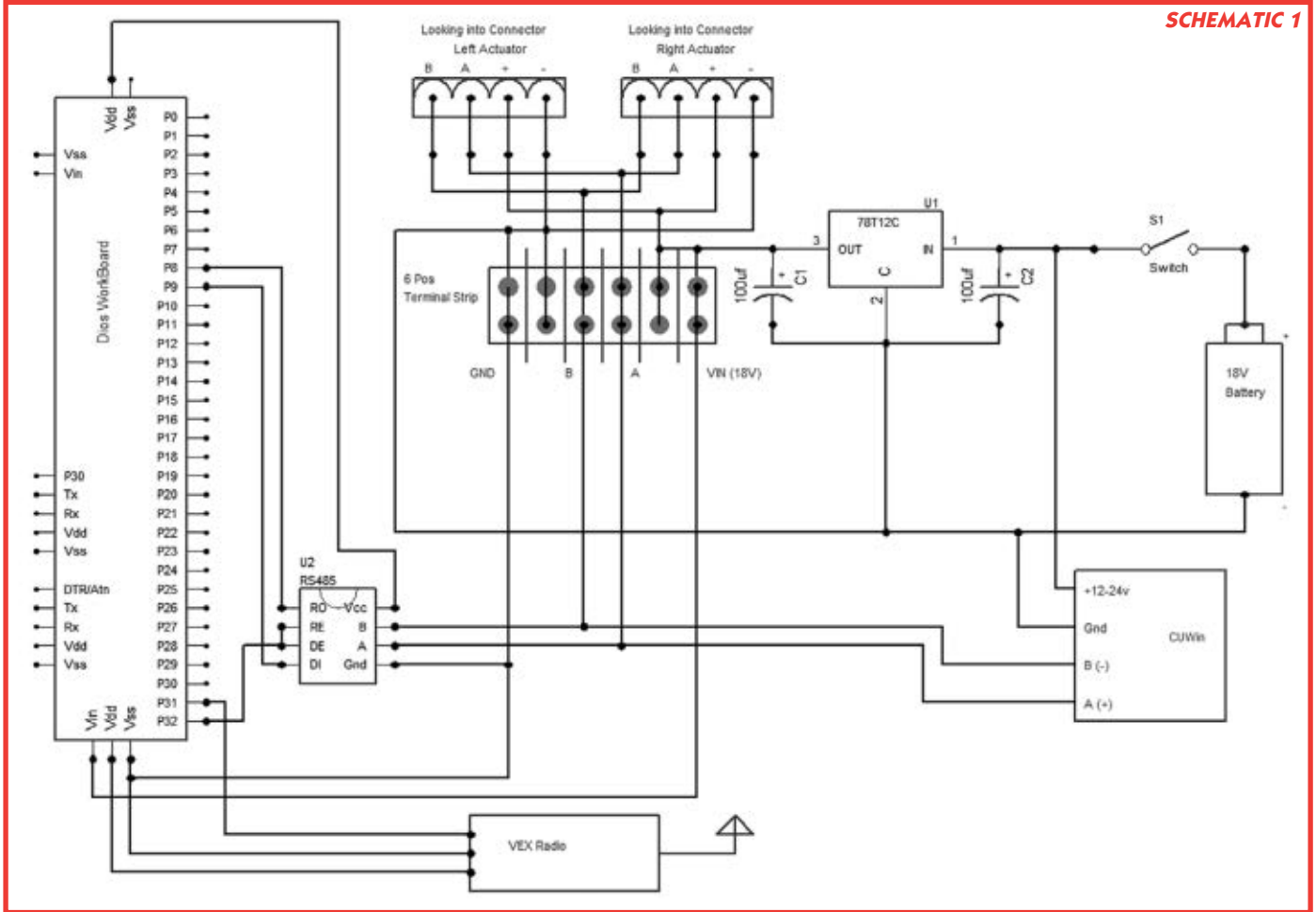
The Ryobi 18V Li-Ion battery has a range of 16-20.5 volts and has a built-in battery tester. For the larger RX-64 this isn't a problem, but it is a little overkill for the smaller RX-28 I am using for the Firebot. Looking at Schematic 1, you can see that I have added a simple 12V regulator circuit to drop the voltage down to more acceptable levels. In many of my early tests, I did not use this regulator and while the RX-28s did not go up in smoke, they did get warm with normal use. For a faster Firebot, leave off the regulator but do so at your own risk. If you do decide to use a regulator, I recommend the Jameco #871658. They are rated at 3A continuous and 5A peak. You will also need a TO-220 heatsink, as they get very warm.



FIGURE 2



FIGURE 3



The greatest challenge in using the Ryobi battery is mounting it. The goal is to mount the battery so that it can be easily removed for charging or replacement. Fortunately, Ryobi has solved the mounting problem for us. They sell a very inexpensive flashlight shown in Figure 3. This little gem cost me \$12.95 and worked out perfectly.

Remove the six screws, pull the flashlight apart, and remove all the components. Just above the two center screw holes is a seam. This is where I cut the light in half. I used a band saw but just about any hand or hack saw would work, as well. When finished, you should have two halves as shown in Figure 4.

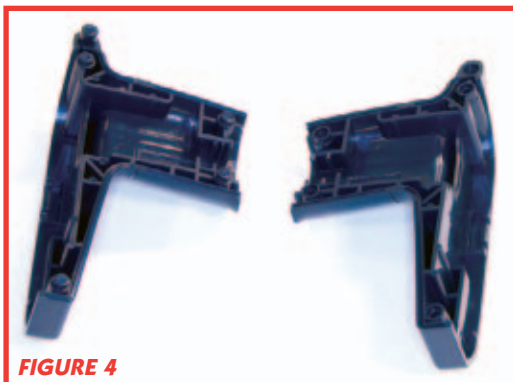


FIGURE 4

Cut off the wires about an inch or so above the battery connector and insert it back into one of the halves. Then using five screws, reassemble the lower portion of the flashlight as shown in Figure 5. It's important that the red wire be located on the right side as shown so that it mates up with the positive terminal on the battery.

I then used the flashlight base to mark the Firebot base. I removed the wheels and then using a jigsaw, cut out the mark. In order to hold the base in position, you need to drill a small 5/64" hole in the base as shown in Figure 6. These will work as small pilot holes to attach some #4 machine screws. You will need about six holes around the base: two



FIGURE 5



FIGURE 6

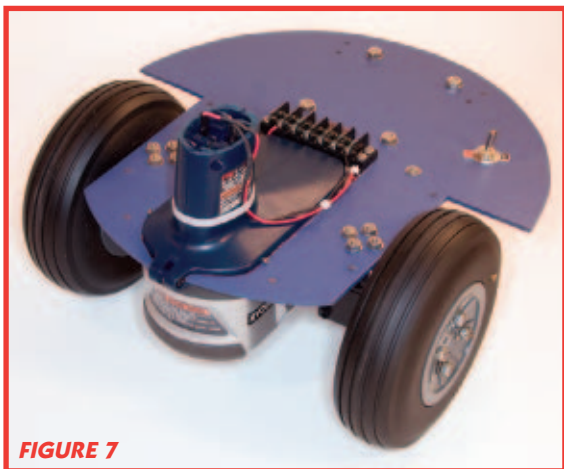


FIGURE 7



FIGURE 8

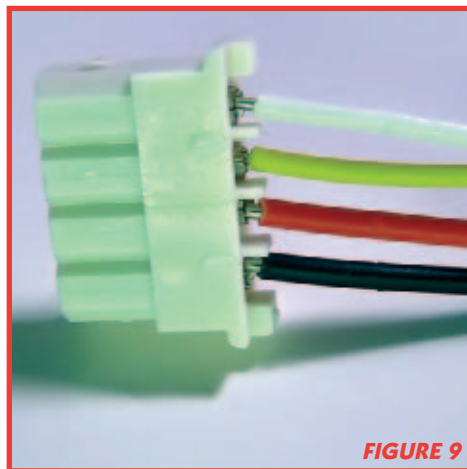


FIGURE 9

in the front and two on each side. Insert these six screws, then slip the base into place. Now you can mark the holes for six more on the underside. With the battery holder in place, insert the lower set of screws. The battery holder should now be held firmly in place and you can insert the battery as a test. A bead of hot glue on the seam will make the battery holder permanent, but I would hold off until you are in the very final steps of your project.

If you look at Schematic 1, you should notice that I use a central, six position barrier strip. I also wire the battery holder up to a switch. When completed, it should look like Figure 7. With the battery mounted on the underside of the Firebot, we free up some space. We also lower the center of gravity which makes the robot more stable.

At this point, you should use a voltmeter and verify that you have power on the two outer positions on the barrier strip when the switch is in the ON position.

!! Important !! This 18V battery has quite a bit of power. If shorted out, it will melt the wires and could start a fire. While the battery itself has protective circuitry, I would not trust it in the event of a direct short.

ADD THE 12 VOLT REGULATOR

As I mentioned previously, I will be adding a 12V regulator to drop the voltage from the 18V battery to safer levels for the RX-28 actuators. Looking at Schematic 1 again, you will see the 12V regulator is simply inserted in between the switch and the barrier strip positive power terminal. The negative side of the battery is connected to

both the regulator and the barrier strip. Place a 100 μ F capacitor on both the input and output side of the regulator. In my case, I modified a Kronos Robotics 5V regulator and used a couple of standoffs to mount it on my base as shown in Figure 8. Another option would be to use a three-position barrier strip to mount the regulator.

CONNECTING THE RX-28 ACTUATORS

When it comes to connecting the RS-28 actuators, the common problem is that the included connectors are not long enough for most applications. Dynamixel knows this, as they included some connector components with each actuator. You will need to make a couple connectors in order to tie the actuators to the center barrier strip.

Using the provided terminals, attach black, red, yellow, and white wires and insert them into the plug as shown in Figure 9. You can use a crimping tool or solder them to the terminals. Brief instructions are included in the RS-28 manual. I started with 12" of wire and cut to actual length, as needed.

Plug the wire into the RX-28 actuator as shown in Figure 10. As you can see, I drilled a hole and used a small wire clamp to hold the wires close to the base. This will help keep the wires from snagging on items.

Run the wires up underneath the barrier strip on the side nearest the battery holder. Drill some holes as needed. Twist the like colors together and connect them to the strip as shown in Figure 11. You will also need to connect a jumper between the red actuator terminal and the outside red power terminal. Do the same with the black actuator

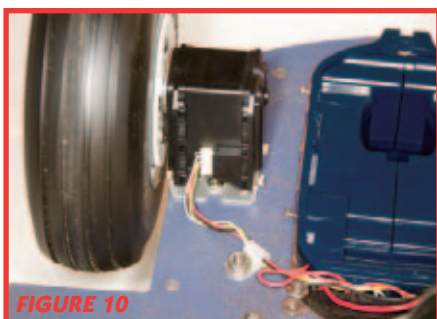


FIGURE 10

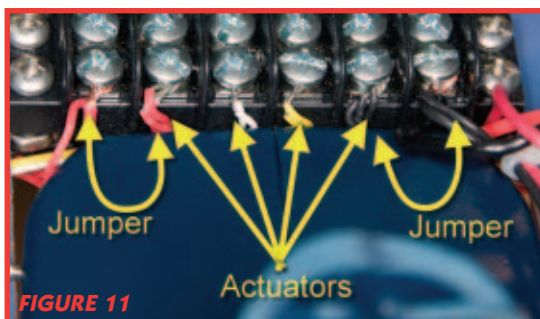


FIGURE 11

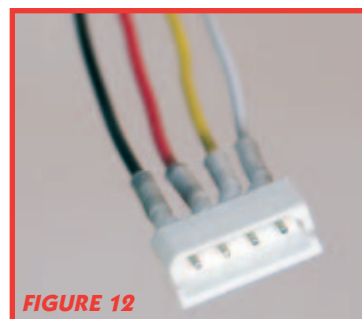


FIGURE 12

terminal and the outside black power terminal.

At this point, you can apply power to the actuators by turning on the switch. When you do so, you should see the LEDs on each actuator go on, then off. If they do not, remove the power and check your connections.

MICROCONTROLLER HOOKUP

Eventually, we will be using a Windows CE computer to control our Firebot. You might ask, why even add a microcontroller to the project if we are going to use another computer? Computers like PCs and CE devices are very powerful but they lack the low level control we need for various sensors we may want to use.

Eventually, we are going to program the DiosPro in a sort of slave mode so that we can issue commands on the RS-485 bus just like we do to the RX-28 actuator. This will allow us to connect whatever sensors we want to our DiosPro and read the results into the CE device — all with a single connection for both the sensors and actuators.

Dynamixel has seen fit to add an additional connector with each RX-64/28 actuator. This is a small male plug that you can attach wires to in order to create a receptacle that will allow you to use one of the included cables to connect your RX actuator to any application.

Attach four 3" pieces of colored wires to one of the included plugs as shown in Figure 12. You will also need to cover the connections with some heat shrink.

Connect the other end of the wires to the barrier strip as shown in Figure 13. The colors on this connector should match those used when you connected the actuator cables. The Dios workboard also needs two power connections. For this, I used a couple SchmartBoard connectors that I always keep on hand. Remove one of the ends from each connector and attach it to the barrier strip as shown in Figure 13. I used a green connector for my Vss (GND) connection and a yellow connector for the Vin connection.

You will need four standoffs in order to connect the workboard to the base as shown in Figure 14. I used 1" standoffs here as it gave me room to route cables.

I have added a chip to the DiosPro workboard. This is an RS-485 chip and you can find a detailed, step-by-step hookup sheet at www.kronosrobotics.com/Projects/RXhookupmain.shtml.

This hookup will show you how to wire the RS-485 chip

to the workboard. It also shows you how to change the ID of a device using the USB2Dynamixel adapter. You will need to change the ID on the left actuator to #2 (see Figure 1 as a reference as to which actuator is the left actuator).

Once the board is mounted, connect the main power terminal (yellow wire) to the Vin pin on the workboard and the GND power terminal (green wire) to the Vss pin on the workboard.

Take one of the RX actuator cables and connect it to the plug you attached to the barrier strip. Take a four pin, right angle header and attach it to the other end of the cable. Then, plug that end of the cable into the workboard as shown in Figure 14.

ACTUATOR INTERFACE TEST

We can now test our hookup. When you turn on the power switch on the Firebot base, you should see the LED on the workboard light up. If it does not, go back and check your connections.

Connect your PC to the program connector on the DiosPro workboard and load the program called "RXreadings.txt" into the Dios. This program takes four readings from the right side actuator (#1) and displays them. Feel free to change the device variable to 2 in order to check the left actuator.

CONNECT A RADIO TO YOUR FIREBOT

You may be asking why you should even mess with a radio if you plan on building an autonomous robot. It's simple. The radio will make it very easy to test your drive system and sensors, and a radio can also be used to take control of a runaway robot.

If you look at Schematic 1, I have connected a Vex radio to port 31. I like the Vex radio for two reasons. First, it's inexpensive. I have seen them online from \$29-\$49. The radio is also designed for use with a microcontroller. The DiosPro has a Vex library built in and you can find a complete interface article at www.kronosrobotics.com/Projects/vextradio.shtml.

Since I used 1" standoffs, I was able to stick most of the Vex radio under the DiosPro workboard as shown in Figure 15. The radio and antenna have mounts that make it easy to place in any location on your base. Connect the Vex

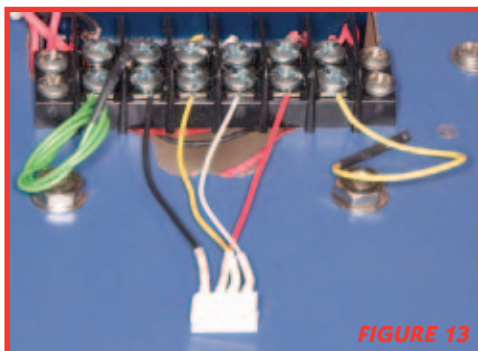


FIGURE 13

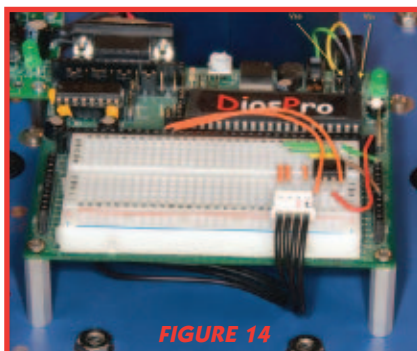


FIGURE 14

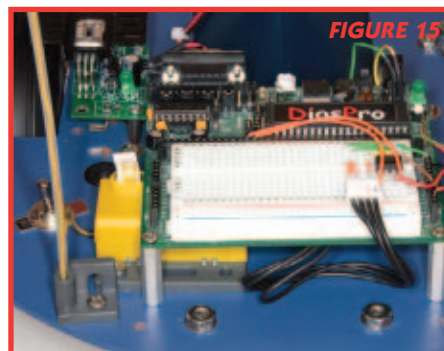


FIGURE 15

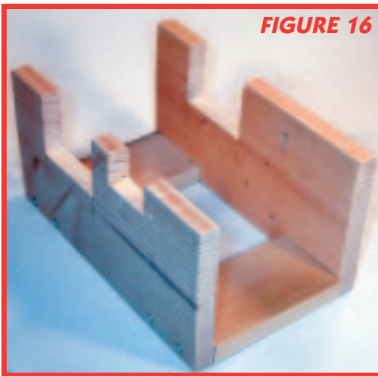


FIGURE 16



FIGURE 17

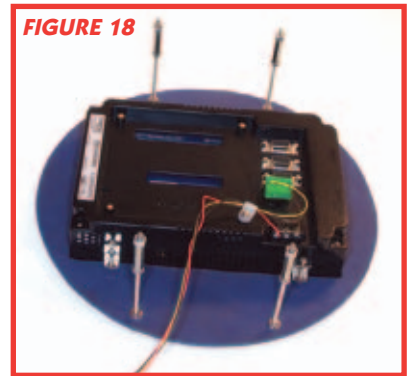


FIGURE 18

power leads to the Vss and Vcc header connectors on the right side of the breadboard. Connect the Vex data lead to port 31 on the workboard.

Load up the "VEX_Radioq.txt" program. This program will output the value of all six Vex channels, along with a status. If the status is 0, then the radio is off or out of range. If the value is 1, then the six data can be considered valid.

Before we move forward, you need to build some sort of support for your Firebot. This will come in handy when you need to do tests with the robot located on your desktop. I have used everything from cardboard boxes to old flour tins to support my various robots while programming or testing. The stand shown in Figure 16 was built out of some scrap.

Okay, you have your Vex radio connected and tested. You have your stand built. It's time for your Firebot to move. Load the program called "FireBotVex.txt" into the Firebot. When

you turn on your Vex radio transmitter, it will automatically calibrate the receiver and allow you to control your Firebot. I have set the program up to mix the channels 1 and 2 so that you have total proportional control over your robot.

After moving the robot around the house for a while, I have to say this is one of the smoothest and quietest robots I have ever built.

ADD THE REAL BRAIN

It's time for our little Firebot to evolve. Let's add a real computer. In this case, I am going to add a CE computer called the CUWIN3500. The CUWIN3500 shown in Figure 17 is manufactured by a company called Comfile Technology. It is designed for use with all kinds of applications, from industrial apps to public kiosks. It has a built-in touch screen but also supports an optional mouse

PARTS LIST

The following are the components needed to build this portion of the project:

Jameco — www.jameco.com

- MC78T12CT 12V regulator chip (part #871658)
- Right angle header (part #103271)
- Toggle switch (part #76241)
- Six position barrier strip (part #231010)
- TO220 heatsink (part #1582663)

SchmartBoard

- SchmartBoard jumpers, 5" yellow
www.schmartboard.com/index.asp?a=11&id=42
- SchmartBoard jumpers, 12" green
www.schmartboard.com/index.asp?a=11&page=a_products_accessories&id=122

COMFILE TECHNOLOGY

- CUWIN3500
www.cubloc.com/product/05_01.php

CrustCrawler

- RX-64
www.crustcrawler.com/motors/RX64/index.php?prod=67
- RX-28
www.crustcrawler.com/motors/RX28/index.php?prod=66
- Treaded wheels
www.crustcrawler.com/products/rover/wheels.php?prod=28

Kronos Robotics

- DiosPro chip
www.kronosrobotics.com/xcart/product.php?productid=16428
- Dios Workboard
www.kronosrobotics.com/xcart/product.php?productid=16566
- RS-485 chip
www.kronosrobotics.com/xcart/product.php?productid=16388
- DiosPro compiler
www.kronosrobotics.com/downloads/DiosSetup.exe
- ZeusPro development environment
www.krmicros.com/Development/ZeusPro/ZeusPro.htm

MISCELLANEOUS ITEMS

Vex radio

You will have to search the Internet to locate a Vex radio. I recently purchased one from an online eBay store for \$35.

Ryobi P700 flashlight

Available at most home centers.

Ryobi lithium-Ion battery and charger

Available online or at home centers.

and keyboard. While we won't be using the touch screen or putting the display to any real graphic use, they will come in handy for testing.

Since the CUWIN3500 is going to be our main controller, we need to be able to control our two actuators, as well as read our Vex radio and other sensors. To do this, we will put the DiosPro in a slave mode and have it act like an actuator. This is a simple test, as the commands to do so have been built into the RX library. The hardware interface is identical, so only the DiosPro program needs to be changed.

The CUWIN3500 has an RS-485 port with its own connector shown in Figure 18. Connect a yellow wire to the + terminal and a brown wire to the – terminal on the green connector, as shown. You will also need two power leads. Connect a red and black wire to the + and – terminals as shown in Figure 18. You may want to add a wire clamp to act as strain relief. This will make removing the brain easier.

The CUWIN3500 comes with a template to help you create a cutout. I created a duplicate base (minus the wheel cutouts) and using the template, cut out the hole for the CUWIN3500. I used the included mounting brackets to lock the unit in place. I used a combination of standoffs and 2-1/2" machine screws to set the upper platform 3-3/8" from the lower platform. The actual height of the DiosPro workboard will dictate how high the platform needs to be in order for the CUWIN3500 to clear the workboard.

Since the CUWIN3500 can be powered from 12V-24V, you may connect to either side of the regulator. As shown in Schematic 1, I recommend the unregulated side. This puts less load on the regulator. Connect the yellow data wire to the data lead to pin 6 on the RS-485 chip and the brown wire to pin 7.

When you turn your main power switch on, your CUWIN3500 should boot. If it does not, turn off the switch and check your connections. Once you know the CUWIN3500 is powered properly, load up the "RXSlavetest.txt" program and program the DiosPro. This program puts the DiosPro into slave mode.

PROGRAMMING THE FIREBOT BRAIN

I am going to use the Zeus as it's an inexpensive development environment that runs on the PC and CE platforms. If you have programmed the DiosPro microcontroller before, you will find the language very similar, as the core engine shares some of the same code segments. The language itself utilizes a very rich set of Basic commands. Several libraries have also been included. The USB2AX library will allow you to communicate with several Dynamixel product lines including the AX, RX, DX, and EX series of actuators and sensors. Let's fire up the brain and load a

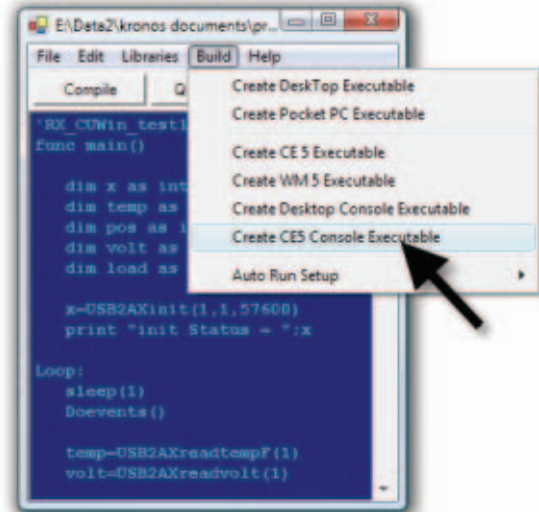


FIGURE 19

program or two. There are a couple ways to program the CUWIN3500. One way is to use an SD card.

Load up the "RX_CUWin_test1.txt" source in ZeusPro. Create a CE5 Console application as shown in Figure 19.

Copy that created executable called "RX_CUWin_test1_CE5C.exe" to an SD card. Once you have the CE executable on the SD card, it's a simple task to insert the SD card into the CUWIN3500 and run the program.

While this works fine for single program tests, it can be time-consuming when doing extensive development. There is a direct method that Microsoft has built into the PC-CE interface. It's important that you have the latest version of ActiveSync when using XP and the latest version of Windows Mobile Device Center when running Vista.

Here is how it works: Once you have the PC and CE device talking, load up ZeusPro and under the "Build/Auto Run Setup" menu, select the Auto Run option as shown in Figure 20.

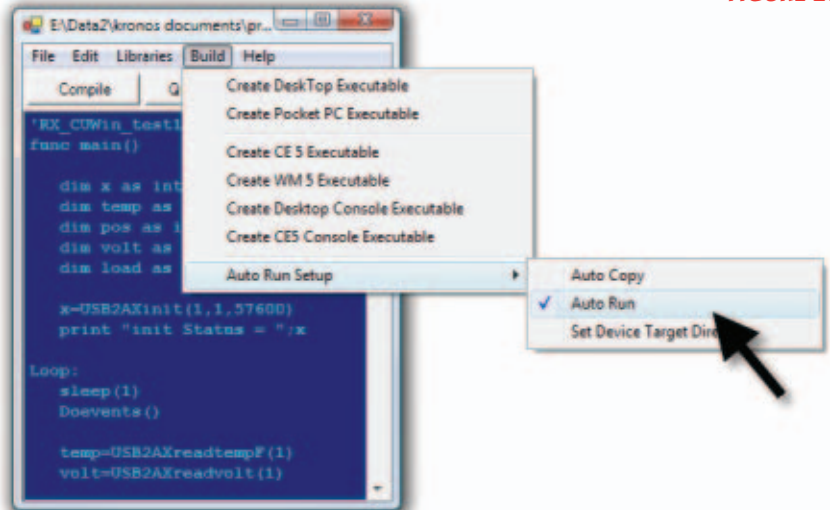


FIGURE 20

Now when you create the executable as you did in Figure 19 it will automatically be copied to the CUWIN3500 and started. How's that for single button programming? A couple things to keep in mind: First, you must stop the program if it's already running before you start it again. The default directory is the root directory. Anything copied to this area will be lost once the CUWIN3500 is rebooted. If you want to keep the program, move it to the Flash disk or an SD card. Second, keep the console display data to a minimum, as CE devices are not known for their high speed display systems.

THE CE PROGRAMS

RX_CUWin_test1.txt

Displays four readings from actuator #1 and displays them on the console.

RXDiosSlaveTest1.txt

Displays Vex radio data taken from the Slave DiosPro.

RXFireBotVEXCE.txt

This is the Firebot remote control program. Make sure the transmitter is on before starting the program.

I have included both the source and compiled versions of these programs. If you decide to use another CE device, you will need to modify the programs and recompile them in order for them to work on your particular unit.

CLOSING THOUGHTS

At first glance, the RX-28 actuators may seem a bit expensive. However, if you compare these with other midsized robot motors, you will find the cost comparable. Two RX-28s and wheels will cost you approx. \$432. The RX-28s have the added benefit of load and speed monitoring, as well as temperature and voltage. On top of all that, they can also be operated in a positional mode like a servo.

WHAT'S NEXT

I have not included any sensor interfaces for the Firebot, as I just don't have the space in this article for them. If you want to experiment yourself, start with the DiosPro interface and get the sensor working stand-alone first, then migrate it to the microcontroller-based control, then to the CE. Next month, I will be working on the larger Megabot and many of the techniques and interfaces I use on the robot can be used on the Firebot.

Be sure to check out the Kronos Robotics website for updates and source files to this project at www.kronosrobotics.com/Projects/megabot.shtml. **SV**



**Finally, a
social network
for you, the
electronics
enthusiast**

DESIGN CIRCUITS

EARN COMMISSION

WRITE BLOGS

RSS FEEDS

BUILD CIRCUITS

POST EVENTS

POST VIDEOS

TEXT CHAT

PROMOTE A JOB

JOIN CLUBS

PROMOTE COMPANIES

MAKE FRIENDS

POST PHOTOS

GET ANSWERS

FIND A JOB

CREATE FORUMS

CHATROOMS

VOIP CHAT

WEBCAMS

CREATE CLUBS

COLLABORATE

AND MUCH MORE!!!

Sign up for your free membership today!

www.SolderByNumbers.com

The Escape Robot's built-in microprocessor enables it to "think" on its own. (KSR4) **\$29.95**



The robot frog moves forward when it detects sound and repeats: start (move forward) -> stop -> left turn -> stop -> right turn -> stop. (KSR2) **\$19.95**



5mm White water clear LED 3.5V 10,000 mcd (AB287) **\$0.56**

20 second voice recorder/playback module.

The electret microphone is on the board. One button records, the other button is momentarily pressed to replay the message.

(pre-assembled) (A96010) **\$6.60**



The Velleman **Personal Scope** is not a graphical multimeter but a complete portable oscilloscope at the size and cost of a good multimeter. (HPS10) **\$146.**

Thousands more items on our web site!

Electronics123.com
Tel: 1-888-549-3749

102 E. Park Ave, Columbiana OH 44408

www.hagisonic.com

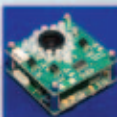
Also, Visit Our Distributor
www.robotshop.ca

AniBat™ : Ultrasonic Sensor



Wider horizontal beam pattern up to 200°
Covers front area with only three sensors
Extremely low 'click noise' of sensor
Range detection: 0 inches ~ 157 inches
Designed for indoor environments
Sensor operates at 40KHz
Real-time signal acquisition

StarGazer™ : Indoor Localization Sensor



High speed indoor localization
Provides position and heading angle
Serial (TTL 3.3-5V) Interface
Worlds best in resolution and cost
Designed for indoor environments
Small size: 50x50x28mm
Attaching Passive Landmark(ID) on ceiling



Other Related Products



StarGazer™ RS1.0



StarGazer™ Interface Board



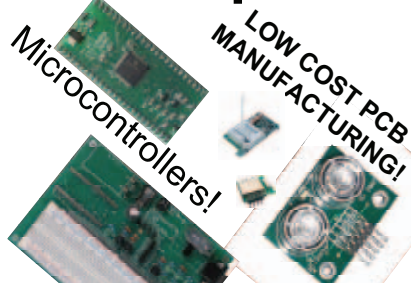
StarGazer™ Indicator



The Premier Robot-Sensor Company

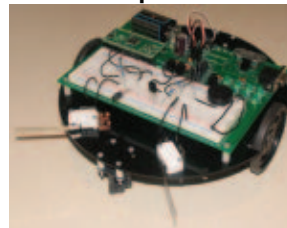
www.hagisonic.com

C Stamp is ...



Boards for Development!

Sensors and Communications!



Displays!

Come see what else at
www.c-stamp.com

ALL ELECTRONICS CORPORATION

THOUSANDS OF ELECTRONIC PARTS AND SUPPLIES

VISIT OUR ONLINE STORE AT
www.allelectronics.com

WALL TRANSFORMERS, ALARMS, FUSES, CABLE TIES, RELAYS, OPTO ELECTRONICS, KNOBS, VIDEO ACCESSORIES, SIRENS, SOLDER ACCESSORIES, MOTORS, DIODES, HEAT SINKS, CAPACITORS, CHOKES, TOOLS, FASTENERS, TERMINAL STRIPS, CRIMP CONNECTORS, L.E.D.S., DISPLAYS, FANS, BREAD-BOARDS, RESISTORS, SOLAR CELLS, BUZZERS, BATTERIES, MAGNETS, CAMERAS, DC-DC CONVERTERS, HEADPHONES, LAMPS, PANEL METERS, SWITCHES, SPEAKERS, PELTIER DEVICES, and much more....

ORDER TOLL FREE
1-800-826-5432

Ask for our FREE 96 page catalog

Robotics Showcase

ROBOT CONTROL MADE EASY

up to
2x120A



- RS232, RC, or Analog input
- Speed or Position Mode
- Dual channel output up to 120A
- Optional single channel to 240A
- Optical Encoder Inputs
- Intelligent Current Limiting
- Thermally Protected
- Field Upgradable Firmware
- Enclosed and Board-Level versions
- and many more advanced features ...

RoboteQ
www.roboteq.com



Medonis Engineering

Android Robot Head Kit

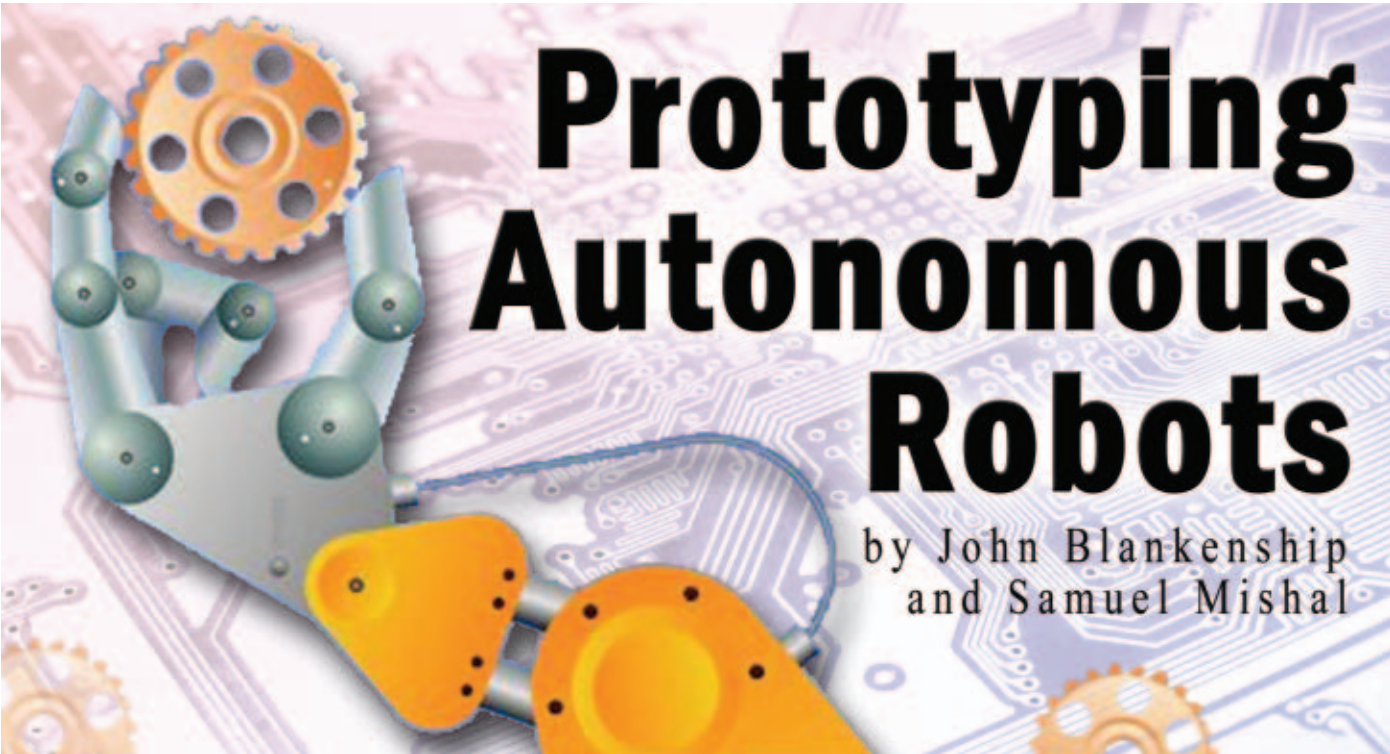


Maxwell

- Lifesize - stands 19" high!
- 5 servos move head, eyes, jaw
- 8 channel servo controller
- Windows control software

Great for Halloween displays!
Human Robot Interaction!
\$350.00

www.medonis.com



Prototyping Autonomous Robots

by John Blankenship
and Samuel Mishal

Building an autonomous robot is a goal of many robot enthusiasts. If you look up autonomous in the dictionary, you will see terms like “self directed” and “self sufficient.” If you apply the first term to a robot, it simply means that the robot is capable of accomplishing a task without human intervention. The second term suggests a deeper commitment. A self sufficient robot might be one that is capable of repairing itself, or at the very least, capable of recharging its own battery so that it can operate independently for extended periods of time.

It is important to differentiate an autonomous robot from an intelligent one. Intelligence generally implies reasoning or deduction. An autonomous robot does not necessarily need to be intelligent, although an autonomous robot may appear to be intelligent to those watching it perform a task — especially if it can independently accomplish its goals despite unexpected changes in its environment. The real difference is in how the decisions are made. This article will only address the topic of an autonomous robot.

As a serious robot hobbyist, you may have built an autonomous robot or at least attempted such a project. If you have, you were probably surprised by the cost and complexity of building something capable of dealing with a variety of environments on its own.

It should be obvious that an autonomous robot needs one or more sensors to monitor the world around it. The real problem is understanding what sensors are actually needed. To help make this point, imagine a variety of situations where humans have to deal with the world. Someone that is blind, for example, would need to rely on other senses such as hearing or touch. Even if someone can see though, they still might need other senses to accomplish some goals efficiently; humans often use hearing to determine where to direct their vision.

Just as people with limited senses can learn to handle the problems they encounter, a robot can be programmed to deal with its environment using only one or two sensors. The programming might, however, be much easier if an

appropriate combination of sensors is chosen. Let's look at a specific example to illustrate the point.

Assume that we are building a robot that can navigate through a cluttered room. This might be possible using only a camera, but the software needed to analyze the images could be very complicated. We could use some form of infrared object detection around the robot's perimeter, but how many of those sensors would we need to ensure failsafe operation? If we have infrared sensors do we need bumper switches too? And what about an ultrasonic or infrared ranger that can measure the distance to objects instead of just detecting that an object is close by? Would one or more of those make it easier to program the robot or would they just be an unnecessary expense?

If our creation is to be truly autonomous, it will also need a way to determine when its battery is low and then the ability to locate a charging station and perform some form of docking procedure to replenish its power.

There are many ways the robot could locate a charging station. It could use a limited vision system or maybe some form of beacon could be placed over or near the charger. In order to orient itself properly, the robot might use an electronic compass or once it gets close to the charger, the robot could follow a line on the floor into the proper docking position.

The whole point here is that there are many ways to create a robot that can accomplish any given set of goals. The choices you make will influence the software that has to be written, as well as the overall performance of the robot. Part of engineering any project is minimizing the overall costs while maximizing performance. Of course, many of the decisions to be made will be affected by the nature of the environment where the robot will operate. Not knowing in advance the expected nature of the environment adds a whole new level of complexity. Even little things — such as objects being moved to new locations or having sharp points — can dramatically alter what sensors the robot might need to ensure that it does not get stuck or damage itself.

One of the most unfortunate aspects of this dilemma is that even if you buy and mount several types of sensors, you won't know if they will work (let alone know if they were an optimal

choice) until your robot is constructed and you start to program. Then, if you decide you need some additional type of sensor or even if you just need more of what you already have, you may need to alter your robot's physical design in order to mount the additional items.

Once you get started, you might find that the microcontroller you chose for your robot is not fast enough or doesn't have enough memory to implement the algorithms needed to complete the project. After you have upgraded your controller and added the needed sensors, you might discover the robot's battery is no longer large enough to provide a reasonable period of operation. You put in a larger battery only to find that the motors you chose to power the robot cannot handle the increased weight of the new battery.

For many robot hobbyists, the above scenario is, sadly, very familiar. Wouldn't it be nice if you could test your programming ideas quickly and

easily with a variety of sensors so that you could make choices before beginning the actual construction? Once you have finalized your test programs, you can use their size and complexity to determine what features your robot's on-board computer should have. The total number of lines of code, for example, can help you estimate how much program memory you will need. If your algorithm makes use of arrays or floating point variables, then you will need a system that supports such operations.

Of course, unless your simulator can model things such as friction, gear ratios, wheel slip, centrifugal forces, and other mechanical issues, you will still need to do some design calculations. Nevertheless, if you know what sensors and microcontroller you will need you can reasonably estimate the size and weight of your robot, as well as the overall current requirements. Even though this information is only an estimate, it provides you with far more information than you had before the simulation.

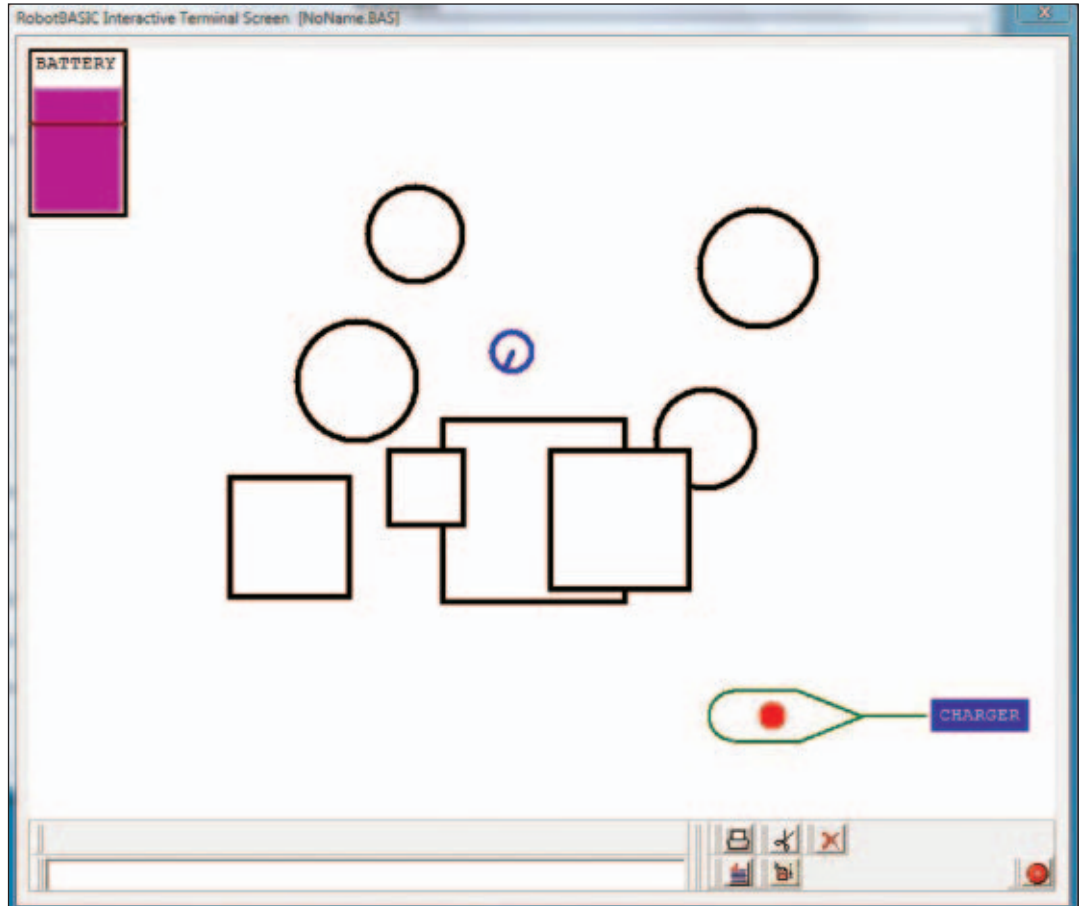


FIGURE 1. The simulated robot wanders until its battery needs to be charged.

Figure 2: This RobotBASIC program creates a simulated autonomous robot.

```
//---Variables & Constants
LOW_LEVEL      = 65
MAX_CHARGE     = 100
BeaconX        = 600
BeaconY        = 520
Battery         = 0

//=====
MainProgram:
  GoSub SetEnvironment
  While true
    gosub RoamAround
    gosub FindBeacon
    gosub FollowLine
    gosub ChargeBattery
  wend
End
//=====
SetEnvironment:
  // Draw 4 circles and 4 squares
  for i=1 to 4
    SetColor Black
    LineWidth 4
    x = random(420) + 120
    y = random(250)+60
    size = random(50)+50
    circle x,y,x+size,y+size
    x = random(450)+100
    y = random(250)+100
    size = random(100)+50
    rectangle x,y,x+size,y+size
  next
  // Draw docking line
  SetColor Green
  LineWidth 2
  gotoxy BeaconX+100,BeaconY
  lineto BeaconX+50,BeaconY
  Lineto BeaconX,BeaconY-20
  Lineto BeaconX-50,BeaconY-20
  Line BeaconX-50, BeaconY+20,
    BeaconX,BeaconY+20
  Lineto BeaconX+50,BeaconY
  Arc BeaconX-70,BeaconY-20,
    BeaconX-30,BeaconY+20,
    DtoR(90),DtoR(180)
  // Draw beacon
  circle BeaconX-30,BeaconY-10,
    BeaconX-10,BeaconY+10,red,red
  // Draw charging station
  Rectangle BeaconX+105,BeaconY-12,
    BeaconX+180,BeaconY+12,
    blue,blue
  SetColor White,Blue
  xyString BeaconX+110,BeaconY-10,
    "CHARGER"
  // Battery meter
  SetColor Black,White
  LineWidth 3
  Rectangle 0,0,76,130,black
  gotoxy 1,123-LOW_LEVEL
  LineTo 75,123-LOW_LEVEL,3,Brown
  SetColor Black,White
  xyString 5,2,"BATTERY"
  // Charge the robot initially
  GoSub PlaceRobot
  GoSub SimulateCharging
  return
//=====
PlaceRobot:
  rLocate BeaconX+80,BeaconY,-90

  // designate the beacon and line colors as
  // non-objects
  rInvisible Green,Red
  rIgnoreCharge False
  Return
//=====
RoamAround:
  while true
    GoSub BatteryMeter
    if Battery < LOW_LEVEL then
      return
    // Fwd until object is found
    while not (rFeel()&14) AND
      not (rBumper()&14))
      rForward 1
    wend
    D=random(100)
    for i=0 to random(50)+50
      if D>50
        rTurn 1
      else
        rTurn -1
      endif
    next
  wend
  return
//=====
FindBeacon:
  cnt=0
  Repeat
    GoSub BatteryMeter
    cnt=cnt+1
    gosub FaceBeacon
    if cnt <20
      gosub ForwardTillBlocked
      // decide to follow L or R
      if (cnt=1) and (rFeel()&8)
        then cnt=10
      if (cnt<10)or(random(20)<2)
        TurnDir = -1
      else
        TurnDir = 1
      endif
      GoSub GoAround
    else
      gosub UnStick
      cnt=0
    endif
  Until rSense()&2 // Line Found
  Return
//=====
FaceBeacon:
  while not rBeacon(Red)
    rTurn 1
  wend
  // Show battery condition
  GoSub BatteryMeter
  return
//=====
ForwardTillBlocked:
  while not (rFeel() & 14) AND
    not (rBumper() & 14)
    rForward 1
    if rSense() then break
  wend
  GoSub BatteryMeter
  return
//=====
GoAround:
```

continued ...

continued ...

```
if rSense() then return
If TurnDir > 0
  FN = 6
Else
  FN = 12
Endif
for i=1 to 20 + random(250)
  While (rFeel()&FN) or
    (rBumper()&4)
    rTurn -TurnDir
  Wend
  rForward 1
  rTurn TurnDir
Next
Return
//=====
UnStick:
if random(100)<50
  then rTurn 180
for i=0 to 100+random(200)
  while not (rBumper()&14)
    rForward 1
  wend
  rTurn random(8)-3
next
Return
//=====
FollowLine:
while rFeel()=0
  rForward 1
  while rSense() & 1
    rTurn 1
  wend
  while rSense() & 4
    rTurn -1
  wend
wend
Return
//=====
ChargeBattery:
rTurn 180
while not rBumper()
  rForward -1
wend
GoSub SimulateCharging
Return
//=====
SimulateCharging:
SetColor LightMagenta
for i= Battery to MAX_CHARGE
  gotoxy 5,125-i
  lineto 70,125-i
  Delay 100
  gotoxy 1,123-LOW_LEVEL
  LineTo 75,123-LOW_LEVEL,3,Brown
next
rCharge MAX_CHARGE
Battery = MAX_CHARGE
Return
//=====
BatteryMeter:
// Show battery condition
Battery=rChargeLevel()
SetColor White
gotoxy 5,120-Battery
lineto 70,120-Battery
gotoxy 1,123-LOW_LEVEL
LineTo 75,123-LOW_LEVEL,3,Brown
Return
//=====
```

RobotBASIC is a totally free, full-featured programming language with an integrated robot simulator that allows you to prototype your ideas before you begin construction of your real robot. You can use it to try out a wide variety of sensors and programming algorithms in different simulated environments.

Let's examine a program that demonstrates how easy the prototyping process can be when using a simulation. This program creates a simulated robot that will wander around a room containing a number of randomly sized and shaped objects. The robot constantly monitors its own battery while it moves. If the battery level drops below a predefined point, the robot will abandon its original behavior and seek a charging station. The robot is assumed to have charging contacts mounted on its backside that are compatible with those on the charging dock.

The charger will be marked with a beacon hanging from the ceiling above it. (Actually, the beacon is above a line on the floor that leads to the charger. More on this in a moment.) Since the beacon is high, the robot can see it even if there are objects on the floor between it and the charger. The robot will make its way to the charger by going between or around obstacles. This ability to autonomously navigate a crowded room is a powerful behavior by itself.

Once the robot is close to the beacon, it locates a line on the floor and follows it directly to the charger. When the charger is encountered, the robot turns around and backs into it, allowing contact plates to make the required electrical connections. When the battery is charged, the robot will move away from the charger and continue wandering around the room.

Figure 1 shows the output screen for our example program. In the upper left hand corner is a display of the battery's condition. The lower right corner shows a line running to the charging station. Notice how the line is looped on the end so that the robot will find the charger no matter which direction it follows the line.

Even though the interior of the environment is extremely cluttered and difficult to navigate, the program in Figure 2 will always allow the robot to find the charger.

In order to analyze the program in Figure 2, let's look first at the MainProgram routine. It creates a random environment and then enters an endless loop where the robot roams randomly until the battery needs charging, looks for the beacon until it finds a line, follows the line until it finds the charger, and then charges the battery.

Each subroutine accomplishes a specific task and — as you would expect — requires sensors appropriate for the problem being solved.

The **RoamAround** routine uses infrared perimeter object detection and bumper switches to determine when objects block the robot's path. (Note that complete information on the sensors is given in the RobotBASIC help files.) When blocked, the robot makes random turns and continues to roam. Although this behavior is not particularly useful, it provides a task to keep the robot busy so we can test the battery charging aspects of the program.

The **FindBeacon** routine enables the robot to autonomously navigate to the charging station. It uses a beacon-detecting sensor that could be made from a color-detecting camera or phototransistor circuit that detects a specific infrared frequency. The robot turns until it faces the beacon and then moves forward until some object blocks its path. Most of the time, the robot will go around the object by using its perimeter sensors to follow the contour of the object. The robot has no way of knowing how far it should follow the edge, so it simply does so for a random distance.

Unfortunately, since the objects in the environment are randomly sized and placed, the robot will occasionally get stuck in a crevasse or other unusual situation. A counter is used that periodically causes the robot to execute a series of random motions that eventually will resolve the problem. This may not seem like a

Extreme Robot Speed Control!

Sidewinder

\$399

- 14V - 50V - Dual **80A** H-bridges - 150A+ Peak!
- Adjustable current limiting
- Temperature limiting
- Three R/C inputs - serial option
- Many mixing options - Flipped Bot Input
- Rugged extruded Aluminum case
- 4.25" x 3.23" x 1.1"

RC Control

BotIQ Favorite!



\$39.99

Scorpion Mini

- 2.5A (6A pk) H-bridge
- 5V - 20V
- 1.6" x .625" x 0.25"

\$159.99

Scorpion XXL

- Dual **20A** H-bridge 45A Peak!
- 5V - 28V
- 2.7" x 1.6" x 0.75"

\$104.99

Scorpion XL

- Dual **13A** H-bridge
- 5V - 28V
- 2.7" x 1.6" x 0.5"

Introducing Dalf

- Closed-loop control of two motors
- Full PID position/velocity loop
- Trapezoidal path generator
- **Windows GUI for all features**
- Giant Servo Model!
- C source for routines provided
- PIC18F6722 CPU

\$250



See www.embeddedelectronics.net

H-bridges: Use with Dalf or with your Micro/Stamp

\$219



OSMC

- Monster power!
- 14-50V **160A!**
- 3.15"x4.5"x1.5"
- 3 wire interface



Simple-H

- 6-28V **25A!**
- 2.25"x2.5"x0.5"
- 3 wire interface
- current & temp protection

\$79

We also do consulting!
Give us a call for a custom motor control to meet your exact needs

ROBOT POWER
www.robotpower.com



Phone: 253-843-2504 • sales@robotpower.com

sophisticated solution, but the robot often appears to demonstrate remarkable insight as it makes its way through a complicated environment to find the beacon. The robot has been tested with hundreds of random environments and has never failed to reach the charger.

This is another issue that makes a simulator a valuable tool while developing an autonomous robot. To ensure that an algorithm works, you must test it on a variety of environments. Using a simulator, you can randomly create environments and conditions, and generate numerous scenarios to exhaustively test your project. All this is achieved automatically without you having to be there for the duration. You can go about your business while the testing is being carried out, assured that no damage will be done to either the robot or the environment. Try modifying this program to create more or less objects, or even create a totally different environment, and see how the robot performs. When the line is detected with the robot's *rSense()* function (line sensors beneath the robot), the robot follows the line until it finds the charger. At that point, the robot makes a 180 degree turn and backs into the station to complete the docking operation.

RobotBASIC has integrated functions for simulating the battery charging and discharging situations. In this example, a *BatteryMeter* routine was added to give users of the program a visual indication of the battery's condition.

The important point here is that you can design the programs needed for this task without having to physically construct the robot. You can determine the type and number of sensors you will need as you conceive and develop the algorithms that handle problems to be solved.

These routines and many others are explained, enhanced, and combined to form powerful autonomous behaviors in the book *Robot Programmer's Bonanza*. You can purchase this book through the *SERVO* webstore at <http://store.servomagazine.com>. The full version of RobotBASIC, along with demo programs and tutorials can be downloaded free from www.RobotBASIC.com.

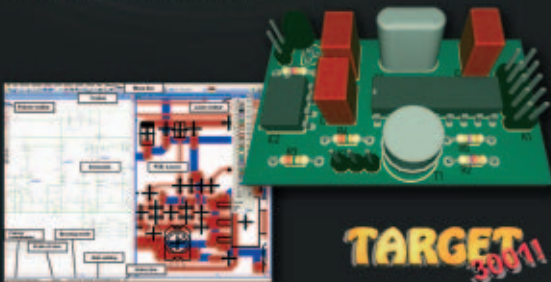
Remember, the program in Figure 2 is only one possible solution. You could have decided to use RobotBASIC's compass or GPS system or even one of RobotBASIC's custom sensors to find the charger. As you use the simulator, you will see that you can explore many more options when you don't have to actually construct a robot to test your ideas. In fact, you may find that just developing the ideas can be almost as satisfying as building the real robot. At the very least, the use of simulations can help you decide which hardware projects are worth your time.

After using the simulation to finalize your algorithms and determine the hardware needed for your project, you can build your robot with a microcontroller or you can use the RobotBASIC I/O commands to control your robot directly from the PC. You can even use RobotBASIC's integrated wireless protocol to control the robot with the very same program used in the simulation. No matter which choice you make, you will soon discover how easy and enjoyable it can be to test your ideas using a simulation. **SV**

PCB-POOL.COM

Download our FREE PCB Layout Software!

- Schematic Capture
- PCB Layout
- Auto Router & Auto Placer
- Complete Component Library
- Professional Version



TARGET 3001

www.free-pcb-software.com

BLDC HARDWARE PART 2

by Fred Eady

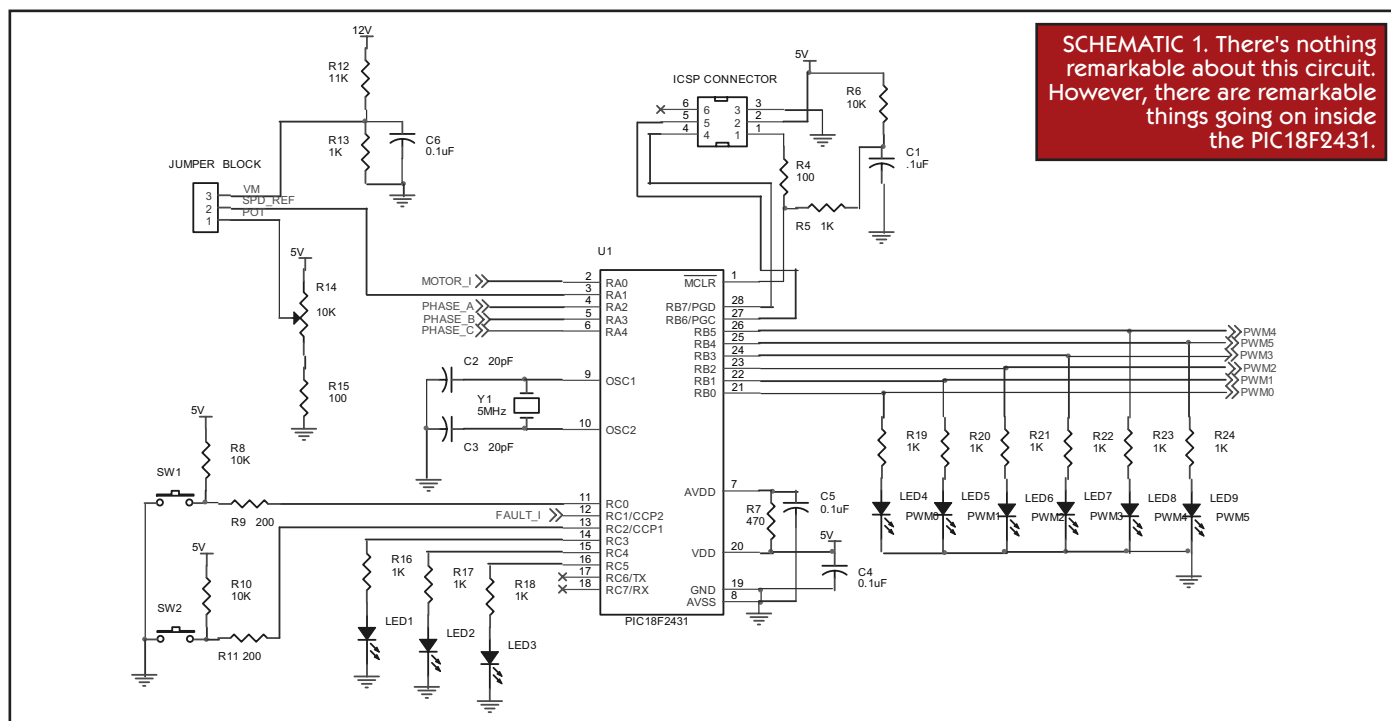
Manually rotating the shaft of my Anaheim Automation BLY171S-24V-4000 BLDC motor with my fingers isn't making it. So, I've scrounged up and assembled some electronic components that I intend to use to force the BLDC motor shaft to rotate in lieu of my motor-shaft-driving fingers.

If you take a look at the specifications for the BLY17 series of brushless DC motors, you'll see that every motor in the family has differing characteristics. With that, it will be useless for me to whip up a one-of-a-kind solution for the motor that is currently sitting on my bench.

What if your BLDC motor application requires a bit more torque than my example project can offer? What if you don't want to use a PIC microcontroller as the motor controller? I've rattled off only a couple of outside-of-the-box possibilities and I'm sure there are many more we could expose. Rather than approach the rest of this project with a goal of manipulating only the '4000, we will assemble a microcontroller-based BLDC motor driver using motor control building blocks that can be scaled and applied to other BLDC motor driver designs, as well as the '4000. Now that you're privy to the plan, let's get started.

The Microcontroller

Last time, I alluded to my preference of the PIC18F2431 microcontroller as the master intelligence for this BLDC



motor driver project. The F2431 was designed specifically for motor control. In particular, it is composed of several specialized motor control subsystems and is well equipped to oversee the control of a BLDC motor such as this one.

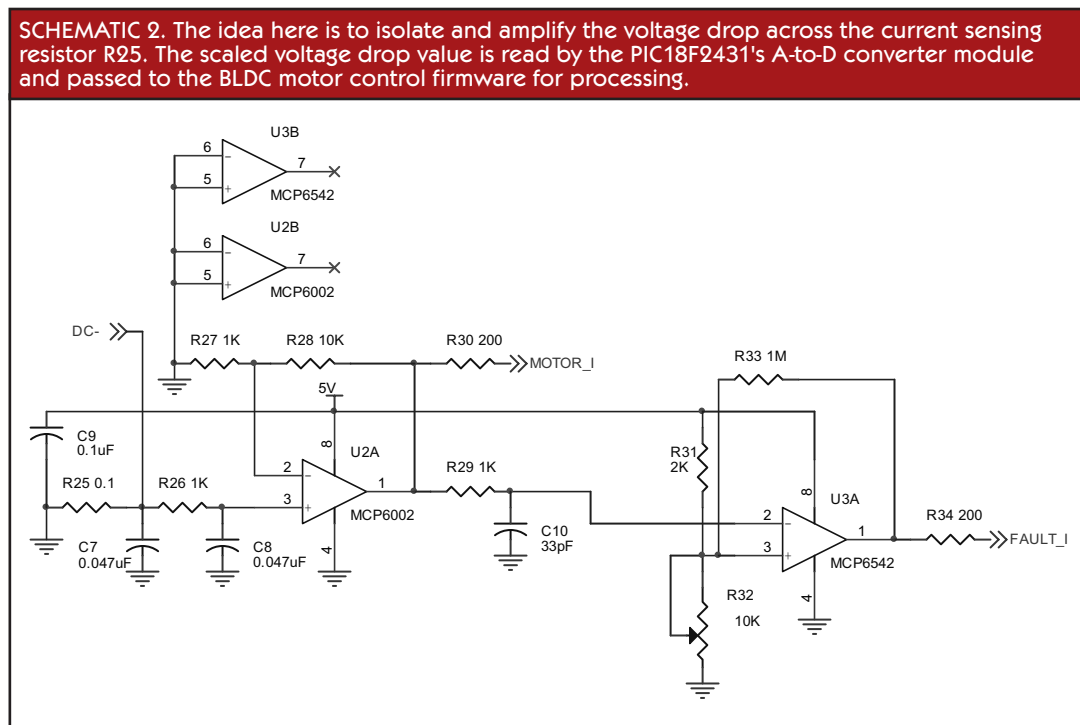
Recall from our previous discussion that a BLDC motor requires a three-phase electronic drive mechanism. Three phases of BLDC motor drive require a total of two PWM signal sources per phase. In addition, each BLDC motor driver phase must be able to operate according to the commutation rules of the BLDC motor. The F2431 motor

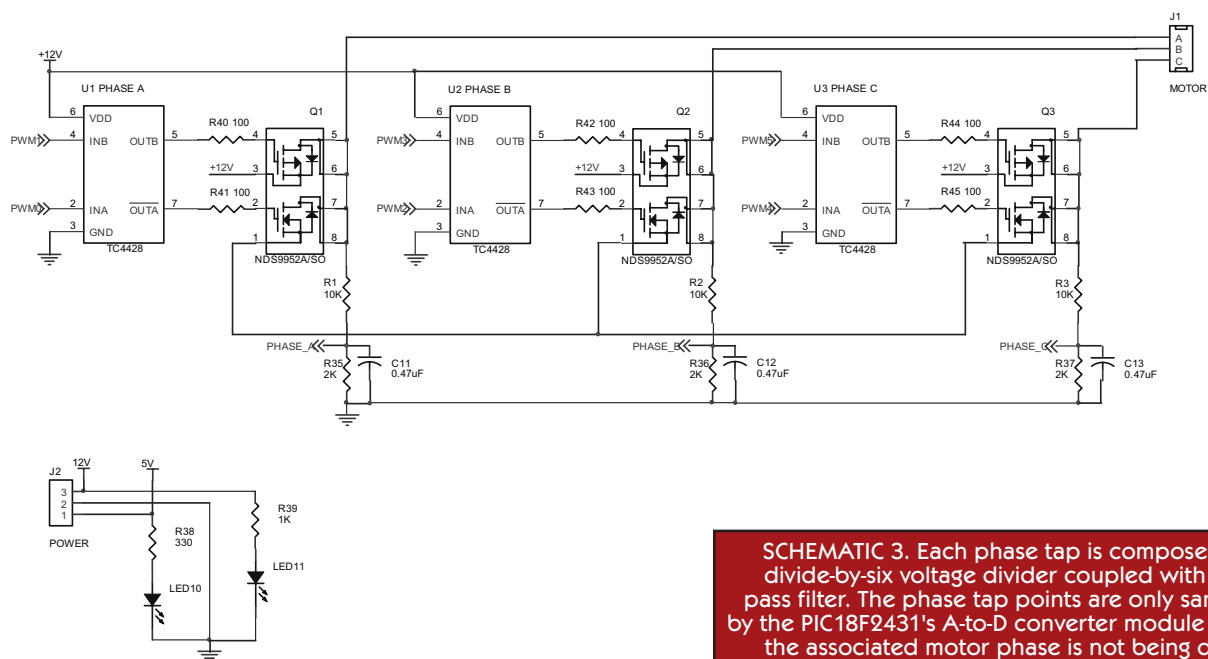
control PWM module has plenty of knobs and switches that enable it to adapt to most any motor control application.

The PIC18F2620 is a great general-purpose microcontroller. However, it is severely PWM challenged when called upon to master a sextet of MOSFETs in a three-phase BLDC motor driver configuration. The F2620 can only muster a pair of PWM ports that won't play well with a BLDC motor's commutation scheme. On the other hand, the F2431 supports six channels of BLDC-friendly PWM in the same package that houses a 28-pin F2620 microcontroller. The

F2431's PWM ports are part of what is called the 14-bit power control PWM module.

As we progress through this project, you'll come to the conclusion that the more we can monitor in the way of motor voltages and currents, the better. I felt that the resources offered by the F2431's analog-to-digital (A-to-D) converter module were sufficient for our project. Otherwise, I could have chosen to include a more robust 40-pin PIC18F4431 as the master BLDC motor driver microcontroller. The F4431 has a fourth pair of PWM channels





SCHEMATIC 3. Each phase tap is composed of a divide-by-six voltage divider coupled with a low pass filter. The phase tap points are only sampled by the PIC18F2431's A-to-D converter module when the associated motor phase is not being driven.

and some extra A-to-D converter inputs that could come in handy in larger and more complex applications.

It really doesn't matter if the F4431 or the F2431 is in control of a BLDC motor as each of the aforementioned microcontrollers contains timer mechanisms that work closely with the on-chip motor control PWM subsystem and high-speed A-to-D converter module. Timing the capture of voltage and current measurements is a very important piece of BLDC motor driver firmware. Precise timing is also essential in the commutation process.

For instance, in this project the F2431's A-to-D converter module will be used to help determine the desired commutation time slot. Being able to trigger an A-to-D capture at the precise moment is very important. The F2431's A-to-D converter module can be triggered in a number of ways, including the overflow of a timer.

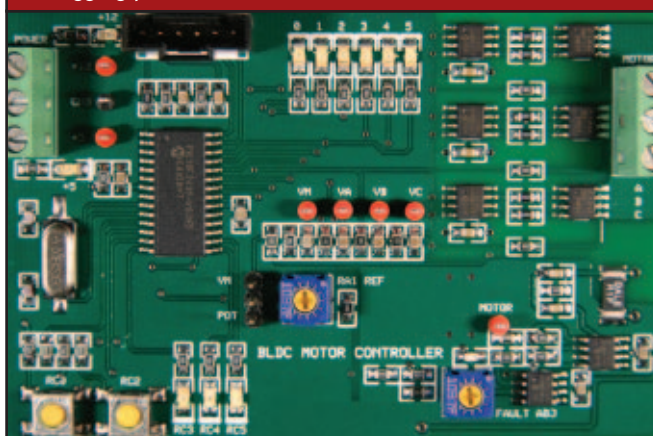
Our intent is to drive a BLDC motor without the aid of Hall effect sensors. However, if you have the need to employ a BLDC motor's Hall effect sensors, the F2431 has a built-in interface module. The F2431 also houses a motion feedback module that is capable of interpreting quadrature inputs. We won't discuss the Hall effect sensor interface module or the motion feedback module here.

I don't anticipate having to use all of the 200 ksp/s (kilo samples per second) bandwidth that the F2431's high speed 10-bit A-to-D converter module

can provide. However, I can foresee maybe taking advantage of its ability to simultaneously sample a pair of A-to-D converter input channels. Hopefully, the BLDC motor shaft won't be moving so fast that the F2431 peripherals can't keep up. I really don't think that the shaft can rotate fast enough to outrun the PIC's timers and A-to-D converter module.

Since we last spoke, I've become a student of BLDC motor control theory. I recall coming across a statement in a Microchip document that suggested running the three-phase controller PWM signal at a minimum of 10 times the maximum BLDC motor shaft speed. The BLY1715-24V-4000 is rated at 4,000 RPM maximum. Recall that BLDC motor speed is a function of the voltage applied to the motor's coils. I can tell you that we will be providing 12 volts DC to the BLY coil structure. Since we won't be supplying 24 volts DC to its stator coils, we can count on not reaching the maximum RPM figure. Thus, a maximum rotational rate of 2,000 RPM is a reasonable assumption at this point. Applying the 10X-PWM rule of thumb puts our PWM frequency design point at 20 kHz. Will a 20 MHz microcontroller processor clock (5 MHz clock input with the F2431's 4X PLL enabled) allow the PWM module to emit a 20 kHz signal? Here's what we know numerically, plugged into the F2431's PWM

PHOTO 1. This board is covered liberally with ground plane copper to help suppress any spurious electrical noise that may be produced while driving the BLDC motor. The unmarked six-pin header directly above the PIC18F2431 microcontroller is the Microchip ICSP programming/ debugging portal.



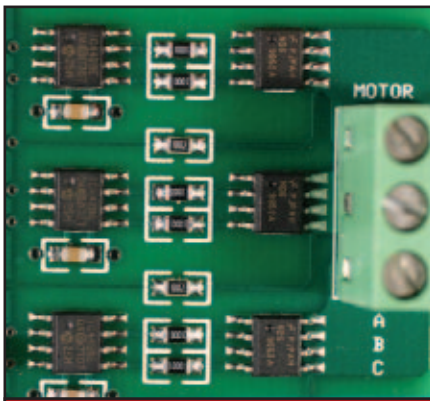


PHOTO 2. The pinouts of the TC4428 and NDS9952A ICs lend themselves to easy PCB layout and mounting using planes of copper, which are on the opposite side of the PCB in this shot.

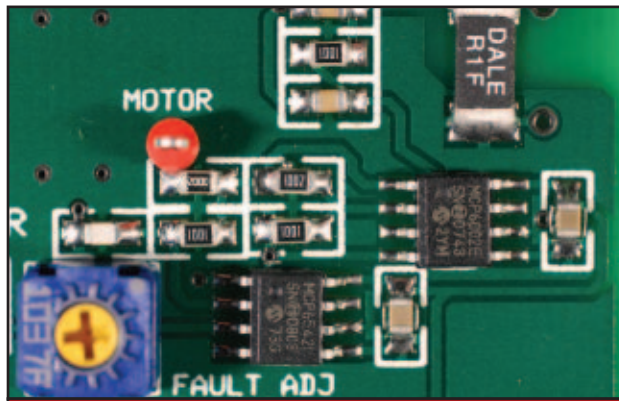


PHOTO 3. I placed a MOTOR test point on the output of the MCP6002 op-amp to make it easy to monitor the scaled current sense voltage using a standard VOM (Volt Ohm Meter). Note the ground plane area that engulfs the analog current monitoring circuitry.

be used as a speed control voltage reference. Resistors R12 and R13 form a 12:1 voltage divider. That is, when the supplied motor voltage is 12.0 VDC, the voltage presented to the VM (Voltage Motor) pin is 1.0 VDC. The voltage at the VM pin of the jumper block can also be used as a reference in the BLDC motor commutation process. The VM voltage is scaled down so as not to numerically overload the F2431's A-to-D

period mathematical equation:

$$\text{PWM Period} = ((\text{PTPER} + 1) * \text{PTMRPS}) / (\text{FOSC} / 4)$$

where:

$$\text{PWM Period} = 1 / \text{PWM Frequency} = 1 / 20000 \text{ Hz} = 0.000050$$

PTPER is the PIC PWM Period Register Value

PTMRPS is the PWM Timer Postscaler Value

FOSC = 20 MHz

The PTMRPS postscaler value is 1 as no postscaler is used in our calculation. Solving for PTPER yields a value of 249. The value of PTPER + 1 (250) will easily fit into the PTPER register. If we need to exceed the 20 kHz PWM frequency, that won't present a problem as the frequency increases as the value of PTPER decreases.

The leaves on the branches of the F2431 family tree can be positively identified by studying Schematic 1. It is important to keep up with the BLDC motor current as each '4000 phase winding's resistance is 1.8Ω. As you can imagine, dropping 12 volts across 1.8Ω in an uncontrolled manner will result in a current level that has the potential to fry the BLDC motor and its three-phase driver circuitry. A-to-D converter input RA0 is tasked to read the op-amp scaled voltage drop across a 0.1Ω current sense resistor monitoring current flowing through the three-phase driver MOSFETs and their associated BLDC motor windings.

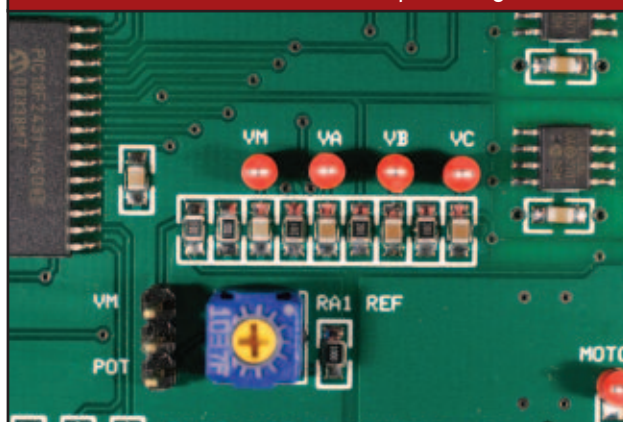
A-to-D converter input RA1 is the input portal for reference voltages that we may need to control the BLDC motor commutation process. For instance, R14, a 10K potentiometer, can

converter module, which can only measure voltages that lie between zero and VDD. If we were incorporating Hall effect sensors in our BLDC motor driver design, their outputs would feed the PIC18F2431's RA2, RA3, and RA4 digital inputs. Since we are not using the BLY Hall effect sensors to determine the BLDC motor's commutation points, RA2, RA3, and RA4 are A-to-D inputs that are measuring BEMF (Back Electromotive Force) supplied via the disconnected BLDC motor phase windings.

Six PWM signals are needed to drive the three-phase BLDC motor electronics. PORTB of the F2431 is dedicated to PWM duty with the exception of the PORTB pins that interface to the PIC programmer/debugger. The LEDs and their associated current limiting resistors attached to each of the PWM lines are included for debugging purposes.

LED1, LED2, and LED3 are available for discretionary use. The same holds true for the pair of pushbutton switches, which are monitored by digital inputs RC0 and RC2. Digital input RC1 is on the lookout for an overcurrent signal that is generated by the current monitoring circuitry in Schematic 2.

PHOTO 4. None of the voltage divider ratios are carved in stone. The voltage divisor values of six and 12 were chosen to more easily mathematically support calculations associated with the 12 volt motor input voltage.



Motor Current Monitor

Microchip manufactures some pretty good analog devices to complement their PIC microcontrollers. The meat in Schematic 2 is provided from the loins of a Microchip MCP6002 operational amplifier and an MCP6542 comparator. Both the 6002 and 6542 are rail-to-rail devices that can operate with a single-rail 5 VDC power supply. The 6002 is a general-purpose op-amp and is well suited for our BLDC motor current monitoring

hardware. The three-phase motor driver MOSFET source pins are all tied together and feed the DC junction at R25. R25 is a one watt precision, 1% tolerance 0.1 Ω current sense resistor (Mouser Electronics part number 71-WSLT2512-0.1-E3). The voltage drop across R25 is proportional to the current that is flowing in the MOSFETs of the three-phase BLDC motor driver. The DC voltage is filtered and presented to the non-inverting input of the 6002 op-amp, configured as a non-inverting amplifier with a gain of 11. The amplified voltage representing the current flowing through the BLDC motor windings is fed to the F2431's RA0 A-to-D converter input from the output pin of the op-amp. The output of the 6002 also feeds the inverting input of the comparator. Resistor R31 and potentiometer R32 form a voltage divider that feeds the non-inverting comparator input.

The voltage divider combination of R31 and R32 sets the overcurrent trip point. The output of the 6542 comparator feeds the F2431's FAULT_I input. As you've already ascertained, we can monitor the BLDC motor winding current actively in firmware using the output voltage from the op-amp or passively with the continual probing of a digital input pin using the comparator.

Three-Phase Half Bridge Motor Driver

The heavy lifting within the three-phase BLDC motor driver circuitry is done by the trio of Fairchild NDS9952A Dual N and P Enhancement Mode Field Effect Transistors (FETs). Each 9952A contains an N-channel MOSFET and a complimentary P-channel MOSFET. I chose this part over the IRF7309 for one simple reason. I was placing a Mouser Electronics order for a *Nuts & Volts* column I'm working on and decided to save some shipping costs and include the parts for this project in the same order. After specifying the IRF7309 on the Mouser ordering page, I was informed that Mouser only offers the 9952A instead. If you wish to use the 7309 in your three-phase BLDC motor driver design, you can order it from Digi-Key, who also has the 9952A in stock. However, you'll find that in single piece quantities you can get two 9952A parts for the price of a single 7309.

Obviously, both parts have the same pinout and packaging. So, how do they compare electrically? Well, the IR part is

just a tad bit heftier than its Fairchild cousin due to the lower ON resistance of the MOSFETs. Even with that, both parts can handle in excess of two amperes of continuous current. However, I don't plan to drive enough current through the BLDC motor windings to justify a large heatsink pad to cool the MOSFETs.

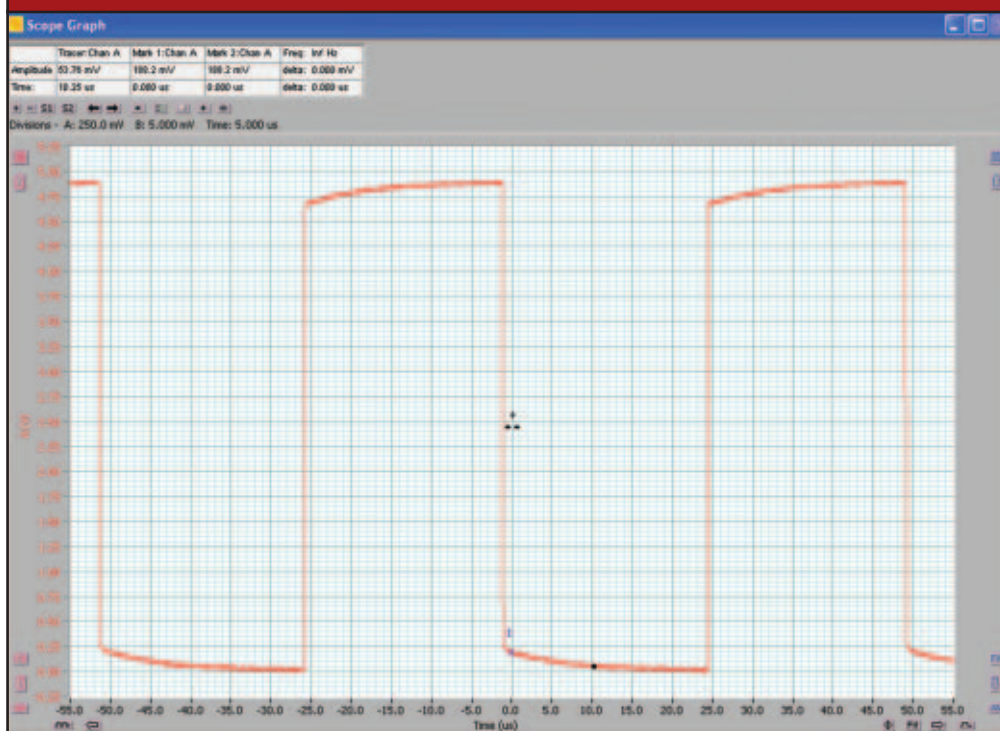
As you can see in Schematic 3, the Microchip TC4428 MOSFET drivers won out over the Microchip TC446X series. The TC4428s are electrically connected to the 9952A MOSFET pairs so as to deactivate the MOSFETs when a logical high is presented to the associated TC4428 input. Normally, this would be disastrous as the default inactive state of a PIC's PWM signal is logically low. That rule does not apply to the F2431. Its power control PWM module allows us to programmatically determine the active and inactive states of the PWM signals with a configuration fuse setting.

Each 9952A phase driver MOSFET pair has all of its drain pins tied to a unique BLDC motor phase winding. Every driver's drain pins are also feeding a divide-by-six voltage divider. Each of the divide-by-six phase taps is filtered using a low-pass filter which is intended to suppress the high frequency PWM signals that are driving the gate pins of the MOSFET phase driver ICs. The filtered voltages at the PHASE_A, PHASE_B, and PHASE_C tap points are the BEMF levels that we will read using the F2431's high-speed 10-bit A-to-D converter module.

From Schematic To Reality

Everything we've covered in the schematics is

SCREENSHOT 1. Everything thus far is working as designed. Understanding how the PIC18F2431's power control PWM module works is a must if we are to successfully commutate our BLDC motor.



represented by physical electronic components in Photo 1. The easiest electronic assembly to "PIC" out of Photo 1 is the three-phase driver. The trio of TC4428 ICs that are driving the 9952A MOSFET pairs are more easily seen in Photo 2.

The 0.1Ω precision current sensing resistor is the bridge between the three-phase driver area and the current monitor circuitry shown in Photo 3. The MOTOR test point allows us to manually measure the current sense voltage that is being fed to the PIC18F2431's A-to-D converter module. I didn't design in a FAULT LED on the output of the MCP6542 comparator. I figured I could use one of the LEDs attached to PORTC to programmatically indicate a current overload via the logic level of the FAULT_I digital input pin (RC1). I also swallowed the LED lure on the PWM outputs. Each of the six PWM signals that are feeding the TC4428 MOSFET drivers is monitored by an LED indicator. The PWM monitor LEDs make it possible to see the PWM commutation patterns without test equipment. The ability to see what these PWM signals are doing is an excellent debugging tool that I'm sure we'll use in the process of building the BLDC motor firmware.

Photo 4 is a view of the reference potentiometer, the divide-by-12 voltage divider, the bottom half of the divide-by-six BEMF voltage dividers and the low-pass filter capacitors. I anticipated the need to manually measure the BEMF voltages during debugging and testing, so I designed in test points for the divide-by-12 voltage point and the three BEMF voltage taps.

That pretty much does it for the hardware. Before we move on, here's a word of caution. After building up your BLDC motor controller, don't apply 12 volts to the board as the TC4428s will attempt to turn on all of the MOSFET pairs simultaneously. Take it from me, the board will get very hot at the MOSFET end before releasing the magic smoke. With that in mind, let's begin our firmware build process by going to school on the F2431's power control PWM module.

Gaining Control Of The Three-Phase Driver

The way our PWM signals behave is determined in the PIC18F2431 configuration fuse settings:

```
// Configuration Fuse Settings for PIC18F2431
__CONFIG(1,IESODIS & FCMDIS & HSPLL);
__CONFIG(2,BORDIS & PWRTDIS & WDTDIS);
__CONFIG(3,PWMOUTEN & LOWPOLLO & HIPOLLO &
MCLREN);
__CONFIG(4,DEBUGEN & LVPDIS & STVREN);
__CONFIG(5,UNPROTECT);
__CONFIG(6,WRTEN);
```

To get a 20 MHz processor clock from the 5 MHz clock input, we must turn on the PLL fuse (HSPLL). The important PWM configuration data is contained in the third configuration word. As you can see in the configuration fuse declarations, we have chosen to enable the PWM module at startup (PWMOUTEN). Recall that our TC4428 drivers want to see a logical low at their inputs to activate

the associated MOSFETs. Thus, the 4428 inputs will need to be modulated with an active low PWM signal. Declaring LOWPOLLO forces the low side driver PWM signals to use a logically low active state. The same holds true for the high side driver PWM signals as we have declared HIPOLLO in the configuration fuse code. With an active low PWM signal, the PWM duty cycle is defined by the low-going portion of the PWM pulse. Earlier, we did the math to determine what value to load into the period register to obtain a 20 kHz PWM signal. The F2431's period register is actually a register pair (PTPERH and PTPERL). This little bit of code should get us a 20 kHz PWM pulse train:

```
PTPERH    = 0;
PTPERL    = 0xFA;          //250 decimal
```

Once we have the desired period dialed in, we'll need to be able to alter the PWM signal's duty cycle. To intelligently manipulate the duty cycle, we need to know how many bits of PWM resolution we have to work with. Here's the equation we need to determine the number of bits we can twiddle:

$$\text{Resolution} = \frac{\log\left(\frac{F_{OSC}}{F_{PWM}}\right)}{\log(2)}$$

Let's plug in the numbers for our 20 kHz PWM signal:

```
FOSC = 20 MHz
FPWM = 20 kHz
```

```
The log of FOSC / FPWM = 3.00
The log(2) = 0.301029996
Resolution = 9.96578 bits
```

We have 10 effective bits of resolution. That means we should be able to see duty cycle changes between the extremes of 0x0000 and 0x03FF. An easy check of our work is to load the duty cycle register with a value equal to half of our bit resolution span. We should get a 50% duty cycle output. However, before we can get anything out of the PIC18F2431's power control PWM module, we must configure it:

```
//*****
//*      CONFIGURE PWM
//*****
//set bit 7 when debugging
FLTCONFIG = 0b00000000;
PTCON0 = 0b00000000;
PTCON1 = 0b00000000;
PWMCON1 = 0b00000000;
PTPERH = 0x00;
PTPERL = 0xFA;          //250 decimal
pwm_control(ON);
```

To help prevent the unintentional release of the 9952A MOSFET pairs' magic smoke during debugging, it is recommended to set the BRLEN bit in the FLTCONFIG register. Setting the BRLEN bit will force the PWM outputs to their inactive states (logically high, in our case) when a breakpoint is encountered. Otherwise, an active state on

the 4428 inputs may hold a MOSFET in the ON state long enough to destroy it. Remember, we're dealing with BLDC motor phase windings that are approaching dead shorts. The BRFFEN bit is clear in our code because we don't have a BLDC motor attached and we are not feeding the three-phase driver circuitry its diet of 12 volts at this time.

The PWM timer control register 0 (PTCON0) is loaded with its default values. The contents of the PTCON0 register indicate that we are using a 1:1 postscale and a 1:1 prescale. Thus, our PWM time base input frequency is FOSC/4 or 5 MHz. The least significant pair of bits in the PTCON0 register is set for free-running mode. In free-running mode, the PWM time base registers (PTMRH and PTMRL) count upward until the incremented count within them matches the value loaded into the PWM period register pair (PTPERH and PTPERL). When the match occurs, the PTMR registers reset. If the PTEN bit is set, the upwards count continues and the process of seeking a matching count continues. The PTEN bit is the most significant bit of the PWM timer control register 1 (PTCON1). We set and clear the PTEN bit within the pwm_control function:

```
//*****
//*      PWM CONTROL
//*****
void pwm_control(char cmd)
{
    switch (cmd)
    {
        case ON:
            PWMCON0 = 0b01011110;
            PTEN = 1;
            break;

        case OFF:
            PTEN = 0;
            PWMCON0 = 0b00001110;
            PORTB = 0xFF;
            break;
    }
}
```

The PWM control register 0 (PWMCON0) contains bits that selectively enable and disable the PWM outputs. The ON value we have loaded into the PWMCON0 register enables all six of the F2431's PWM outputs, as well as the PWM time base timer. Calling the pwm_controller function with an OFF parameter disables the PWM time base timer and the PWM module. When the PWM module is disabled, it no longer has control over PORTB. So, to avoid the possibility of destroying the three-phase driver circuitry, all of the 4428 inputs are logically instructed to turn off all of the three-phase driver MOSFETs (PORTB = 0xFF). The least significant byte of the PWMCON0 register value puts the PWM outputs into independent mode. In independent mode, the PWM output pairs don't complement their output waveforms. In that we called the pwm_controller function with an ON parameter, we can load the duty cycle registers with a 50% duty cycle value of 0x01FF to verify our duty cycle math. Each duty cycle register pair is responsible for a pair of PWM outputs:

```
PDC0H = 0x01;           //PWM0 and PWM1
PDC0L = 0xFF;
PDC1H = 0x01;           //PWM2 and PWM3
PDC1L = 0xFF;
PDC2H = 0x01;           //PWM4 and PWM5
PDC2L = 0xFF;
OVDCOND = 0b00100000;   //PMW5 active only
OVDCONS = 0b00000000;
```

The key to commutating our BLDC motor are the output override control (OVDCOND) and output state (OVDCONS) registers. By coding the bit pattern you see in the duty cycle code snippet, I have enabled PWM5 and forced all of the other PWM outputs to an inactive state (logically high). We'll use the override registers to selectively enable and disable PWM signals in the desired BLDC commutation sequence.

I've included a complete copy of the PWM code we've written thus far in the download package on the *SERVO* website (www.servomagazine.com). As a result of running the PWM code we've assembled so far, the CleverScope capture of the PWM signal you see in Screenshot 1 has a period of 50 μ s, which equates to a frequency of 20 kHz. Splash one! Our duty cycle math is on the ball, as well. The duty cycle of the PWM signal in Screenshot 1 is within a microsecond of 50%. Splash two!

BLDC Firmware Is In Your Future

When we reconvene, we'll exercise the PIC18F2431's A-to-D converter module and get acquainted with the current sensing circuitry. I've included an interim ExpressPCB printed circuit board (PCB) layout file in the download package along with the code we discussed. Once the hardware is all checked out, we'll nail down the PCB layout and write some code to spin the BLDC motor shaft without any help from our fingers. **SV**

Fred Eady can be reached via email at fred@edtp.com.

SOURCES

CleverScope
Saelig
www.saelig.com

PIC18F2431
MCP6002
MCP6542
TC4428
Microchip
www.microchip.com

NDS9952A
Fairchild Semiconductor
www.fairchildsemi.com

BLY171S-24V-4000 BLDC Motor
Anaheim Automation
www.anaheimautomation.com

The code for the BLDC motor driver is compiled using HI-TECH PRO for the PIC18 family.
H-TECH Software
www.htsoft.com

Software Platforms For Service Robots

by MICHAEL SOMBY

What is special about service robotics as compared to industrial robotics? Imagine it gets dark in the living room and you turn on the light. For most industrial robots, such a change in the illumination pattern would be a catastrophic event. Those robots are designed to work in closely controlled industrial settings. Their computer vision systems are not able to cope with such changes in the illumination without reconfiguration or reprogramming. On the other hand, interacting with the dynamic and changing world is a survival requirement for most service robots.

New pieces of furniture, random obstacles, walking people, missing or unknown objects, unprofessional users, children, pets, cars on the road — those features comprise the dynamic environment where a service robot needs to be able to operate. Besides that, most service robots are mobile and work in close proximity to people.

The unique requirements of service robotics triggered the creation of an industry which develops new technology specifically designed for service robotics applications.

This article reviews several of the software products available on the market for this particular industry.

Introduction

Building a robot which makes up rooms, cleans houses, or serves breakfast is the dream of every service roboticist.

A company which delivers such a product at a reasonable price will most likely make huge sales.

The military looks for robots which can be used to fight wars; farmers would love robots to grow crops. Nearly every industry has a use for service robots. Examples include:

- Robotic Elder care
- Automatic Health Scanning and Monitoring (healthcare)
- Robotic Surgery (healthcare)
- Robotic Delivery (hotels, hospitals, restaurants, offices)
- Robotic Crop Picking (agriculture)
- Unmanned Convoys (defense)
- Bomb Disposal (defense)
- Security, Patrols
- Entertainment (home, museums, parks)
- Cleaning
- Mining
- Fire Fighting

There are so many potential applications, and the good news is that the technology is getting there.

Service Robots of Today

Although service robotics is a relatively young industry, it has gone through various iterations of technology improvement.

Teleoperation was probably the first step in the evolution of practical service robots. Many of today's deployed service robots still have all their motions initiated and controlled by a human operator. The military robots deployed in Iraq and Afghanistan, successful robotic surgery robots, and healthcare telepresence robots are good examples of these types of robots.

The introduction of

Healthcare telepresence robot (InTouch Technologies, Inc.).



autonomous navigation and obstacle avoidance took service robotics to the next level. Indoor navigation came first, followed by outdoor self-driving technology.

Indoor navigation technology relies heavily on laser range finders, infrared/ultrasound obstacle detectors, touch sensors, computer vision algorithms, and occupancy maps. The outdoor robotic driving technology was propelled by the three DARPA robotic car races. These

competitions triggered creation of several technologies such as LADAR data processing and associated computer vision algorithms.

Today, a growing number of companies offer autonomous indoor service robots and outdoor unmanned vehicles (UVs) including commercially successful indoor robotic vacuum cleaners, warehouse robots, robotic lawn mowers, delivery robots, agricultural robotic vehicles, and robotic car strap-on kits.

The military unmanned vehicles are rapidly becoming multi-purpose, reconfigurable platforms which can be fitted with various mission-specific payload modules. Depending on the mission, the UVs can carry payloads which detect intrusions, detect enemy fire, detect sniper scopes, attack enemies, or check RFID tags attached to controlled items. Autonomous mobile robots can be programmed to work together with other robots or humans as a team.

The best example of such an approach being implemented on a massive scale is Future Combat Systems (FCS), one of the Pentagon's ambitious modernization programs. FCS defines how a number of manned and unmanned platforms carrying various payloads will work together in the future network-centric battle space.

Common architectures (4D/RCS), mission planning, interpretability of the systems (JAUS, STANAG 4586), smart sensors/payloads, data sharing, and exchange (C4I) are some of the areas which are being addressed.

Although the FCS program is still referred to as a program of the "future," it turns out that it's harder to build a robot which can reliably fetch a beer from a refrigerator in a typical home than building an unmanned combat vehicle which can search for, identify, and blow up a SCUD missile launcher.

Service robotics technology had to move beyond autonomous navigation by



Warehouse robots (KIVA Systems).



Indoor mobile robots (MobileRobots, Inc.).

introducing learning and social service robots — the next iteration of the technology.

This learning technology was the key enabler which allowed robots to intelligently and autonomously utilize robotic arm manipulators in dynamic environments.

The latest versions of learning control systems can discover object properties, interact with humans, and even guess a human's intentions. The software which guides learning and social robots is based on a new principle which is a dramatic departure from the traditional approaches as it enables the robots to learn new missions/behaviors directly from humans without reprogramming. Instead of defining what the robot needs to do, the software defines how the robot will learn what it needs to do.

Imagine you are a lucky guy who just got a service robot. You want the robot to make up your room every day, pick oranges in your garden, or palletize boxes in your garage. As the robot has just arrived at your place, you would expect the robot to learn from you all the things you want it to do. The scenario has become a reality with advances in commercial control systems specifically designed for learning service robots and the multitude of research projects in the domain. Robot training is somewhat different from programming as it doesn't require technical or programming expertise from the trainer. That's



Strap-on car autonomy system (Kairos Autonomi).



Service robot training (Skilligent LLC).

why the learning robots need to be social — they have to be able to engage humans directly by initiating hardwired reactions in human brains.

At first glance, learning and social robots might not look as cool as military UVs. Nevertheless, the level of autonomy of the service robots must be significantly higher; it took a major breakthrough in technology in order to build a robot which can intelligently utilize its arm or learn what gestures and facial expressions to use in order to communicate with a human.

Now that we've looked at the evolution of service robotics, let's begin our overview of the state-of-the-art robotic software products. These products can be considered a productized source of robotic intelligence.

Software Robotic Platforms

Today's service robotics market is often compared to the early PC market. Several competing robotic software platforms

emerged on the market that were mostly incompatible (sound familiar?). Table 1 provides a summary of the relevant features of several (but not all) of these most prominent players.

The feature products can be split into two base categories: pure software frameworks and off-the-shelf robotic brains.

Products in the "purists" category included Microsoft, Gostai, OROCOS, and Player. They all provide runtime environments, hardware abstraction layers, development and debugging tools, and simulation environments to users who are actually building robotic brains, using the platforms as programming frameworks.

Products from the second category (MobileRobots, Skilligent) represent complete robotic intelligence systems which can be installed on a robot to make it autonomous. These control systems are built for the end user rather than for programmers.

Products that are "in between" (Evolution Robotics,

iRobot AWARE) require application-level/mission-level programming to turn them into complete intelligent systems.

A few software architectures, programming patterns, and methodologies have been proven successful when applied to robotic control systems. Purists claim that their software platforms /frameworks combine all the best by providing the simplest, most scalable, or most proven *foundations* for building robotic control systems while leaving the job of actually building the systems to the end users. In contrast, vendors of robotic intelligence systems provide complete solutions which include everything a robot will need to do its job autonomously, including the major pieces of the puzzle such as robot learning, social interaction, autonomous

Table 1

	Microsoft Robotics Studio .5	Microsoft Robots	Skilligent	iRobot AWARE 2.0	Gostai Ubu	Evolution Robotics ERSP 3.1	OROCOS	Player, Stage, Gazebo
Open Source	No	No	Yes	No	Partial	No	Yes	Yes
Free of Charge	Edu/hby	No	No	No	Selected platforms	No	Yes	Yes
Windows	Yes	Yes	Yes	???	Yes	Yes	No	Yes (simul. only)
Linux	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Distributed Services Architecture	Yes	No	Yes	Yes	Yes	No	No	Yes (limited)
Fault-Tolerance	No	No	Yes	Yes	No	No	No	No
JAUS Compliant	No	No	Yes	Yes (???)	No	No	No	No
Graphical OCU	Yes (Web)	Yes	Yes	Yes	Yes	Yes	No	No
Graphical Drag-and-Drop IDE	Yes	No	No	No	Yes	Yes	No	No
Built-in Robotic Arm Control	No	Yes	Yes	Yes	No	No	Yes	No
Built-in Visual Object Recognition	No	No	Yes	No	No	Yes	No	No
Built-in Localization System	No	Yes	Yes	No (pieces only)	No	Yes	No	No
Robot Learning and Social Interaction	No	No	Yes	Yes	No	No	No	No
Simulation Environment	Yes	Yes	No	Yes	Yes (Webots)	No	No	Yes
Reusable Service Building Blocks	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No
Real-Time	No	No	No	No	No	No	Yes	No

navigation, and computer vision systems.

So, depending on what you want (work on the robotics intelligence yourself or just add an off-the-shelf robotic brain to your robot), you can decide which product you'll need.

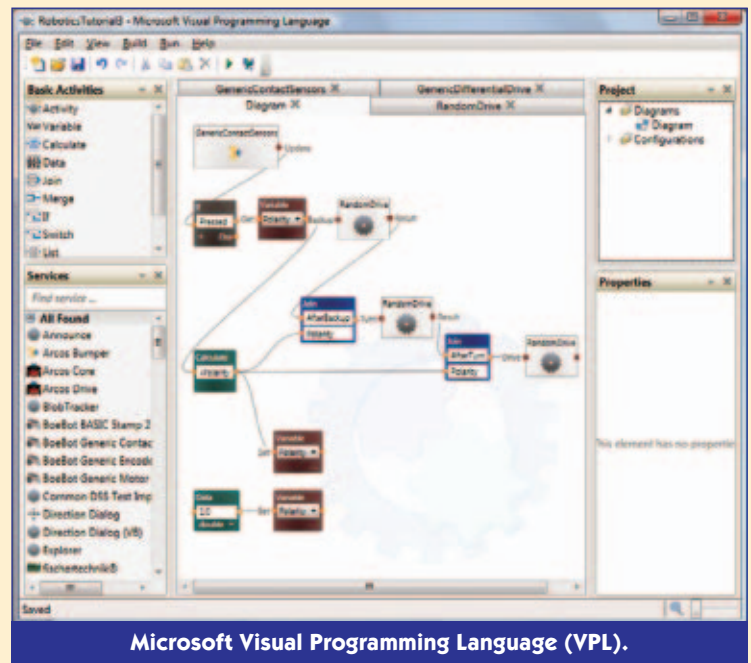
Products like Skillogent software can run as a standalone system or can be integrated on the platform of your choice.

Here are some observations to consider:

- Two of the systems (OROCOS and Player/Stage) are free and open source.
- Most systems support both Linux and Windows. The major exceptions are Microsoft Robotics Developer Studio which is a Windows-only platform, and OROCOS which is a Linux-only system.
- iRobot AWARE 2.0 and Skillogent support fault-tolerance. These software packages are suitable for building redundant vehicle control systems which can tolerate major electronics failure like a broken computer or a faulty sensor.
- JAUS is supported by Skillogent.
- Most systems support distributed services architectures which simplify putting together robotic applications from large, loosely coupled software components potentially running on networked computers.
- Skillogent is the only vendor whose system can be trained by an unprofessional user through a robot learning and social interaction method.
- MobileRobots offers one of the most mature indoor navigation, obstacle avoidance, and mapping systems which relies heavily on laser range scanners carried by the robot.
- Built-in indoor navigation/localization capabilities are present in the MobileRobots, Evolution Robotics, and Skillogent systems.
- Built-in visual object recognition capabilities are provided by the Evolution Robotics system (SIFT algorithm) and the Skillogent system (improved Harris Corner Detector algorithm). The rest of the vendors offer third-party software for these purposes.
- Graphical OCUs (Operator Control Units), drag-n-drop graphical development environments, simulation environments, and libraries of reusable service building blocks are present in most systems in one form or another. Those features are becoming the "must haves" for all robotic software platforms.
- MobileRobots and iRobot created their software products primarily for use with their own hardware robotic platforms. Although the software is specifically built, it is designed in a hardware-agnostic fashion to simplify integration with new robots.

Microsoft Robotics Developer Studio 1.5

Microsoft Robotics Developer



Studio (MSRS) is an example of a purist approach. It provides a runtime environment, a graphical drag-n-drop service creation environment, and a simulation environment, but does not have its own built-in intelligence components such as a computer vision system, a navigation system, or a robot learning system. Instead, it relies on a growing number of partners, who offer the "intelligence content" for the framework.

The runtime environment (which is a foundation of the platform) is called DSS (Decentralized Software Services). It connects together software modules called services running locally or on the network (REST). For example, a navigation system could be treated as a service, and a computer vision system would be a different service. The services are loosely coupled and typically don't know about each other's existence.

An application-specific code orchestrates those services, sends queries, and receives responses back. It controls the robot that way. If a service is located on a remote server, a patented network protocol DSSP is used to communicate to the server.

As all the services are running in parallel, there is a chance that the application-level orchestration code may get confused by multiple responses simultaneously coming from various services. The answer to the problem is a special library called CCR (Concurrency and Coordination Runtime), which provides a consistent and scalable way to program asynchronous operations and coordinate among multiple responses. DSS makes use of CCR. The runtime environment comes with a graphical programming language called VPL which is used to orchestrate multiple





Robots manufactured by MobileRobots.



Architecture of MobileRobots' software.

asynchronous invocations of decentralized services managed by the DSS/CCR framework. VPL is actually domain-independent and can be used outside of the robotics domain.

The robotics-specific portion of the Microsoft robotic platform consists of a robotic simulation environment, a set of robotics tutorials with sample code, and a nice online courseware package. The platform also defines a set of APIs (Application Programming Interfaces) for standard robotic services such as a web camera. APIs are a form of hardware abstraction layer (HAL).

Originally greeted with a mixture of enthusiasm, caution, and skepticism by the world of robotics researchers and hobbyists, the platform turned out to be overkill for small hobby/research projects. Writing DSS services and connecting them to a platform is a major exercise in programming which can be an unnecessary distraction from the final goal of a project. In the end, one still has to look for a third-party library which would handle object recognition for example, as the platform doesn't have any built-in "intelligence" as previously mentioned. On the other hand, if

collector, a source of random delays. How well the IT-style architecture of the Microsoft platform fits the real-time (or even "soft" real-time) domain is yet to be fully revealed.

Several companies and teams use the Microsoft platform with success. Robosoft, a company building autonomous robots, uses the Microsoft platform as a software framework for all of their robots. Coroware uses it as a programming and simulation environment for their CoroBot robots built for the research and education market. Princeton's Urban Grand Challenge team used MSRS as a software framework for a control system built for their unmanned ground vehicle.

MobileRobots

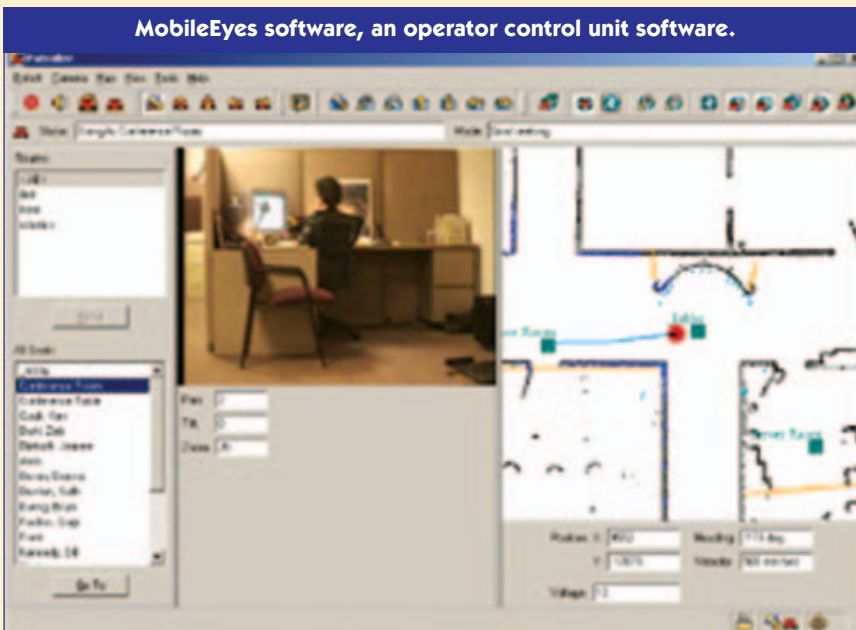
MobileRobots produces autonomous robots which are the de-facto standard robotic platforms for researchers around the globe. The company also diversified into service robotics by introducing delivery robots, security patrol robots, and telepresence robots.

Besides designing the nice hardware, autonomous

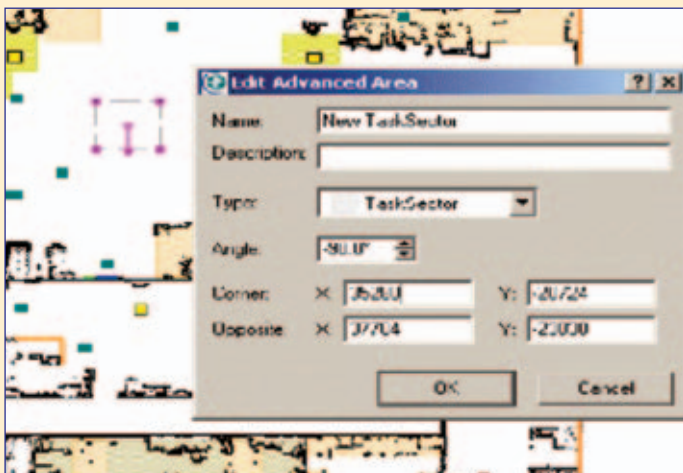
indoor navigation technology is the core competency of MobileRobots. The company has developed a suite of technologies including both software and sensors which enable the mobile robots to reliably navigate and explore various indoor (or indoor-like) environments. As a result, the software platform demonstrates great performance when it comes to autonomous indoor navigation.

The API of the platform is called ARIA (Advanced Robotics Interface Application). The core software package is called Autonomous Robotic Navigation and Localization (ARNL). It enables the robots to navigate both indoor and outdoor environments with high precision. The software package makes heavy use of laser range scanners, encoders on wheels, and (optionally) inertial measurement units (IMU), video cameras, and GPS sensors (outdoor).

An OCU software package called



MobileEyes software, an operator control unit software.



Defining a new task for a robot in MobileRobots' TrainingFactory.



A robot controlled by Skilligent software uses its arm to grab an object from a trainer's hand.

MobileEyes is a part of the platform. The software allows wireless control. You click on a map to send a robot to a particular destination. A simulation environment, MobileSim, is based on an open-source Stage simulator (discussed next). The platform comes with three computer vision software packages. A package called Advanced Color Tracking Software (ACTS) allows color tracking over a wider variety of lighting conditions. The other packages include omni-vision dewarping and stereo-vision range-finding software. The platform does not have its own graphical drag-n-drop service creation environment like other products, but a software program called TrainingFactory allows setting up typical task/mission plans (e.g., patrolling or delivery) in an intuitive way without programming.

trained enough to do its job autonomously.

Skilligent developed a computer vision system capable of recognizing objects under real life conditions from a large database of known objects. The same vision system recognizes gestures used for robot training and provides multiple bottom-up visual attention signals.

The bottom-up visual attention system is capable of selecting objects which might be relevant to the current task being learned. The object representations are then stored in an image database integrated with a database of known skills and behaviors.

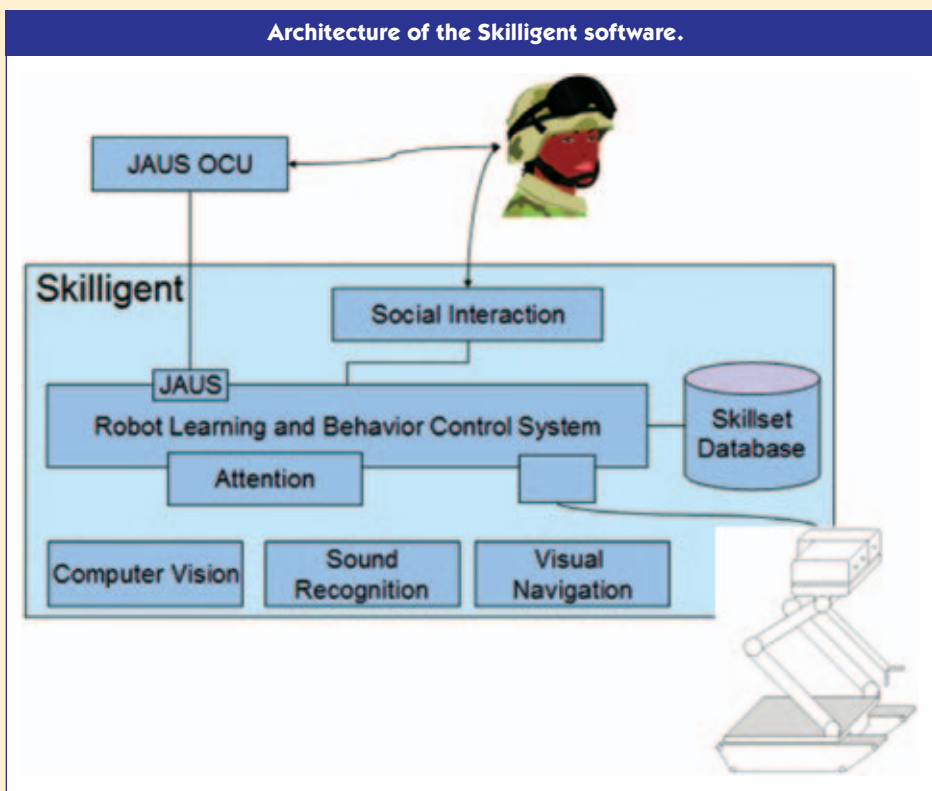
A visual robot navigation system (part of the product) allows a robot or a UAV to navigate in known environments. The environments are explored and

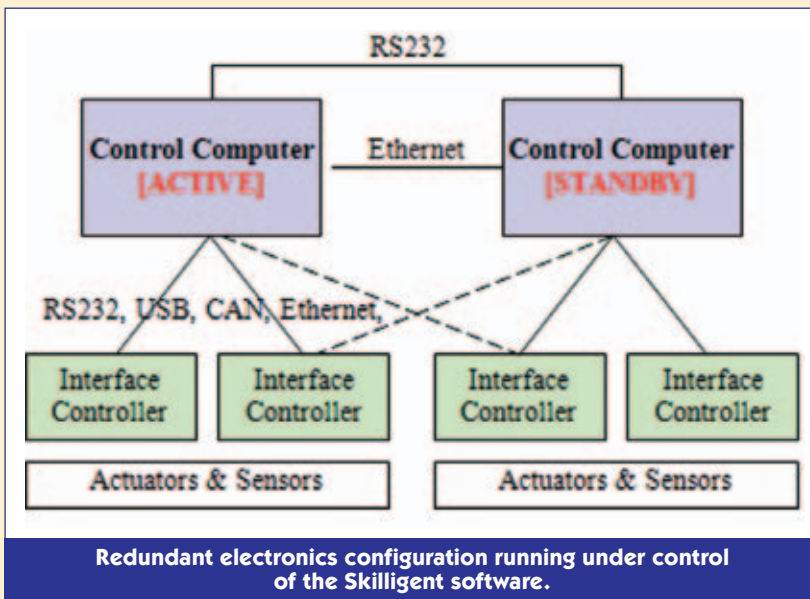
Skilligent

Skilligent is a leader in commercial robot learning and social interaction technology. Their product — called Robot Learning and Behavior Control System — is a complete robotic intelligence system for giving multi-task service robots the capability of utilizing robotic arm manipulators.

The philosophy of the product is to eliminate programming by introducing a fully trainable control system. After integrating the software into a robot (this still might require programming), a user starts communicating to the robot via gestures, attracting the robot's attention to objects or places, and invoking "reflexive behaviors" through known stimuli. By interacting with the robot and giving demonstrations, the user can develop new behaviors and new stimuli in the robot. The iterative training process keeps going on until the robot is

Architecture of the Skilligent software.





input causes the task learning algorithms to update the internal generalized task representation. The iterative process goes on until the robot learns. In technical terms, this means that the generalized task representation causes the robot to behave in a way which matches the expectations of the trainer.

This iterative process could be called robot programming by demonstration, but as the user doesn't have to know what is really going on in the robot's software, the term "programming" might not be fully appropriate. Instead, Skilligent uses the term "robot training" which underlines the fact that the user doesn't really have to know how the software works in order to be able to successfully train the robot. This quality ensures that a non-professional user (such as a warehouse worker) can successfully train the robot. The Skilligent software product is designed for multi-task service robots. This

mapped out during a training session. There is also a way to create digital maps from existing images of a territory (e.g., for aerial image-based navigation). The navigation system uses a sequence of observed visual landmarks for position triangulation. Between the observations, the system uses a dead-reckoning mechanism to estimate the robot's position.

A combination of the generalization algorithms is the key to the robot learning technology. For example, during a robot training session, a robot learns from a human how to perform a task. The task learning algorithms build a generalized task representation as a sequence of steps, decision points, conditions, and hierarchical skills. The generalized representation is totally hidden from the end user. The only way for a user to check that the robot's internal task representation is correct is to actually instruct the robot to perform the task autonomously. While observing how the robot acts autonomously, the trainer checks if the generalized task representation is correct (the robot has learned what it needs to do). If not, the trainer can interrupt the robot at any time and make additional demonstrations. The additional

means that the robots can be adapted to new tasks without reprogramming. The robot learning technology is the key factor which makes it possible.

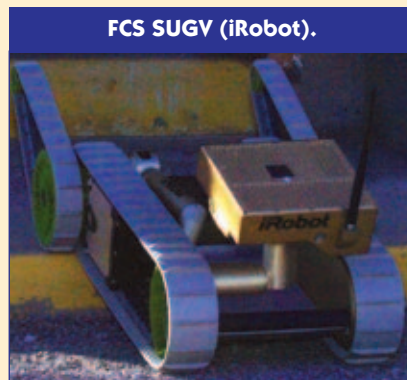
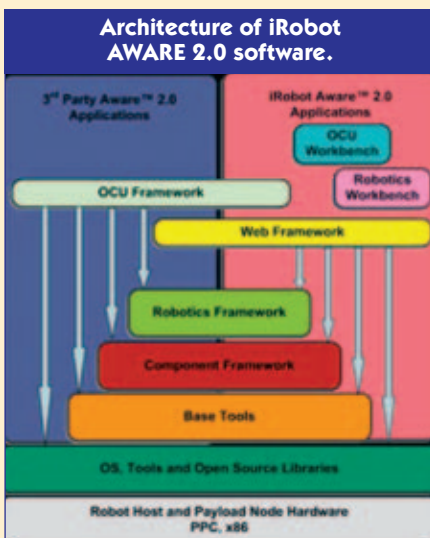
The software relies on multiple artificial intelligence algorithms working in parallel. It requires a pretty powerful CPU (such as Intel Core Duo) to allow the robot to work on the same timescale as the human trainer. This implies that a robot which uses the software has to be controlled by a laptop-class motherboard or a wirelessly connected desktop computer.

The second major product of Skilligent is called a Fault-Tolerant Control Framework — a JAUS-compliant "pure" robotic software platform targeted at fault-tolerant vehicle control systems. The platform is said to be originally designed as a software framework for highly redundant avionics and communications systems.

The platform has a built-in mechanism for handling hardware and software faults including failures of control computers and sensors. The system has a built-in distributed shared memory facility which automatically replicates critical mission state information from an active computer to one or more standby computers. In addition, the distributed shared memory synchronizes memory caches across a cluster of control computers. A low latency UDP-based protocol is used for data synchronization. The software platform is optimized for building reliable distributed control systems.

Realizing that a fault-tolerant platform might be overkill for many service robotics projects which don't require redundancy and high reliability due to cost limitations, Skilligent created a lightweight version of the framework with most fault-tolerance features disabled. The lightweight platform is included in the Robot Learning and Behavior Control System (the "artificial intelligence package").

Skilligent integrated its Robot Learning



The Skilligent product is the closest shot to the goal of building a complete (software) robotic brain which comes with everything needed to add an off-the-shelf intelligence to a service robot.

iRobot AWARE 2.0

AWARE 2.0 is a software platform developed for a range of military robots produced by iRobot.

Many of these robots are already deployed with the US Army, law enforcement agencies, and foreign militaries. iRobot encourages third party developers to build add-ons to their robots, however, any payloads or technology must be able to integrate with AWARE. (License fees may apply.)

There are two versions of the platform: an "Integrator version" supports Python scripting language as the only means of creating robotic applications — primarily as a glue code to test a new payload. This version costs less and is useful for quick proof-of-concept applications using simple Python programming. The full version — called Enterprise edition — provides a full-blown C++ development environment aimed at building high-performance, fully-integrated applications. The development environment claims to be the same one used internally by the iRobot engineering team.

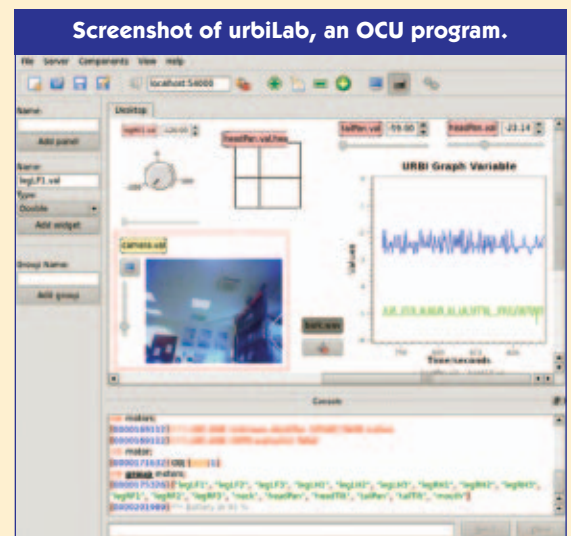
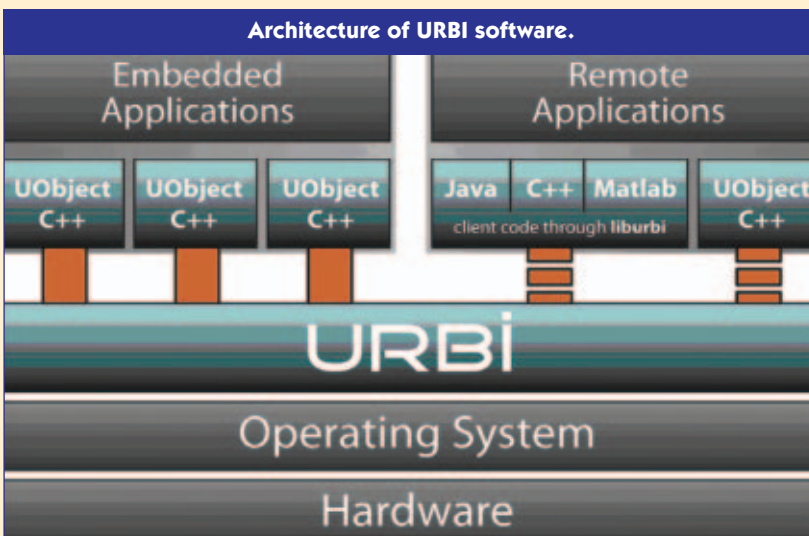
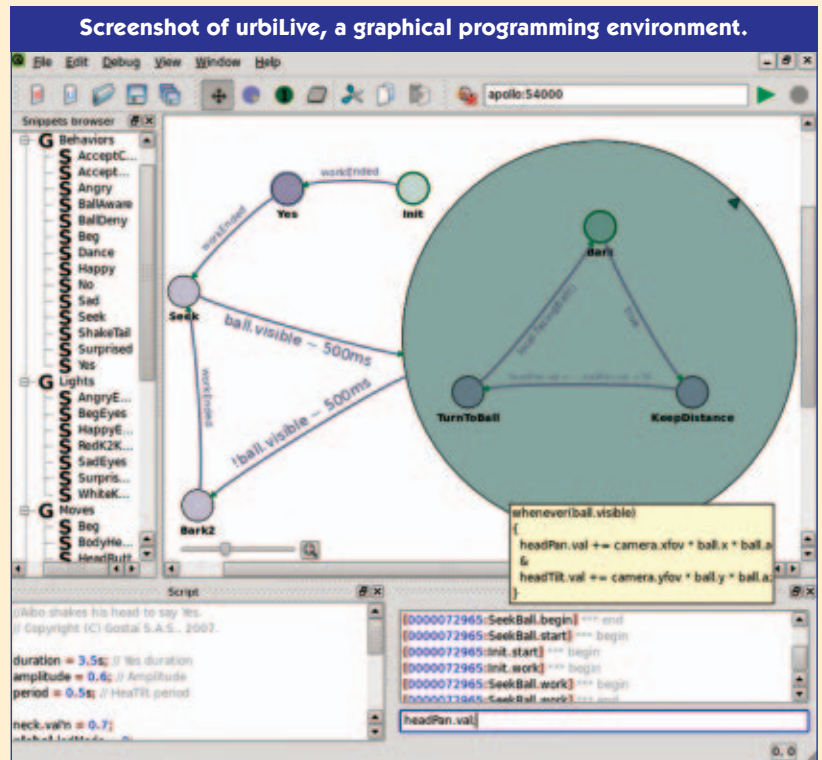
The platform comes with a simulation environment and is integrated with operator control units (OCU) sold by iRobot. AWARE 2.0 doesn't appear to have a graphical drag-and-drop development environment. It does have a built-in data messaging system designed around a publisher-subscriber model. Interested software modules running somewhere in the onboard network can subscribe to receiving updates

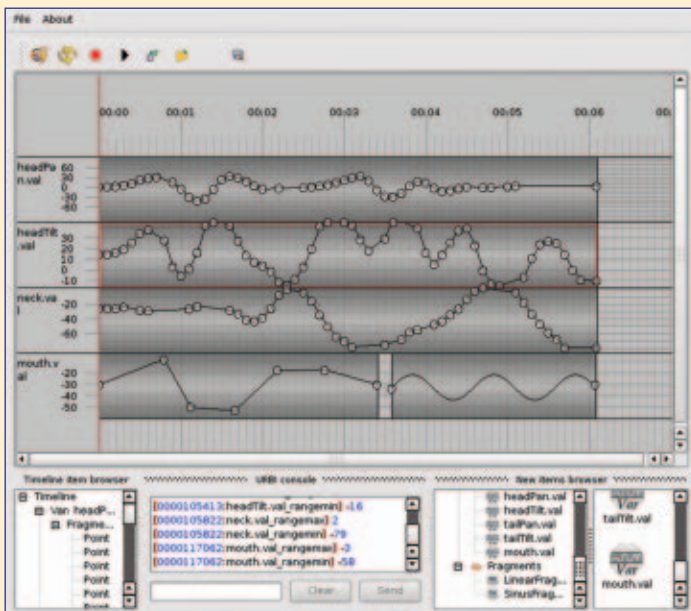
coming from a particular publisher, say for example, a sensor. The multicasting nature of the messaging system allows building process-level, fault-tolerant architectures.

The software platform comes with many built-in functions including self-collision avoidance, inverse kinematics, and remote control from an OCU. Other advanced AI functions such as computer vision or autonomous navigation are outside the scope of AWARE and are provided by external payload modules. iRobot offers a clear path to revenues for third-party developers.

Gostai Urbi

Gostai's robotic software platform Urbi includes a runtime environment, a nice set of development tools, an



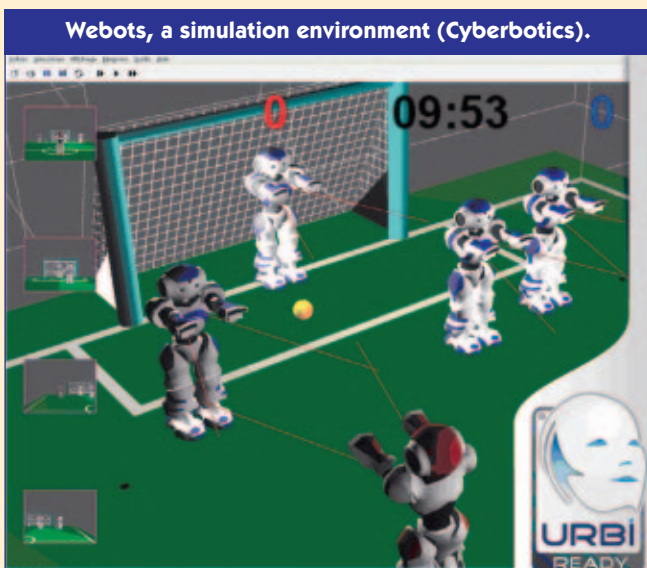


Screenshot of urbiMove, an animations editor.

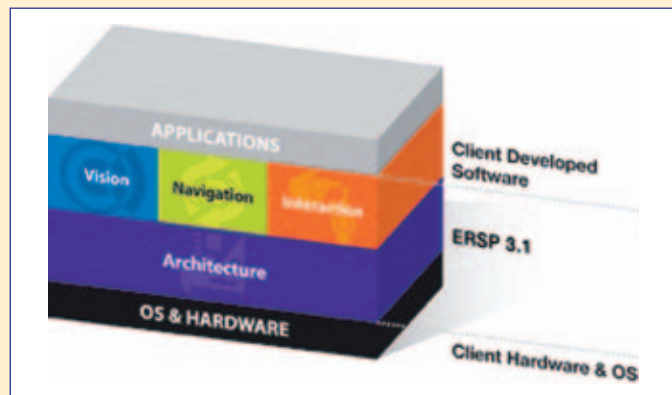
OCU program, and a hardware abstraction layer for several robots. One of the great features of the platform is its accompanying set of graphical development tools called Urbi Studio which simplifies the creation of robotics applications. The tools are called urbiLive, urbiMove, and urbiLab.

The centerpiece is urbiLive – a graphical drag-and-drop environment which helps design behaviors. The tool converts graphical behavior diagrams into urbiScript, a scripting language interpreted by Urbi's runtime engine. urbiMove is used to graphically design animations, dances, and postures for various robots. Movements can be recorded and replayed later. urbiLab is a customizable graphical OCU for robots running Urbi.

Urbi doesn't have its own simulation environment. Instead, it relies on a popular commercial simulation environment called Webots sold by Cyberbotics.



Webots, a simulation environment (Cyberbotics).



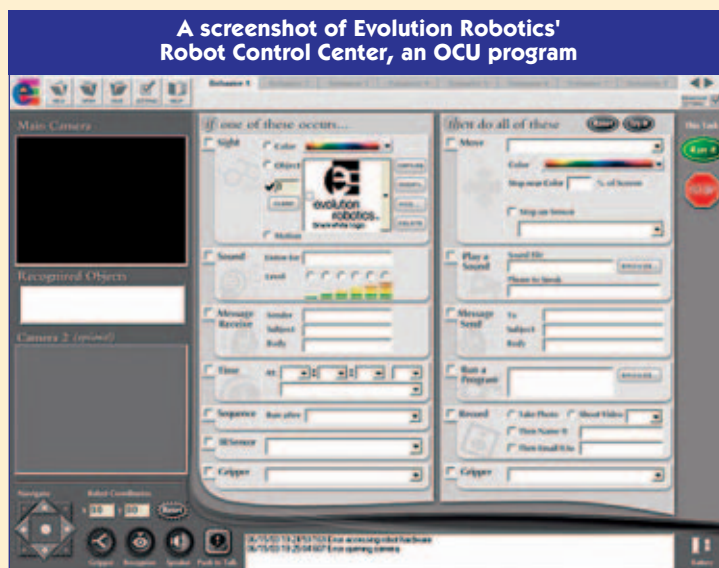
Architecture of ERSP 3.1 software.

Gostai's novel scripting language called urbiScript features dedicated abstractions to handle parallelism and event-based programming from within C++, Java, or MatLab. A distributed component architecture called UObject is used to create robotic applications from high-level service building blocks potentially running on networked computers. The runtime engine orchestrates these loosely coupled services.

Urbi is a pure software platform which provides a clean programming framework for engineers and researchers. The intelligence components such as computer vision, robot learning, navigation, and object recognition are provided by Gostai's partners.

Evolution Robotics' ERSP 3.1

Evolution Robotics was one of the first companies to introduce a commercial implementation of a generic software robotic platform. Their product – called ERSP – combines a cross-platform runtime environment, a graphical drag-and-drop development environment, a visual object recognition system (ViRP), and a vision-based localization and mapping (vSLAM) system. The computer vision system and the vSLAM system are based on the SIFT algorithm,



A screenshot of Evolution Robotics' Robot Control Center, an OCU program

invented by David Lowe.

A recent innovation from Evolution Robotics is called NorthStar which is a robot localization system based on an infrared beacon technology.

The platform comes with a graphical toolkit which can be used for building programs based on a number of reusable software building blocks. The building blocks called behaviors are activated/deactivated in runtime by a higher-level program called a task. ERSP runs on both Windows and Linux. Besides the graphical toolkit, ERSP supports development of programs in the Python scripting language.

A program called Robot Control Center allows defining missions/tasks for the ER1 personal robot as sequences of IF-THEN statements. Besides that, the application allows remote control.

ERSP does not come with a simulation environment and does not support distributed services architecture. Although ERSP is a mature system, it doesn't seem to be an actively developed product as the company hasn't made new releases in a while.

Evolution Robotics owns the patent on robotics-based hardware abstraction layer (HAL). This permits robot control software to be written in a robot-independent manner.

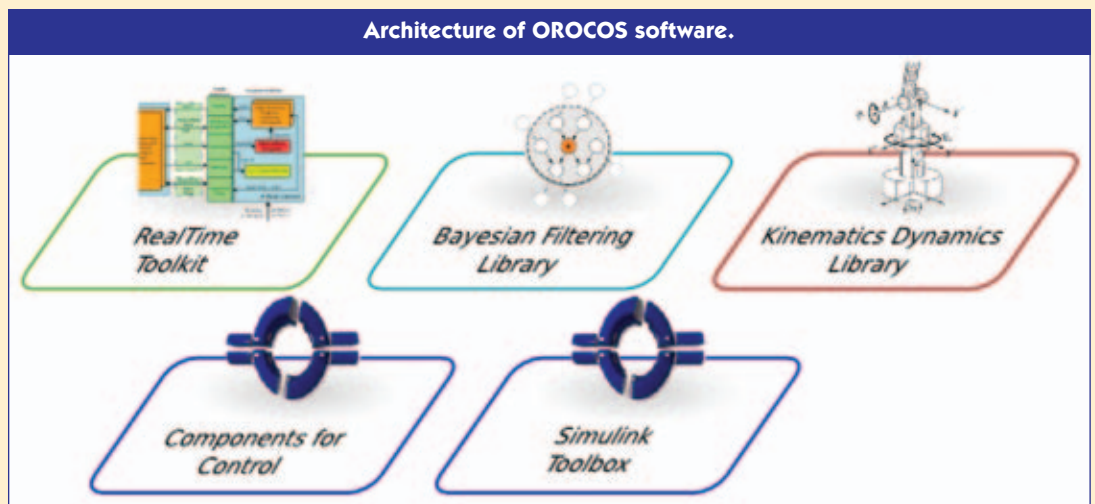
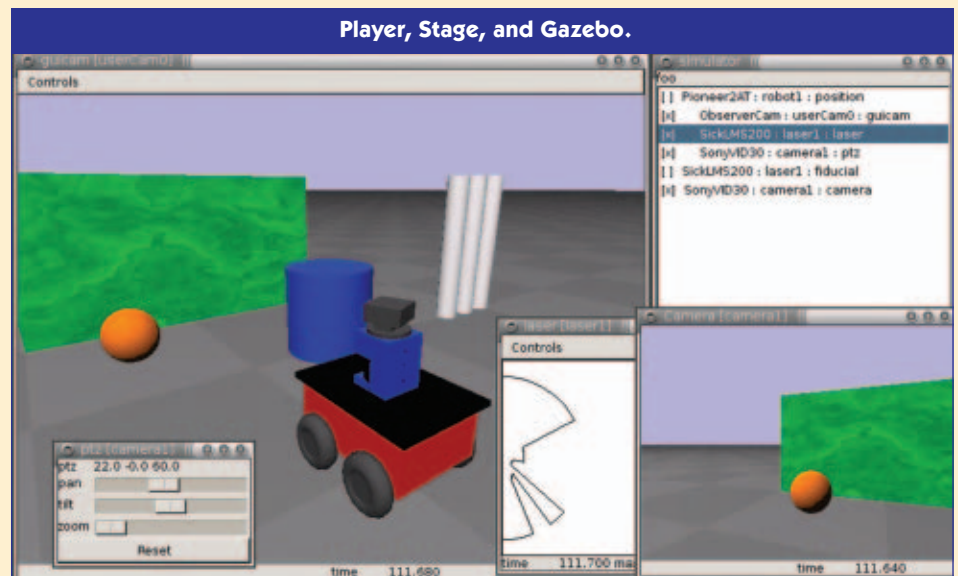
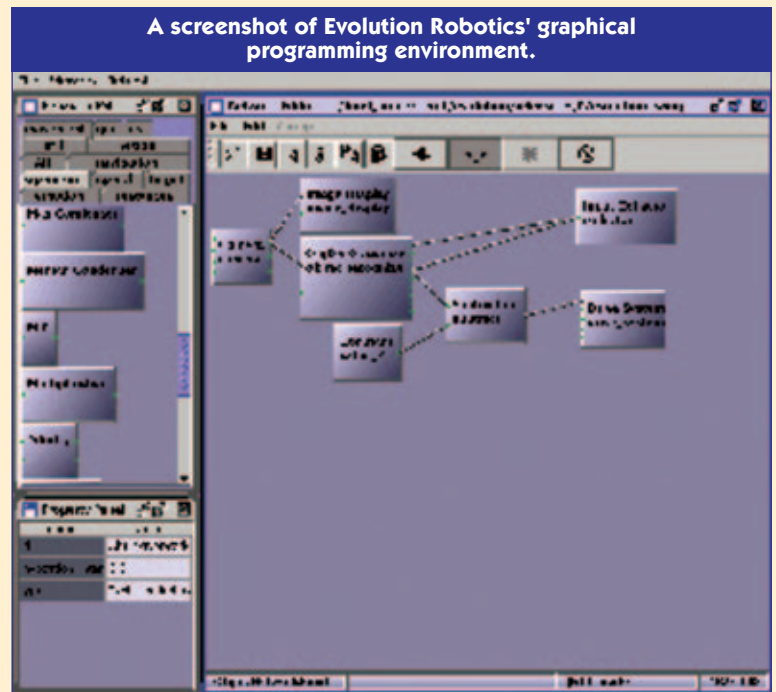
OROCOS

OROCOS is an open source set of libraries for advanced motion and robot control. It comes with its own runtime environment optimized for real-time applications. The environment uses lock-free buffers which makes it suitable for time-deterministic, real-time applications.

OROCOS comes with a set of reusable components and a set of drivers for selected robotic hardware.

A kinematics and dynamics library is an application-independent framework for modeling and computation of kinematics chains such as robots, biomechanical human models, computer-animated figures, and machine tools. OROCOS doesn't come with a graphical drag-and-drop development environment or a simulation environment.

The Berlin Racing Team of the 2007 Urban



Grand Challenge Competition used the OROCOS Real-Time Toolkit as the framework for building their vehicle control system. They were one of the semifinalists selected by DARPA.

Player, Stage, Gazebo

Player is a popular open source TCP/IP-based hardware abstraction layer for a growing number of robotic hardware modules. Stage and Gazebo provide accompanying simulation environments. The environments support multi-robot simulations which are suitable for research in swarm- or team-working of applications.

When running, Player provides a simple interface to the robot's sensors and actuators over the IP network for reading data from sensors, writing commands to actuators, and configuring devices on the fly. A number of open source software modules have been developed for Player including computer vision and navigation/mapping modules.

Other Platforms

There are several other robotic software platforms available on the market, such as CLARAty, YARP, Pyro, and Edinburgh Robotics' DevBot. I haven't had a chance to study these platforms closer, but here is some general information:

- CLARAty is a software platform built by NASA and then released as an open source project. The software seems to provide a set of interesting algorithms such as visual heading and pose estimations, and visual wheel sinking estimation.
- Edinburgh Robotics' DevBot is a robotic software platform which simplifies development of distributed robotic control systems by providing a unified distributed hardware abstraction layer.
- Pyro (Python Robotics) is an open source Python-based robot programming environment. It comes with a nice curriculum, so it can be used for classroom robotics.
- YARP ("Yet Another Robot Platform") is an open source set of libraries, protocols, and tools to keep modules and devices cleanly decoupled.

Importance of JAUS Standard

The US Department of Defense (DoD) is the world's largest single buyer of service robots. JAUS is mandated for use by all of the programs in the DoD Joint Ground Robotics Enterprise (JGRE). This basically means that if a company wants to sell robots to the DoD, the robots must be JAUS compatible. (Even if you aren't planning to sell your robots to the DoD, initially, it probably makes sense to support JAUS, just in case.)

JAUS stands for Joint Architecture for Unmanned Systems. Although it is called an "architecture," it can be treated as a UDP/RS-232 binary protocol primarily used today for communication between mobile ground robots

and their OCUs. Also, JAUS can be used as a standard interface for connecting new payloads to reconfigurable robotic platforms. Finally, it can be used as a protocol for internal communication between various modules of a robotic control system (that's when JAUS can be thought of as an architecture). There are both open source and commercial implementations of JAUS.

Conclusion

So, what is the best software package? That question can only be answered by you, really. Here are my recommendations.

If you are using robotics for education purposes and LEGO Mindstorms doesn't fit your curriculum, I would recommend Microsoft Robotics Studio. The Visual programming language and simulation environment would be an easy way to script your robots.

If you are a robot designer looking for a complete robotic intelligence package for your service robot, my vote definitely goes to Skilligent. Their software product has everything needed to turn a bunch of motors, actuators, cameras, and sensors into a learning and social creature. (And you don't have to be a software guru to make it work.)

If you are a researcher looking for a robotic platform, check if one of MobileRobots' platforms fits into your budget. These robots won't break in two weeks and come with full indoor/outdoor navigation capabilities. If buying hardware is not in your plans, take a look at Gostai Urbi software which is a clean and easy software framework for robot programming. ERSP would be a good choice if your research requires vision/navigation capabilities. Finally, if your research budget approaches zero, take a look at Player/Stage, OROCOS, or Microsoft Robotics Studio.

If you are thinking about actually using a robot to automate delivery processes or security, MobileRobots would be a good choice.

If you are a creator of the latest sensor/actuator technology, make sure your product is compatible with iRobot AWARE 2.0. If they like your technology, they might want to resell it. Be ready to pay some license fees up front in order to get into the game.

If you are building a real-time vehicle control system, check out OROCOS which was designed in real-time fashion from the very beginning. Skilligent would be your option if you want to achieve fault-tolerance via redundancy.

Whatever platform you choose, opportunities in robotics are constantly growing and improving, and this is a very good thing. **SV**

Contact the Author

Michael Somby is a professional control systems engineer. In his leisure time, he is a long-time enthusiast of service robotics and artificial intelligence. His email address is msomby@yahoo.com

AN INTRODUCTION TO PNEUMATIC SYSTEMS

PART 1: by David A. Ward

SINGLE-ACTING CYLINDER



FIGURE 1.

DOUBLE-ACTING CYLINDER

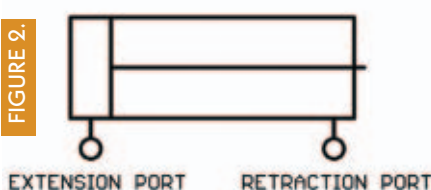


FIGURE 2.

4 WAY 2 POSITION
5 PORTED CONTROL VALVE

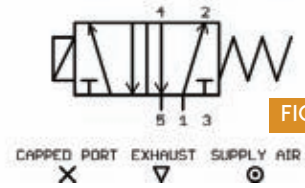


FIGURE 3.

If you're looking for a way to add muscle to your robots, then as Bob Dylan would say, the answer may be may be blowin' in the wind in the form of compressed air and pneumatic controls.

I have wanted to incorporate pneumatics into some of my personal projects and (as an instructor) into my industrial control systems course for some time now, but wasn't sure where to begin. Pneumatic trainers are available for classroom instruction, but the ones I have checked into cost anywhere from \$3,500 up to nearly \$20,000. I recently came across a great website and article, "A Pneumatic Tinker-Toy

Set for the Home Imagineer" at www.phantasmechanics.com/air/air2.html which pointed me in the right direction. This article listed pneumatic part numbers from Grainger Industrial Supply and showed how to interconnect them. Another good resource is *Basic Pneumatics: An Introduction to Industrial Compressed Air Systems and Components*, by Jay F. Hooper (ISBN 978-0-89089-119-3).

Pneumatics can offer some great muscle for projects at a much lower price, lower weight, and simpler design than using electrical components to accomplish the same task. While the nearly \$200 price tag for the system shown in this article may seem high to the hobbyist, it appears quite reasonable when compared to the commercially available pneumatic training systems I have looked into.

This first article will show the reader how to set up a simple pneumatic, double-acting cylinder and pneumatic control circuit that can exert a great deal of force at a very rapid speed, if necessary. The second article will demonstrate how to add electronic controls. This circuit can be controlled with 24 VDC and therefore lends itself to electronic and computer control.

To begin with, let's look at the basic pneumatic components that will be needed for this introductory system (see Photo 1). The system output consists of a double-acting pneumatic cylinder that has a 3/4" diameter piston inside that can travel 6" (see Photo 2). It is a double-acting cylinder which requires air pressure to be applied to either end to

POSITION 1 NON-ENERGIZED COIL
4 WAY 2 POSITION 5 PORTED CONTROL VALVE

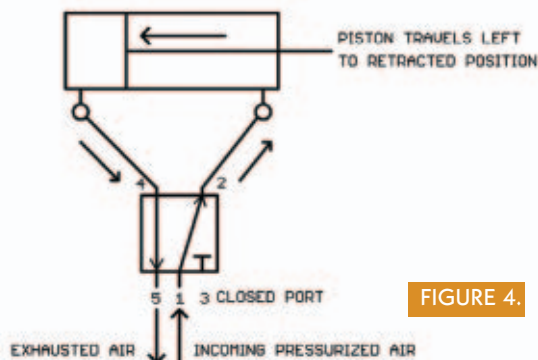
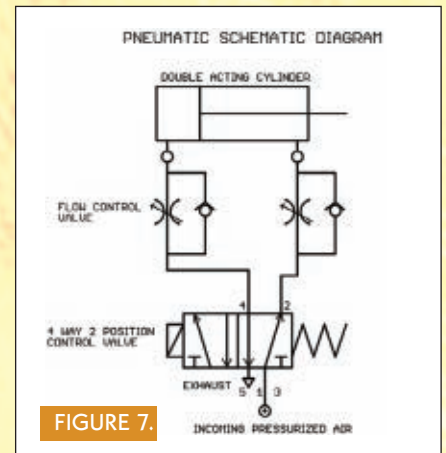
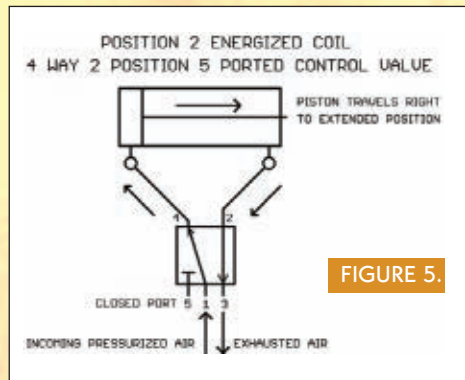
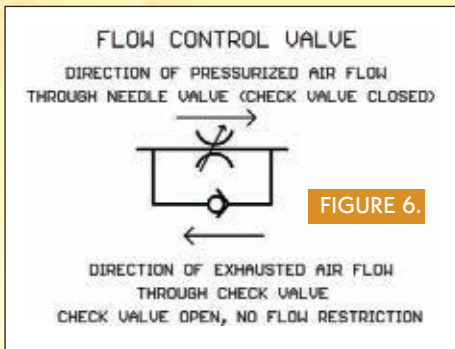


FIGURE 4.



both extend and retract the piston and its rod. There are single-acting cylinders as well, that use air pressure to extend the piston and rod, and use a spring to return them to their retracted position. Figure 1 shows a pneumatic schematic symbol for a single-acting cylinder and Figure 2 shows the symbol for a double-acting cylinder. The cylinder is the muscle of the system. To calculate the amount of force available from a cylinder, simply multiply the applied air pressure in pounds per square inch (PSI) times the area of the piston. If the piston were 3/4" in diameter, its area would be found using the equation to find the area of a circle: $\text{Area} = \pi \times R^2$. The radius of a 3/4" circle is 1/2 of the 3/4" diameter or 0.375"; 0.375 squared is 0.141 sq in x 3.1416 (π) = 0.442 sq in. So, if 100 PSI were applied to the piston it would exert a force of 100 PSI x 0.442 sq in or 44.2 pounds of pushing or pulling force. To increase the force, simply increase the applied air pressure or choose a cylinder with a larger diameter piston.

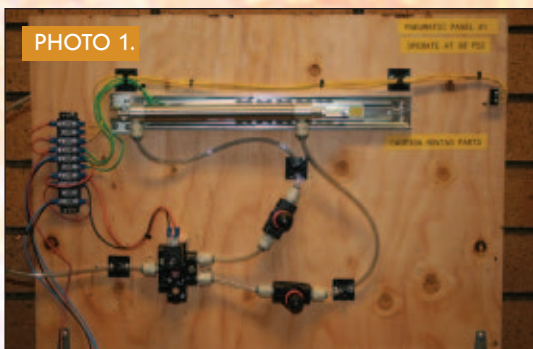
To control the air flow to the cylinder, a pneumatic valve is needed (see Photo 3). The valve shown in Photo 3 is a two position four-way valve which has five ports or openings in it. It is controlled with an electric coil that operates at 24 VDC. Other coils or valves can be purchased that operate at 12 VDC and 120 VAC. This particular valve is rated to operate at pressures from 50 to 150 PSI. To understand how to read the pneumatic schematic symbol for this valve, refer to Figures 3, 4, and 5. Figure 3 shows the full schematic symbol for this valve; at first glance, it may not appear very clear. It is actually showing both valve positions at the same time when the coil is energized on the left side of the symbol, and when the coil is not energized on the right side. The zigzag line to the right side of the symbol represents a spring that shifts the valve so that the right-hand box is the flow condition when

the solenoid or coil is not energized.

The smaller box attached to the left side with a diagonal line in it represents the solenoid or electric coil, and the left-hand box is the flow condition when it is energized. The "X" means a capped or plugged port; a triangle symbol indicates an exhausting port; and the circle with a dot in the center indicates the incoming pressurized air or supply line. To make it easier to understand how the valve and cylinder function together, we can split the symbol in half and show each position separately.

Figure 4 shows the valve in the non-energized coil position with the double-acting cylinder symbol above it. The pressurized air flow is from port 1 through the valve to port 2, then to the retracting port of the cylinder forcing it to the left. If, however, the extending port to the left side of the piston is not allowed to exhaust the air trapped behind it, the piston will only travel to the left until the trapped air is pressurized to the same pressure as the incoming air applied to the right side of the piston. Therefore, the trapped air from the left side of the piston exhausts into port 4 through the valve to port 5, as shown by the arrow pointing from port 4 to port 5. The "T" symbol located over port 3 shows that it is closed so that the pressurized air does not simply exit the valve there but must travel up through the cylinder.

Figure 5 shows the valve in the energized coil position. The pressurized air from the compressor comes into port 1, flows through the valve and out of port 4. This is shown by the diagonal arrow with its arrow pointing in the direction of the air flow. Port 5 has a "T" symbol next to it which indicates that it is closed so that the incoming pressurized air does not flow back out at that port. The air then travels through lines to the extending port of the cylinder, forcing the piston and



rod to the right. The trapped air on the right side of the piston exhausts into port 2 through the valve and out of port 3. All of these ports are clearly labeled on the valve body itself.

There is another item that you will want to add to your system so that you can control the travel speed of the cylinder in each direction: a flow control valve (see Photo 4). Without flow control valves in the circuit, the piston can travel very rapidly back and forth depending on what the cylinder is controlling. The pneumatic schematic symbol for the flow control valve is shown in Figure 6. The flow control valve is direction-dependant and must be placed in the circuit in the correct direction with the arrow pointing in the direction of the pressurized air flow. The flow control valve consists of a one-way check valve in parallel with a needle valve. When pressurized air is applied to the left side of the flow control valve, the one-way check valve will close forcing all of the air to travel through its needle valve side. As the needle is turned in, it will restrict how much air can flow through the valve in a given amount of time. When the pressurized air is removed from the left side of the valve, the one-way check valve can open and let air exhaust back through it from the right side to the left side with very little resistance. So, if these valves are placed in a circuit facing the wrong direction the valve will not provide any restriction to the incoming pressurized air and will restrict any exhausting air flow. At first, I didn't see why these flow control valves were necessary, but after operating the cylinder without them in the lines, I really grew to appreciate them. The piston will typically move much faster than you need when the flow control valves are not included in the circuit. Another item you can add is an exhaust muffler (see Photo 5). These items added to the exhausting air will quiet the system down a bit.

Now all that is needed is some "pneumatic wiring" and connectors. The connectors and tubing are shown in Photo 6. Simply cut the tubing with a sharp blade and push it into the connector. Use polyurethane tubing rated at up to 140 PSI. Figure 7 shows the complete pneumatic schematic diagram.

After connecting the circuit together, you can actually test it without energizing the coil. However, before you apply pressurized air to the system, be sure that the travelling piston and rod will not damage anything or hurt anyone because it will likely move when you connect pressurized air to the control valve. You will notice in Photo 1 the piston has been connected to a drawer slide to guide the cylinder back and forth. Your compressor or air supply line should have a regulator on it so you can control the pressure to the system. I operate my system at 80 PSI since it has no load connected to the piston and rod. After applying the pressurized air to the system, turn the red plastic screw located on the end of the control valve 1/4 turn clockwise. This should activate the valve to move to its second position and send the piston and rod to the extended position. Turning the screw back counter-clock-wise 1/4 turn will return the valve to its first position and the piston and rod to their retracted position.

Next month, we will demonstrate how to add electronic and computer control to this basic pneumatic circuit. **SV**



PHOTO 5.



PHOTO 3.



PHOTO 4.



PHOTO 6.

PNEUMATIC PARTS LIST

WWW.GRAINGER.COM

Part #	Description	Cost
6W129	Air cylinder, 3/4" bore, 6" stroke	\$27.40
6W164	Clevis rod kit for 3/4" cylinder	\$2.65
6W163	Bracket, pivot for 3/4" cylinder	\$2.30
4XT32	Valve, four way, two position, five port, n24VDC \$64.62	
6ZC07	Valve, flow control (two needed at \$18.89 each)	
\$37.78		
4HL92	Tubing, polyurethane, 100'	\$43.56
4HN10	Connector, male, 10 pack	\$11.35
1A325	Muffler, exhaust (two needed at \$1.97 each)	
\$3.94		
Total		\$193.60

Note that not all connectors are listed here. A trip to a local home center may be necessary to get the connections for your compressor.

The *SERVO* Webstore

Attention Subscribers ask about your discount on prices marked with an *

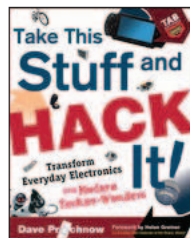


"EDITOR'S PICKS"



Getting Started in Electronics Book

The master electronics teacher and author, Forrest Mims, provides both theory and fun projects in this great introductory text.
\$19.95



Take This Stuff and Hack It!

Can't afford those nifty electronics kits? No problem with this book, which shows you how to turn everyday items into captivating, educational projects.
\$27.95

Editor Bryan Bergeron's recommended reads.

Find these and many more great titles in the *SERVO* Magazine Webstore!



ROBOTICS

Kickin' Bot

by Grant Imahara

Enter the arena of the metal gladiators!

Do you have what it takes to build a battle-ready robot? You do now! Here are the plans, step-by-step directions, and expert advice that will put you in competition — while you have a heck of a lot of fun getting there. Grant Imahara, the creator of the popular BattleBot Deadblow, shares everything he's learned about robot design, tools, and techniques for metal working and the parts you need and where to get them.
\$24.95



Forbidden LEGO

by Ulrik Pilegaard / Mike Dooley

*Build the Models
Your Parents
Warned You
Against.*

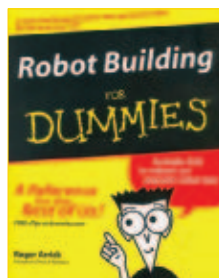
Forbidden LEGO introduces you to the type of free-style building that LEGOs master builders do for fun in the back room. Using LEGO bricks in combination with common household materials (from rubber bands and glue to plastic spoons and ping-pong balls) along with some very unorthodox building techniques, you'll learn to create working models that LEGO would never endorse. **\$24.95**



Robot Building for Dummies

by Roger Arrick / Nancy Stevenson

Discover what robots can do and how they work. Find out how to build your own robot and program it to perform tasks. Ready to enter the robot world? This book is your passport! It walks you through building your very own little metal assistant from a kit, dressing it up, giving it a brain, programming it to do things, even making it talk. Along the way, you'll gather some tidbits about robot history, enthusiasts' groups, and more.
\$24.95



Build Your Own Humanoid Robots

by Karl Williams

GREAT 'DROIDS, INDEED!

This unique guide to sophisticated robotics projects brings humanoid robot construction home to the hobbyist. Written by a well-known figure in the robotics community, *Build Your Own Humanoid Robots* provides step-by-step directions for six exciting projects, each costing less than \$300. Together, they form the essential ingredients for making your own humanoid robot. **\$24.95***



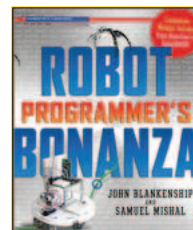
Robot Programmer's Bonanza

by

John Blankenship,
Samuel Mishal

The first hands-on
programming guide
for today's robot
hobbyist!

Get ready to reach into your programming toolbox and control a robot like never before! *Robot Programmer's Bonanza* is the one-stop guide for everyone from robot novices to advanced hobbyists who are ready to go beyond just building robots and start programming them to perform useful tasks.
\$29.95



Robotics Demystified

by Edwin Wise

YOU DON'T NEED ARTIFICIAL INTELLIGENCE TO LEARN ROBOTICS! Now anyone with an interest in robotics can gain a deeper understanding — without formal training, unlimited time, or a genius IQ. In *Robotics Demystified*, expert robot builder and author Edwin Wise provides an effective and totally painless way to learn about the technologies used to build robots! **\$19.95**



We accept VISA, MC, AMEX,
and DISCOVER
Prices do not include shipping and
may be subject to change.

To order call 1-800-783-4624

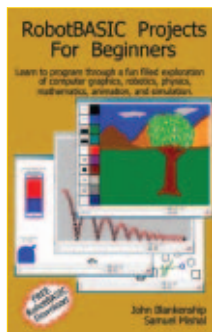
RobotBASIC Projects For Beginners

by John Blankenship, Samuel Mishal

If you want to learn how to program, this is the book for you.

Most texts on programming offer dry, boring examples that are difficult to follow. In this book, a wide variety of interesting and relevant subjects are

explored using a problem-solving methodology that develops logical thinking skills while making learning fun. RobotBASIC a powerful, yet extremely easy-to-use, computer language available for any Windows-based PC is used throughout the text. **\$14.95**



NEW!

*It's a new year!
Time to get the
gears of
innovation
rolling.*

Visit my online store @
www.servomagazine.com



SERVO Magazine Bundles



Published by T & L Publications, Inc.

\$57
per bundle

Save \$10
off the
normal
price!!

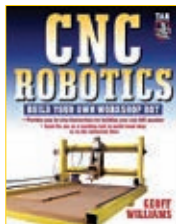
Now you can get one year's worth of all your favorite articles from SERVO Magazine in a convenient bundle of print copies. Available for years 04, 05, 06, 07, and 08.

CNC Robotics

by Geoff Williams

CNC Robotics gives you step-by-step, illustrated directions for designing, constructing, and testing a fully functional CNC robot that saves you 80 percent of the price of an off-the-shelf bot — and that can be customized

to suit your purposes exactly, because you designed it. Written by an accomplished workshop bot designer/builder, this book gives you everything you need. **\$34.95**



Linux Robotics

by D. Jay Newman

If you want your robot to have more brains than microcontrollers can deliver — if you want a truly intelligent, high-capability robot — everything you need is right here. *Linux Robotics* gives you step-by-step directions for "Zeppo," a super-smart, single-board-powered robot that can be built by any hobbyist. You also get complete instructions for incorporating Linux single boards into your own unique robotic designs. No programming experience is required. This book includes access to all the downloadable programs you need. **\$34.95**



The Amateur Scientist 3.0 The Complete Collection

by Bright Science, LLC

There are 1,000 projects on this CD, not to mention the additional technical info and bonus features. It doesn't matter if you're a complete novice looking to do their first science fair project or a super tech-head gadget freak; there are enough projects on the single CD-ROM to keep you and 50 of your friends busy for a lifetime! **\$26.99**



5 CD-ROMs & Hat Special
Only \$105.95 (includes shipping)!
www.servomagazine.com

Or order online www.servomagazine.com

SPECIAL OFFERS

Robot Builder's Cookbook

by Owen Bishop

This is a book for first-time robot builders, advanced builders wanting to know more about programming robots, and students in further and higher education tackling microcontroller-based practical work. They will all find this book a unique and exciting source of projects, ideas, and techniques to be combined into a wide range of fascinating robots.

\$29.95 Sale Price \$24.95



Building Robots with LEGO Mindstorms NXT

by Mario Ferrari, Giulio Ferrari

This book unleashes the full power and potential of the tools, sensors, and components that make up LEGO MINDSTORMS NXT. It also provides a unique insight on newer studless building techniques, as well as interfacing with the traditional studded beams. Some of the world's leading LEGO MINDSTORMS inventors share their knowledge and development secrets.

\$29.95 Sale Price \$24.95



Circuit Analysis Demystified

by David McMahon

Here's the sure cure for CIRCUIT PARALYSIS!

Need to learn circuit analysis but experiencing some resistance in your brain waves? No stress!

Circuit Analysis Demystified will give you the jolt you need to understand this complex subject without getting your circuits crossed. Simple enough for a beginner, but challenging enough for an advanced student.

\$19.95 Sale Price \$15.95



The Definitive Guide to Building Java Robots

by Scott Preston

The Definitive Guide to Building Java Robots is for educators, students, hobbyists, and startups looking for Java/hardware interaction. This book shows you how to use your PC to build robots, and how you can interface with a microcontroller to do the basics. You'll learn to design your robot to navigate, see, speak, recognize your face, listen to you, and build maps.

\$55.95 Sale Price \$47.95



Beginner's Guide to Embedded C Programming Combo

by Chuck Hellebuyck

The C language has been covered in many books but none as dedicated to the embedded microcontroller beginner.

Through his down-to-earth style of writing, Chuck Hellebuyck delivers

a step-by-step introduction to learning how to program microcontrollers with the C language. The PICkit 2 Starter Package is used for all the projects in the book and includes the HI-TECH PICC-Lite Compiler and a PIC16F690.

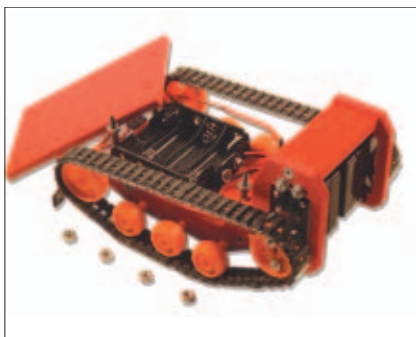
Book and Kit can be purchased separately.

Reg Price 89.95 Sale \$86.95



PROJECTS

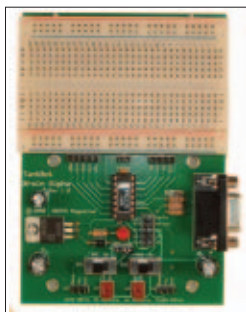
Tankbot Kit & Alpha Brain Kit



The TankBot gives you everything you need to start building your own sturdy servo-motor robotic tracked vehicle. Powered by two modified R/C servos (44 oz-in torque). The assembled Tankbot measures 7 inches long, 4.75 inches wide, and 3.5 inches high. The top deck is drilled ready for the Alpha Brain. For more info & pictures, visit the *SERVO* Webstore at www.servomagazine.com.

\$59.95 Sale Price \$56.95

As seen in the Sept. issue Tankbot/Brain Alpha by Ron Hackett. A bi-monthly series filled with projects and experiments to challenge you through your learning process while you grow your fully expandable Brain Alpha PCB!

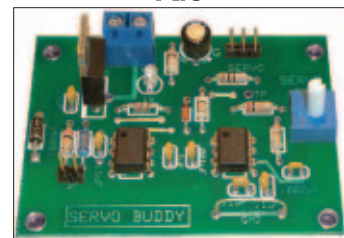


The Alpha Brain kit is an excellent way to get started in robotics. The brain — a PICAXE-14A — is easily programmed in a dialect of Basic that includes many powerful commands. At the end of each of Ron's articles, he will include programming for that month's project. The kit also includes a 400 point breadboard which allows for considerable expansion and experimentation. For more info & pictures, visit the *SERVO* Webstore at www.servomagazine.com.

\$99.95 Sale Price \$85.95

Combo Price \$ 138.95

The SERVO Buddy Kit



An inexpensive circuit you can build to control a servo without a microcontroller!

For more information, please check out your May 2008 issue or go to our website @

www.servomagazine.com

Subscriber's Price \$39.55

Non-Subscriber's Price \$43.95

PCB includes article reprint.

Give the gift that lasts all year!



Go to www.servomagazine.com
to enter your gift order!

Be sure to use promotion code Y8WXMx when ordering.

Subscribe online at: www.servomagazine.com
or call: 877-525-2539 (toll free) / 818-487-4545 (outside US)

U.S. Prices 1 Yr - \$24.95 2 Yr - \$45.95 3 Yr - \$65.95

TWIN TWEAKS

THIS MONTH: The San Diego Job



We've all been there — you've outsmarted the alarm system, outmaneuvered the guards, and outmatched the inevitable laser security devices to finally arrive at the safe. You've smartly brought a robot along to do your dirty work, and just as your mechanical minion is twisting in the combination the police arrive because your robot was too slow!

We're talking about optimization, of course. You have an initial design, but things can always be improved through analysis and iteration. Such a process might even be more commonplace in the realm of engineering innovation than the introduction of

completely new ideas, because often the most stunning advances are only made by standing on the shoulders of giants.

Better Than Dressing Up In A Clown Suit

Evan has made it to his fourth year of majoring in mechanical engineering at the University of California, San Diego, and he must make it through the capstone design courses before he can claim his degree. The capstone design course is known as Fundamental Principles of Mechanical Design, and is given the

fearsome designation of MAE 156A/B. But what project could possibly draw upon the comprehensive course of study undertaken by mechanical engineers, including everything from computer programming and circuit design to materials science and three dimensional dynamics? A robotics project, of course!

Dr. Nathan Delson, the co-instructor and mastermind for the course along with Dr. Matthew Hedayatnia, prefers to think of the task as a mechatronics project because of the tightly intertwined nature of the mechanical and electronic aspects of the project. MAE

156A students are tasked with building a robot for the mechatronics project every year for a prescribed task. Veteran *SERVO* readers might remember that Evan has already had to build one robot at UCSD for the lower division course MAE 3, and it was a robot that stacked rings. For MAE 156A, students were tasked with optimizing a safe-cracking robot. An unoptimized "Safe-Cracking Robot" (even

FIGURE 1. THE UNOPTIMIZED SAFE-CRACKER.

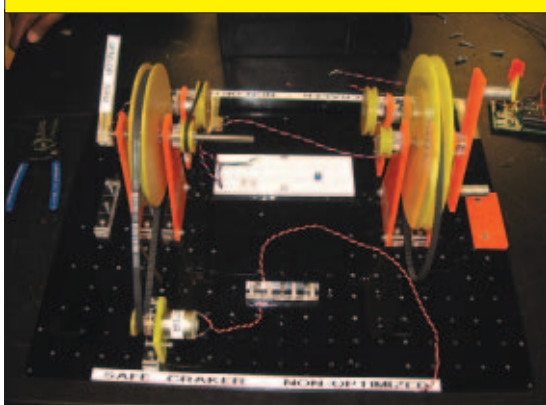


FIGURE 2. OUR KITS OF PARTS.



the spelling was unoptimized) was demonstrated to the class during one of our lectures (engineers have all the fun). The mesmerizing combination of spinning sprockets and fanciful flywheels was basically divided into two halves. The first half was a scanning arm that used three phototransistors to read three black lines printed on a pattern of concentric circles (the safe combination). The second half of the robot was dominated by a flywheel with an objective of repeating the pattern read on the scanning side and pausing at the location of the black lines within a five degree range of accuracy.

Two potentiometers were used to read the positions of the scanning arm and flywheels, and two small DC motors were used to provide the locomotion. The brain of the robot was a PIC (Programmable Intelligent Computer) microcontroller that was programmed using MicroCode Studio Plus from Microchip and mounted on a Lab-X2 board from microEngineering Labs, Inc. The motors were run through a PWM controller known as the MotoMaster Rev. 1.1.

Our task was simple: Optimize the robot design to make a safe-cracker that was as fast and as accurate as possible. For the first four weeks of the class, we were only permitted to use parts that were provided in a kit, but in the final three weeks of the project we were permitted to use parts from outside vendors using a budget of \$30.

Gone In 3.6 Million Seconds

We may have been given the design of the unoptimized robot, but that didn't mean that we were given an unoptimized robot to begin with. Our kit of parts generously included many pre-cut acrylic pieces that allowed us to build the basic structure of the robot, but we still had to spend some unexpected time threading a lot of the mounting holes in our base board. After some quality time with the taps and dies,

however, the unoptimized robot came together fairly quickly.

With a structurally complete robot, it was easy to see that there was a lot that could be done. The gear ratios needed to be recalculated for the potentiometers, the scanning arm, and the flywheel; the brass bushings were begging to be replaced by real bearings; and the timing belts and smooth o-rings could not have possibly been the most effective method of power transmission. And, of course, some overzealous brainstorming resulted in some ideas that were not exactly within the letter of the rules (replacing the potentiometers with encoders, using better motors, etc.), but we still had plenty of legal ideas to implement as a team.

Students were grouped into teams of five, and after three or four years of mechanical engineering education everyone had developed their own unique skill set. Evan was fortunate to have an awesome team that consisted of Vikram Juneja, Andrew Lee, Brandon Reynante, and Kevin Tsai. We all knew that the best way to tackle this robot was to do things the way they were done in the real world — by delegation. Vik was our programmer, Brandon took care of the controls, Andrew and Kevin tended to the mechanics of the bot, and I helmed the circuitry.

Everyone was required to select two areas of optimization for the robot, and popular areas of focus were gear ratios and power train selection, but since I had volunteered

to take care of the circuit I needed to follow the breadboard crumbs.

Lock, Stock, And Three Smoking Phototransistors

My two areas of optimization were to reduce the stiffness of the wire bundle originating with the phototransistors and to increase the circuit performance. Wishing to improve circuit performance is fantastically broad, so I would look in particular at reducing power dissipation, reducing noise, and increasing the sensitivity of the phototransistors.

Optimizing the wire bundle was certainly an iterative process. The original unoptimized design held two glaring flaws: the wires used were solid tinned copper 22 AWG wires, and there were 12 of them originating with the three phototransistors. To make matters worse, in our original design we twisted the wires into a yarn, which actually increased their stiffness in a way described by a fancy equation, shown in Equation 1:

EQUATION 1:

$$\sigma_y / \sigma_f = \cos^2 a [1 - (k_2 * c s c a)]$$

where σ_y is the tensile strength of the yarn (coil), σ_f is the tensile strength of the fiber, a is the helix angle, and k_2 is expressed by Equation 2:

EQUATION 2:

$$k_2 = (d * Q / \mu)^{1/2} / 3 * L_f$$

FIGURE 3. THE TEAM WORKING ON THE BOT.
 FROM LEFT TO RIGHT: ANDREW,
 KEVIN, BRANDON, AND VIK.



FIGURE 4. MAKING PROGRESS!

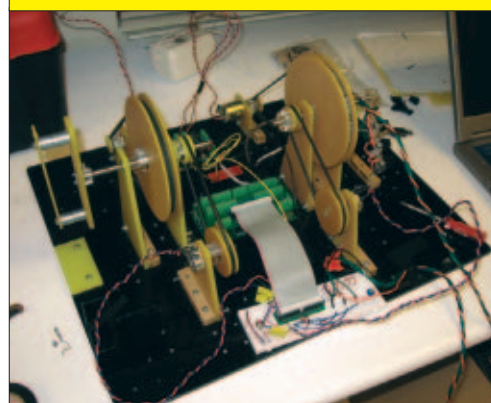




FIGURE 5. EVOLUTION OF THE WIRE BUNDLE.

where d is the fiber diameter, μ is the coefficient of friction between the fibers, L_f is the fiber length, and Q is the fiber migration length. Quite complicated!

The excessive stiffness in the wire bundle did certainly inhibit the motion of the scanning arm, and sometimes it wouldn't even be able to complete a full revolution because the wires would catch on something. To address this problem, I proposed a number of new designs. First and foremost, I wanted to replace the solid wires with stranded, but because we first had to optimize the bot using kit parts that would have to wait until later. Secondly, I wanted to reduce the number of wires in the wire bundle. Each phototransistor was the origin of four wires: two went to a five volt source; one went to ground with a $100\ \Omega$ pulldown resistor; and the fourth was the signal wire.

Wiring up the three phototransistors to have a common power and a common ground would reduce the number of wires in the bundle from 12 to five, and this seemed like a much more manageable number to

flail around on our scanning arm. Once I was able to, I replaced the solid wires with stranded, and each design iteration did indeed improve the speed and consistency

of the scanning arm rotations.

My second area of focus was the circuit, and I proposed to improve the robot performance by reducing power dissipation, increasing the sensitivity of the phototransistors, and reducing noise. The phototransistors were my initial focus because they were integral to the functioning of the robot and they were being unduly troublesome. The phototransistors were horribly inconsistent in reading the black lines on the test pattern. We were scratching our heads until we realized that we had so brainlessly copied the unoptimized robot that we had even mimicked the incorrect wiring of the phototransistors.

A tricky schematic led us to mix up the leads of the second power wire and the signal wire, and while the wily teaching assistants were savvy enough to compensate for the mix-up with a little unconventional color coding, we learned the hard way that hooking up these phototransistors incorrectly was a surefire way to kill them.

After wiring the phototransistors correctly, we were still having

problems. We pulled out the multimeter for a diagnosis, and we found that the phototransistors were not reading five volts for a reflective surface like they should have been. I thought that the problem might lie in the connections to the Lab-X2 board. The Lab-X2 has four banks of eight pins each. Each pin can source up to 25 mA; each bank can source up to 100 mA; and the four banks together can source a maximum of 200 mA.

The three phototransistors — which could draw a maximum of 40 mA each — were originally all connected to a single bank of pins. We moved one to a separate bank, but the sensitivity problems persisted. After calibrating the phototransistors before and after moving them all to separate banks, we finally determined that each phototransistor needed to be on a separate bank of pins for the best sensitivity. And being good engineers, of course we had the data to back it up.

The Not So Silent Partner

My most unique contribution to the circuit optimization was in noise reduction. I knew from working with robot kits like Asuro from Arexx Engineering that capacitors could be added across the DC motor leads to reduce noise, but to effectively implement the change on our safe-cracker I had to prove that such a change would, in fact, improve our performance.

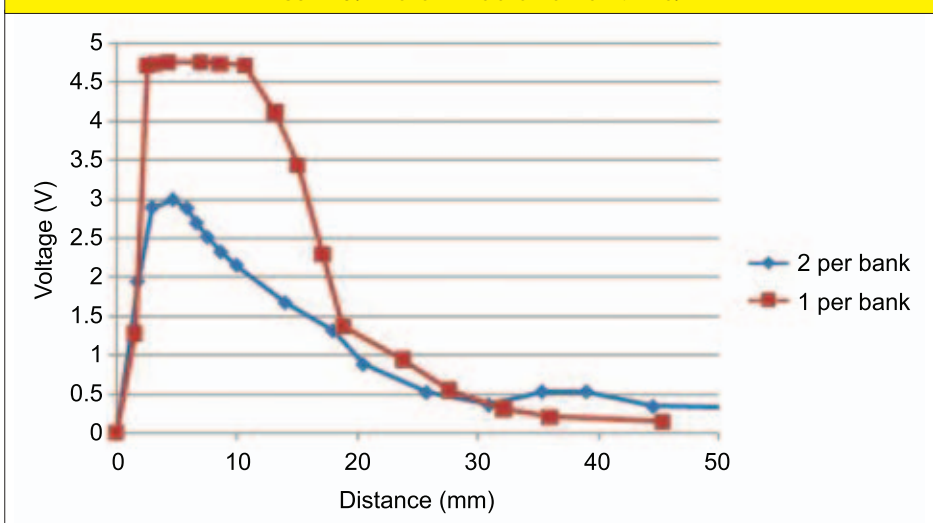
I discovered that the capacitor added in parallel to the motor is known as a bypass capacitor, and it can be used to restrict the ripple voltage to a specific value according to Equation 3:

EQUATION 3:

$$C_b > L^*(I_{avg}/\Delta V)^2$$

where C_b is the value (in Farads) of the bypass capacitor, I_{avg} is the average current draw of the motor, L is the inductance of the motor, and ΔV is the ripple voltage.

FIGURE 6. PHOTOTRANSISTOR SENSITIVITIES.



The ripple voltage is a quantification of the noise associated with the inductive load applied when the DC motor starts up or changes direction. We were unsure if this inductive load was very troublesome, and the best way to check would be by using an oscilloscope. The oscilloscope in the design studio, however, had a bad ground lead and was not suitable for testing.

I went ahead and soldered in the bypass capacitors anyway — if anything, we were still abiding by good engineering practices by taking measures to reduce noise. The value of the bypass capacitor was chosen by plotting C_b against ΔV using Equation 3, and the resulting plot is shown in Figure 7. The value of the capacitor was also chosen based on convenience — I had some 22 μF capacitors handy, so they were the ones that made their way onto the robot.

My optimization was vindicated when the addition of the bypass capacitors did appear to have a positive effect on the robot. The bypass capacitors appeared to reduce the oscillation time of the flywheel as it zeroed in on the correct positions. Oscillations! What oscillations? Why, the oscillations begat by a PD control algorithm, of course, because my teammates were all hard at work on their own optimizations.

The Great Robot Caper

Brandon was our team controls freak, and he successfully implemented a PD (proportional derivative) control algorithm using Zeigler-Nichols Tuning. The technique of Zeigler-Nichols tuning is a way of assigning the proper gain values by increasing gain until the critical gain is reached. The critical gain is when the system (in our case, the flywheel) begins to oscillate.

This method took quite a lot of trial and error, but the finished control algorithm was quite nice. When the flywheel was far away from the target position the motor accelerated, and once the flywheel entered a certain

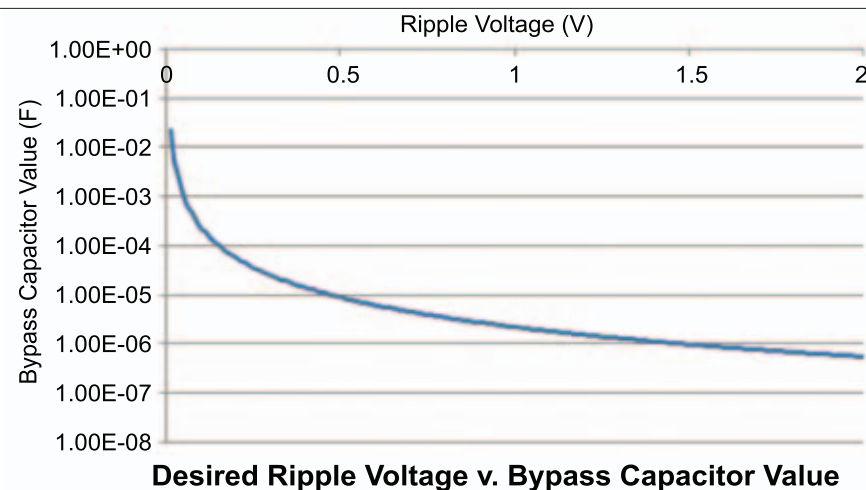


FIGURE 7. SELECTING A BYPASS CAPACITOR.

proximity to the target position the motor decelerated. Usually, the overzealous flywheel would overshoot the target position a few times, but it would quickly oscillate to the correct position.

As the flywheel oscillated to the correct position, the inductive load was applied, and as the oscillations became finer, noise did become an issue. It was nice that my optimization was vindicated, but my other teammates also made dramatic improvements.

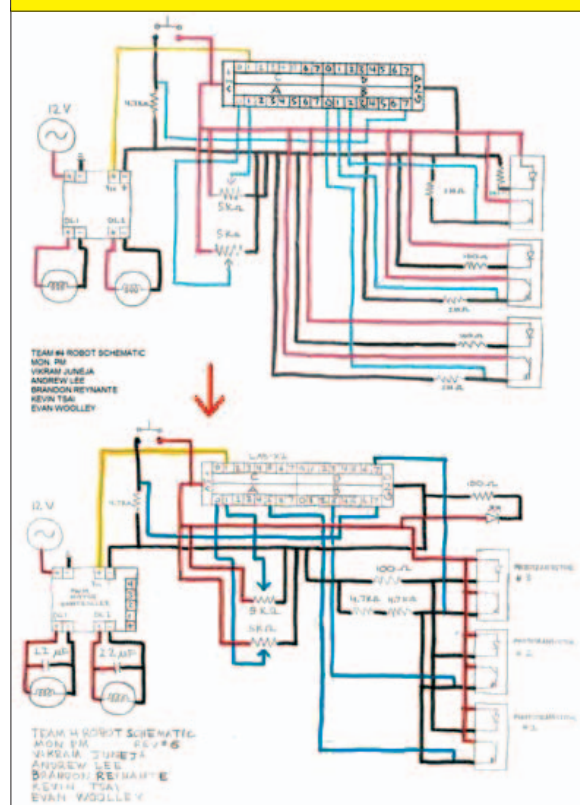
Perhaps the most glaring shortcoming of the unoptimized robot was the fact that it relied on brass bushings to keep the shaft running smoothly instead of real bearings. Our first order of business when we became able to use outside parts was to order some real bearings, and even though we found the cheapest ones that we could find, they still gobbled up a significant portion of our budget. Kevin took the lead on implementing the bearings, and pretty soon the pulleys were spinning along a lot more smoothly than they were before.

Another big change was in the gear ratio, an

aspect that Andrew focused on. The calculation for the ideal gear ratio was done using Euler's method to solve a system of first order ordinary differential equations. The output of the calculation — done using trusty Matlab — is shown in Figure 9.

We were surprised to find that the original unoptimized gear ratio on the flywheel side (1:7.8) was stunningly close to the optimal gear

FIGURE 8. EVOLUTION OF THE CIRCUIT.



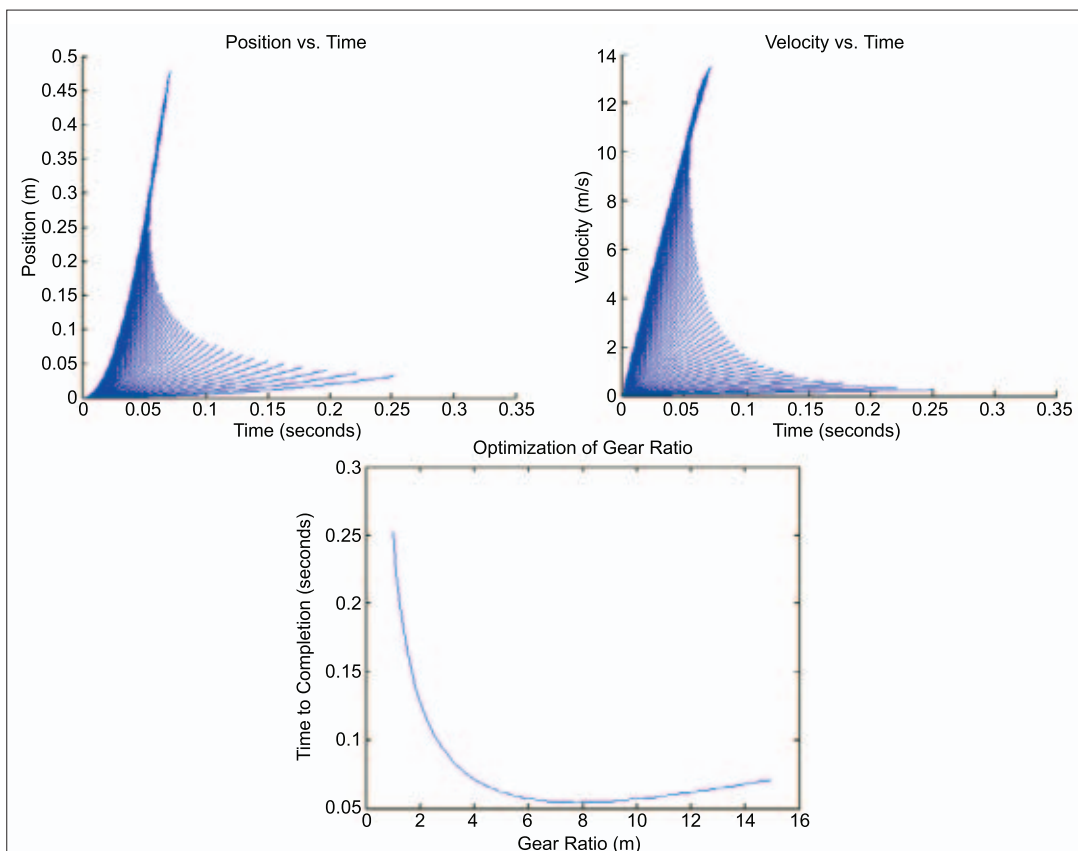


FIGURE 9. GEAR RATIO OPTIMIZATION.

ratio, so we turned to the other gear ratio — that for the potentiometers. On the scanning side, we ended up changing the ratio for optimal sensitivity, and on the flywheel side, we implemented a gear ratio that would give the potentiometer a greater range of motion.

We also considered some power train changes, believing that a gear train would be subject to fewer frictional losses than a belt drive. We

cut out some gears on the trusty Lasercamm, but some difficulties in implementation led to an incorrigible amount of backlash. The backlash outweighed any gains in the friction department, so our gear train experiment was quickly aborted and we returned to the far more forgiving belt and sprocket system.

And, of course, none of these optimizations would have amounted to anything without a working

terrible for calculation time, and once the print commands were excised, the robot's accuracy markedly improved. We finally had a fast and accurate robot, but there is always more to be done.

Our bot matured considerably in the days before the competition. For one, we replaced the sprocket drive on the flywheel side with a friction drive as a way to increase the sensitivity on that side (before it was limited by

the size of the sprocket teeth). We also played with changing the pulldown resistors on the signal wires of the phototransistors, but we stuck with our long-standing design of two 4.7K resistors in series.

We also drew some inspiration from our fellow students, who had come up with some truly creative solutions. Many teams implemented a mechanical brake system to prevent

program, which was written by Vik. By rearranging some of the commands, we were able to reduce the loop time. An elegant implementation of the control algorithm was also a major improvement over the unoptimized code.

The Unusual Suspects

Despite all of our optimizations, the robot was still giving us trouble by being dishearteningly inconsistent. The ordeal itself was a protracted affair, and like many enigmatic and supremely frustrating problems this one had an easy solution. Our code was fraught with print commands which were great for debugging but

FIGURE 10. A SEMI-OPTIMIZED SAFE-CRACKER.

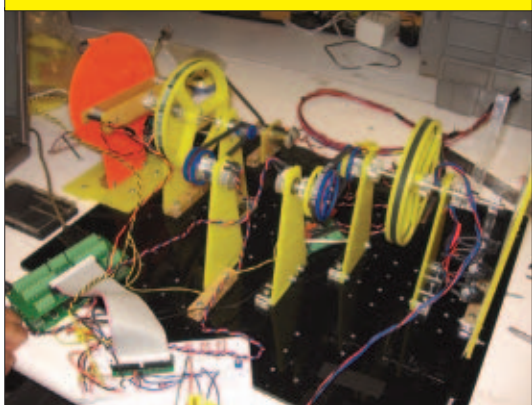


FIGURE 11. FRICTION DRIVE.

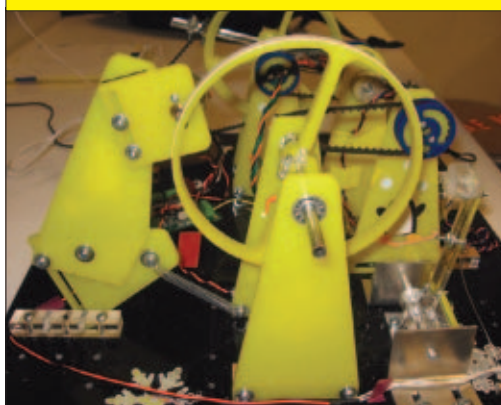




FIGURE 12. THE OPTIMIZED SAFE-CRACKER.



FIGURE 13. WELCOME TO THE PENGUIN FOREST.

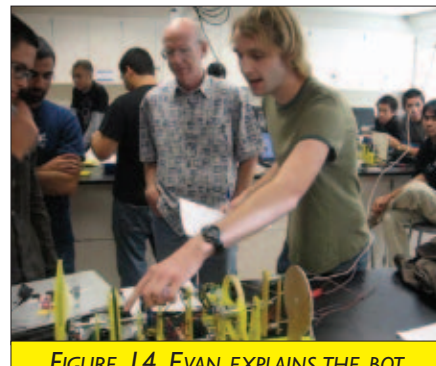


FIGURE 14. EVAN EXPLAINS THE BOT AT THE COMPETITION.

overshoot on the flywheel side, with the most popular braking tool being a tubular solenoid. We also saw a number of creatively mounted gear trains and some very bold designs that included a fixed axle across the scanning and flywheel sides. There was even a robot that spun the test pattern for the phototransistors instead of spinning the little trouble makers.

We attempted a mechanical brake system with a tubular solenoid, but the change was not included in the final robot. One ingenious maneuver that we did implement was to use a hollow shaft to route the wires so that we didn't have to spin the bundle. All we needed was one final change before we were ready for competition — a change that was documented in a list of optimizations as "decorated the robot with festive stickers to increase aesthetic value and cool factor." Besides, what robot wouldn't want to be decked out with silvery snowflakes and precocious penguins?

The competition — though not as adversarial as some of the competitions that we have seen — was still an exhilarating experience. This, like all robot contests, was really a tournament of ideas where students had the chance to put their knowledge to the test and see what their peers had imagined.

Crime Doesn't Pay, But An Engineering Degree Does

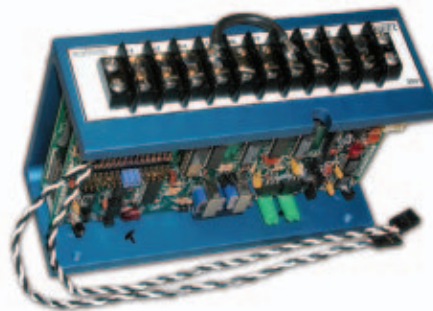
At the competition, we were able to accurately reproduce three test patterns and the fourth mystery pattern, even chalking the speedy competition time of 3.25 s — much better than an unoptimized run of nearly 12 s. Our times may not have been the fastest in the class, but we're sure that we would have given Sir Charles Litten a run for his money. Overall, the MAE 156A robot project was stressful, exciting, frustrating, fulfilling, and a host of other paradoxical combinations that made it a good analog

for the experience of working on a project in the real world. In addition to the robot itself, our team was tasked with performing a Failure Mode and Effects Analysis (FMEA), remaining cognizant of Design for Assembly (DFA), and tracking our task organization with a detailed Gantt chart. Basically, the real world can rest assured after engineering students have run the gauntlet of classes like MAE 156. **SV**

RECOMMENDED WEBSITE

For more information, go to www.maelabs.ucsd.edu/mae156/

STEER WINNING ROBOTS WITHOUT SERVOS!



Perform proportional speed, direction, and steering with only two Radio/Control channels for vehicles using two separate brush-type electric motors mounted right and left with our **mixing RDRF dual speed control**. Used in many successful competitive robots. Single joystick operation: up goes straight ahead, down is reverse. Pure right or left twirls vehicle as motors turn opposite directions. In between stick positions completely proportional. Plugs in like a servo to your Futaba, JR, Hitec, or similar radio. Compatible with gyro steering stabilization. Various volt and amp sizes available. The RDRF47E 55V 75A per motor unit pictured above. www.vantec.com

VANTEC

Order at
(888) 929-5055

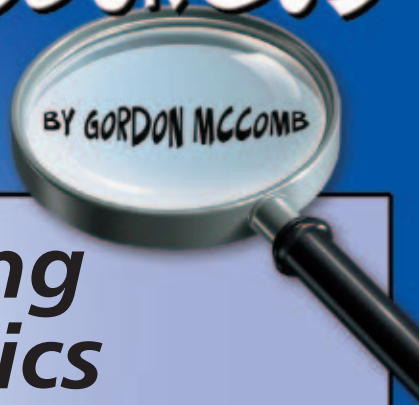
SPECIAL THANKS TO

Dr. Nathan Delson, Dr. Matthew Hedayatnia, Josh Miller, Michael Sankur, Chris Cassidy, Steve Roberts, and everyone else that made the MAE 156A Robot Contest possible!



ROBOTICS RESOURCES

Tune in each month for a heads-up on where to get all of your "robotics resources" for the best prices!



Kits for Learning About Electronics

Robotics is a multi-discipline science. Among its main elements are mechanics, electronics, and programming. Ready-made robot kits offer to lessen the learning curve of each of these disciplines. At some point, however, it's useful — to be a well-rounded robotologist — to learn at least the basics of all three.

In past columns, we've covered learning more about mechanical aspects of robot design, including materials selection, fasteners, and construction techniques. In this month's column, we'll discuss learning more about the electronics side of things. To keep things as basic as possible this time around, we'll concentrate on using "learning lab" kits and the tutorials that come with them to learn more about electronics. These kits come with all the parts you need to follow the step-by-step guide that's included with the product. These learning labs are a great way to begin your electronics education.

Analog or Digital Electronics

Most learning lab kits for electronics can be roughly divided into two distinct camps: analog electronics and digital electronics. Analog electronics are centered around the basics of electricity and how it flows in a circuit made of wires and other common electronic components. These fundamental components include resistors, capacitors, switches,

batteries, indicator lights, and so on.

Digital electronics delves into the world of computers and microcontrollers. These kits are designed to teach things such as binary logic and other computer science fundamentals — what is a bit, byte, or word, how individual bits can be synchronized to transmit data, and so on. There are other forms of learning lab kits (of course) including:

- **Hybrid.** These kits contains tutorials for both analog and digital.
- **RF.** These kits are designed to help you learn more about radio frequency communications.
- **Special function.** Let's not forget all the kits that are designed to be assembled to construct a particular device, such as an FM broadcast station or electronic digital dice or a noise generator. While these kits aren't designed to offer much experimentation in different designs, most special function kits contain a theory of operation section that discusses how the circuit works.

Of the two main types — analog or digital — beginners should opt for a kit that teaches the basics of ordinary analog electronics, even though more and more circuits (especially in robotics) are the digital variety. Even digital circuits use many types of analog components — for instance, resistors to limit current to light emitting diodes or capacitors to help remove electric noise and smooth out

voltage levels. Learning the purpose of these and other common electronic components goes a long way in the digital world, and to effectively design or work with digital circuits, you need a firm grounding in general (i.e., analog) electronics theory and practice.

What's in the Typical Learning Lab Kit?

Learning lab kits are composed of three main parts:

- 1) *Solderless breadboard.* The breadboard allows you to plug in wires and other components to build up a circuit. There's no soldering involved; hence the name "solderless." The breadboard is composed of multiple rows and columns. The columns are electrically connected together, to form "tie points" for the various components that you are using in the circuit. The size and complexity of the breadboard may vary, depending on the level and cost of the kit. A very basic kit may contain only a small breadboard which is useful for just simple circuits. The more elaborate kits come with a larger breadboard, allowing you to construct more elaborate designs.
- 2) *Tutorial or guide book.* An integral part of the kit is the step-by-step tutorial or guide book that instructs you on how to build each circuit. The quality of kits varies greatly here. A guide book that merely shows you how to connect up various parts



misses the idea of learning by using an experimenter's kit. Each circuit, session, or lesson should contain a section on theory of operation. The lessons usually build upon one another and are intended to be constructed in more or less sequential order. Kits are often sold by the number of projects or circuits explained in the guide book.

3) *Electronics parts.* The kit contains a bag (or three!) of parts, including pre-stripped insulated wire used for connecting things on the solderless breadboard. Generally, the more circuits a kit is designed to teach you about, the more components are included. The better kits provide for convenient storage of the parts when they're not in use. There's nothing more annoying than searching for that 330 ohm resistor in a pile of switches, wires, diodes, transistors, and other parts. If your kit doesn't come with a small parts bin with individual drawers, invest in one yourself. It will save you a lot of time and frustration.

Many learning lab kits provide a kind of workspace for easier experimentation, rather than just a solderless breadboard. The workspace (the "lab" part) includes mounted switches, dials (potentiometers), indicator lights, battery hookup posts, and other features to make your work easier. The breadboard is placed in the center of this workspace and you connect wires to the various components. While this is a handy feature, it's not an absolute requirement. I'd strongly suggest it, however, for learning in a classroom setting or for younger students.

While we're on the subject of the lab workspace, there are some kits — typically designed for a fairly newbie audience — that do not include the solderless breadboard. ALL of the components you use are pre-mounted in the workspace.

Components are connected together using spring posts; you tweak the spring to one side and slide the connecting wire into it. The tension of the spring keeps the wires

in place. This design is typical of the all-purpose kit that explains analog, digital, and even some RF electronics. These kinds of kits are great for young novices, but for an older audience I personally recommend learning lab kits centered around the solderless breadboard, as these are capable of more elaborate circuit designs.

Selecting the Right Learning Lab Kit for You

When looking for a good experimenter's kit, don't be swayed by just the number of projects or circuits contained in the guide book. With some kits, the projects are only simple variations of one another, so in a so-called 300-in-one kit (300 projects), there's really only 50-75 distinctly different circuits.

That's not bad, actually, as you don't need tons and tons of different circuits to

learn the basics of electronics, but you do want some variation in the projects presented. So, if possible, see if you can get a sneak peak at the guide book. Or, at least carefully read the list of projects/circuits that the kit encompasses.

Naturally, the price of the kit increases proportionally to the number of projects it provides. If you're on a budget, a basic kit that explores 50 to 100 projects is perfectly fine, as long as the manual is reasonably well written and actually teaches electronics as it explains how to connect the various parts together. Prices vary from \$25-\$30 for a basic learning lab kit for younger students, to well over \$250 for an advanced kit that explains analog and digital electronics at a high school or college level.

Making Your Own Kit

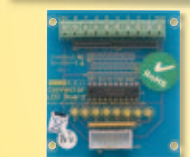
Of course, you don't absolutely need to buy a ready-made kit to learn about electronics. You can put your own kit

Elxol 3rd Generation Low Cost USB Data I/O Module

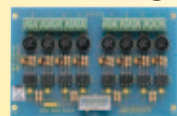
Need to get data into or out of a USB port? Here's what you need...

- 24 independently programmable Input/Output pins grouped into 3 ports.
- Single module high-speed digital Input/Output solution.
- Up to 128 modules can be connected to a single PC with capabilities of further expansion.
- Easy to connect by 0.1" pitch headers to suit standard IDC connectors.
- Integrated Type-B USB connector.
- On-board unique serial number in EEPROM and custom programmable FLASH microcontroller.
- Both USB enumeration information and microcontroller can be re-programmed to suit customer needs.
- Module powered by the USB from the PC.

Great range of I/O-24 peripherals, too...



CONNECTOR/LED BOARD
Provides screw terminal connections plus optional either pull up or pull down resistors and LED port status indication.



OPTO INPUT BOARD
Provides 8 Opto isolated inputs for each of the I/O-24 pins with LED indications.



RELAY OUTPUT BOARD
Provides 8 isolated relay contact outputs suitable for a variety of loads.



SWITCH/PUSH BUTTON BOARD
Provides 8 switch/push buttons with LED indicators showing the status of the I/O pin.



Visit our website www.orteches.com

Ortech Education Systems

**1119 - 9th Ave. S. • Moorhead, MN 56560
218.287.1379**

ROBOTICS RESOURCES

together by buying a solderless breadboard, pre-stripped wire, and an assortment of parts. The how-to tutorials can be found on the Internet at various sites, and in books such as *Getting Started in Electronics* by Forrest Mims.

Note, however, that some books and websites assume you already know how to read a schematic diagram to understand how to construct the circuit. If you're just starting out, you want plans that show a solderless

breadboard and how the components and wires actually connect together.

Buy components based on the parts list provided in the book or on the website. To save money on shipping and handling charges, try to get all your parts from just one or two sources. For the most common parts, you're usually better off buying a small package quantity of them. This is particularly true of the components that are very low cost to begin with, such as standard 1/8 or 1/4 watt resistors, disc capacitors, and light emitting diodes (LEDs).

Some mail order firms (such as Electronix Express) sell pre-packaged "prototyping kits." These are ideal when making up your own learning lab kit because they've already put together the most commonly used components, and in the case of Electronix Express, they also include the divided drawers in which to keep the parts sorted. You can separately purchase any specific parts called for in the book(s) or websites you are using for specific circuit plans.

Electronics123 offers numerous electronics learning lab kits and provides quantity discounts.



As with many of the kit sources here, Ramsey Electronics offers learning lab kits and specialty solder-required construction kits.



Sources

The following represents a selection of online resources for electronic learning lab kits, as well as some of the general electronics supply outlets in case you wish to put together your own learning modules. Do note that there are a number of learning lab kits that are resold through many online retailers. We've listed only a few of those retailers here, as many are regular supporters and advertisers of *SERVO & Nuts & Volts Magazine*.

All Electronics
www.allelectronics.com

Sells surplus motors, including stepper, DC, and gear motors. Also offers a printed catalog mailed to current customers. Retail stores in the Los Angeles area.

B.G. Micro
www.bgmicro.com

Source for surplus motors. Check their site often so you are sure to catch the best deals.



Carl's Electronics

www.electronickits.com

Volume retailer of all forms of electronic kits, including learning lab kits. Reseller of the Elenco Snap Circuit kits.

Circuit Specialists

www.circuitspecialists.com

Full source for components, plus made-to-order kit service.

Design Notes

www.designnotes.com

Electronic learning lab kits, including spring-based lab kits and replacement parts.

Electronics 123

www.electronics123.com

Online resource for kits, components, books, and more.

Electronix Express

www.elexp.com

Sells individual components, as well as prototyping kits and learning lab kits.

Elenco

www.elenco.com

Elenco offers Snap Circuits — a unique form of educational kit ideally suited for beginners under 12. Available through distributors.

Hobby Engineering

www.hobbyengineering.com

Learning lab kits for first-timers through advanced users.

HSC Electronic Supply

www.halted.com

Mail order, with walk-in retail stores in Northern California.

Jameco Electronics

www.jameco.com

Jameco carries a wide variety of components, solderless breadboards, and learning lab kits. Online or mail order with printed catalog.

Jaycar Electronics

www.jaycar.com

Source for solder and non-solder science and electronics learning lab kits, including their unique Short Circuit series where circuits are reproduced on pre-printed, schematic-like project cards.

Mouser Electronics

www.mouser.com

Mouser is a one-stop-shop for electronics components. Online or mail order with printed catalog.

RadioShack

www.radioshack.com

Online sales and local retail stores; stocks learning lab kits, solderless breadboards, and individual components.

Ramsey Electronics

www.ramseyelectronics.com

Offers a wide variety of learning lab and other educational kits. Their PL300 (300 projects) is a good example of an intermediate kit well suited for students and adult first-time participants. **SV**

Celebrating Our 10th Year Anniversary
Learning Circuits: Hands On with Electronic Lab Kits!
— ALL LAB KITS INCLUDE STEP-BY-STEP MANUALS

Build Your Own AM/FM Radio!	130 in 1 Electronic Projects No Experience Needed	30 Educational, Exciting Projects Safe & Solderless!	300 Electronic Experiments for one low price!
\$15.99	\$79.99	\$29.99	\$119.99

1-800-957-6867
www.DesignNotes.com
DesignNotes.com
What Your Electronic Hobby Stores Used To Be

AP CIRCUITS
PCB Fabrication Since 1984

As low as...
\$9.95
each!

Two Boards
Two Layers
Two Masks
One Legend

Unmasked boards ship next day!

www.apcircuits.com

VISA MasterCard PayPal IPC MEMBER ASSOCIATION CONNECTING ELECTRONICS INDUSTRIES

Gordon McComb can be reached at
robots@robotoid.com.



Then and NOW

GOVERNMENT AND MILITARY ROBOTS

by Tom Carroll

I have long been interested in military weapons. The meanest, sneakiest, most powerful, and most complex weapons seemed to always attract my attention. Adding robotics to these types of weapon systems makes it just that much more appealing to me.

Military robots — ground, sea, and air versions — have made the conflicts in Iraq and Afghanistan safer and more efficient for our troops. The Military Channel and History Channel's *Future Weapons* make it clear that new military technology is rapidly moving ahead, despite the economy. Let's look at a few of the older and newer robotic weapon systems.

Goliath

The name Goliath instantly drums up visions of something large. The Goliath self-propelled mine developed in WW II by the Germans was one of the most unique and ingenious weapons of war ever created. Weighing in at over 800 pounds and almost five feet long, the 'demolition charge carrier' contained over 130 pounds of TNT. There were several variations such as a gas engine only powered unit and a gas/generator/motor version. Figure 1 (taken by Black Dragon Leather at the Aberdeen US Army Ordnance Museum) shows the distinctive German tank tread style.

Controlled by a trailing cable connected to a control panel, the remotely-guided anti-tank/anti-personnel mine was a slow 6 MPH,

had poor ground clearance, and was vulnerable to anti-tank weapons or even rifle fire due to thin armor. It was also very expensive to produce for the low success rate. Two 5 kW Bosch electric motors and an internal 13.6 HP gas motor/generator system were contained in the two foot high tracked mine. They were deployed in Italy in 1943 and later in Normandy.

Prowler

Commander H.R. (Bart) Everett,

now retired from the Space and Naval Warfare Systems Center, San Diego, CA, has long been involved with robots, even as a kid. One of the robots that he was involved with back in the mid '80s was the Prowler. Prowler stood for 'Programmable Robot Observer With Logical Enemy Response' — a typical acronym where a project team comes up with a nice-sounding name for the project then has to find a string of words that fit with the key word.

The Prowler was the first outdoor

Nikola Tesla

Many say Nikola Tesla built the first true robot. His 'wireless torpedo' was designed to make it into a weapon of war that removed people from the action.

Tesla's idea was to have nations stop building battleships that shell each other across miles of open sea, and instead use these unmanned undersea craft to duel with each other to a win or draw. Figure A shows the unique, iron-hulled "robot boat".

His invention, as many later historians would say, "was the birth of robotics." Tesla's own description of the robot boat was "In fact, it is simply an enlarged torpedo shell, thirty-six and a half feet long, loaded with other torpedoes to discharge. Like a torpedo, also, it has its own propelling device. But here the likeness stops. The ordinary torpedo, once launched, plunges head on blindly and no known power can turn it one way or another. It hits or misses, according to the trueness with which it is aimed at its launching."

A patent titled "Method of and Apparatus for Controlling Mechanism of Moving Vessels or Vehicles," No. 613,809, was granted to Tesla shortly after a demonstration for the Navy. These torpedoes were not true submarines as there were no diving planes to counteract the slight positive buoyancy, so they were more of a surface operating torpedo with a detonator in the bow.

FIGURE A. Tesla's robot boat.



robotic sentry surveillance system. Developed by Robot Defense Systems, it was first operated as a teleoperated vehicle (Figure 2). The first Prowler prototype was equipped with two M60 machine guns and a grenade launcher.

The company marketing person, Walt Lee, made the following comment about the robot back in 1984: "In the United States, we don't foresee the Prowler armed with lethal weapons, or, if <the need> is, there will always be a human required in the loop for activation. But there are countries where there aren't the same political or moral considerations."

The teleoperated version was based on Standard Manufacturing's six-wheel hydrostatic drive system with power for the hydraulics coming from a diesel engine. Don Davis, the security specialist at one of Robot Defense Systems' customers, made the following observation: "The driving factor is the cost of a 24 hour guard. In the long run, a robot is cheaper, and it doesn't get bored, lonesome, or fall asleep. Plus, a robot will always do what you tell it."

As technology developed and inexpensive GPS navigation became available, computer autonomy was achieved. In 1985, the Prowler demonstrated the ability to follow a 500 foot fence at the Army's Fort Lewis in Washington, though the chain link fence was modified a bit with retro-reflective strips to assist the vehicle's side-looking laser rangefinder. A telescopic 28 foot mast allowed a camera to look over tall obstacles. High unit costs (\$200-\$300K) and few customers caused the company to go bankrupt in 1986.

PackBot

Generally, when those of us who are interested in experimental robotics hear the word 'iRobot,' we may think of Isaac Asimov's famous series of short stories. When it refers to a company, most of us think of the Roomba and several other styles of home robots. We've all heard that iRobot has sold millions of Roomba's

around the world, but many may not know that the company has been very successful with military groups around the world purchasing its PackBot, (over 2,000 and counting). The robot (shown in Figure 3) earned its name by being able to be 'packed' in to a specific site.

The PackBot Scout is a low cost model and it runs under a Pentium processor. The Scout weighs in at about 40 pounds; light enough to be carried on a single soldier's back along with the remote control panel. This little guy can actually carry over 2-1/2 times its weight and scoot along at a pace of 5.3 MPH that could keep a soldier with a full pack following it a bit out of breath. It has been proven to be quite adept in military roles, with its ability to be thrown through a window or doorway, land on a concrete floor, and then go to work 'scouting' the interior of a building.

As with all the PackBot models, it can be customized and includes eight internal bays for additional modules, and an array of sensors for video and thermal sensing of potential subjects. The most unique feature of all the PackBot robots is the QuickFlip — tracked articulations or 'flippers' that are capable of righting it when it becomes upside down, climbing up stairs, or tackling grades as steep as 60 degrees.

The PackBot Explorer is an advanced version of the basic Scout. It is built on the same chassis and the weight and the dimensions are similar to the Scout. The Explorer has an additional suite of sensors and optics to be used by personnel who control the robot as it explores caves, bunkers, tunnels, and hideouts, feeding the information back to a controller in a safe location. These advanced warning eyes and ears give soldiers a heads-up on what to expect when entering dangerous situations. The same models have been used successfully in providing police the ability to safely converse with

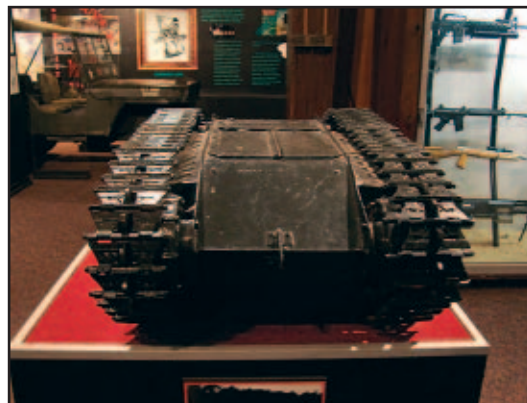


FIGURE 1. Goliath Demolition Charge Carrier.

persons of interest in hostage or sniper situations.

The PackBot EOD — or Explosive Ordnance Disposal — is the robot that most people have seen on the news with its use in discovering, disabling, and removing IEDs (improvised explosive devices) in Iraq and Afghanistan. Quite often, these explosives are buried in the middle of a dirt road that terrorists know military vehicles will be traversing. A simple pair of wires that lead off to a hiding place sends a signal to the shell to detonate when a vehicle is seen passing over the shell's site.

The EOD PackBot is slower and smaller compared with the others, but can go into tunnels and sewers to seek out IEDs. Its attached arm and gripper end-effector can inspect or detonate the IED after it's detected by on-board, ground penetrating radar or chemical sensors attuned to the nitrogen compound signature of the explosive. The use of the EOD PackBot has saved many lives when an IED has gone off unexpectedly, destroying the

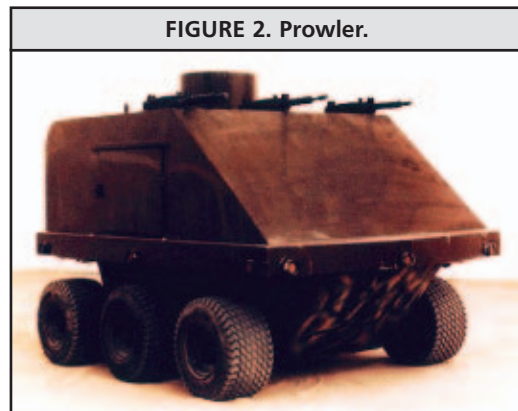


FIGURE 2. Prowler.

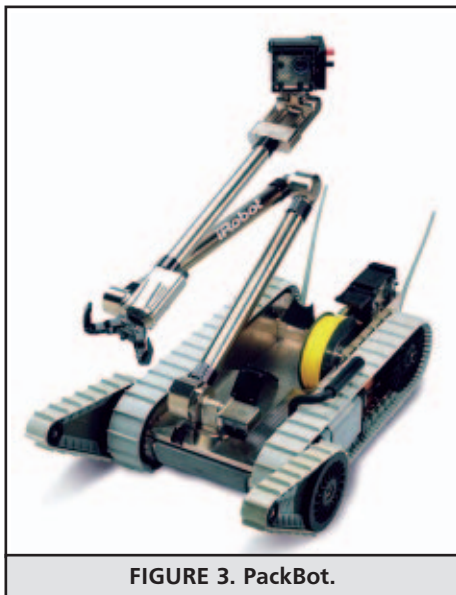


FIGURE 3. PackBot.

robot in the process. iRobot has received many PackBots back at its Massachusetts factory that were nothing more than a box of twisted metal fragments.

PackBot SUGV

In the spring of '07, iRobot and Boeing teamed up to develop the SUGV 300 series of tactical robots that are smaller and lighter (30 pounds) than the standard combat-proven PackBots. Also called the SUGV Early, they have been mainly



FIGURE 4. iRobot PackBot SUGV.

used by our military to gather situational awareness in battle conditions while keeping actual troops out of harm's way.

As with the other PackBots, the SUGVs have the same articulated tracks to allow them to traverse rubble and enter into dangerous or other areas inaccessible to a soldier. Though its use has been primarily by the military for search missions and disarming IEDs, civil agencies have used it for hazardous police work (Figure 4).

Armed Robots

The subject of arming remotely-controlled or even autonomous robots

has long concerned civilian and military authorities alike. The concern arises when an operator's away from the scene of action and cannot clearly see what is around the robot he or she is controlling. A decision to fire at a target may turn out to be a grievous mistake. This concern is even more critical with an autonomous robot that may see a child with a toy rifle and react

erroneously to a non-existent threat. We have however, successfully used the Predator aerial platform with live bombs and Hellfire missiles to take out actual enemy targets, but many lawmakers — and civilian experts — are still concerned.

The US Joint Forces Command's Project Alpha rapid idea analysis group is focusing on the concept of developing robots that are capable of replacing humans to perform many, if not most, combat missions. The study is titled, "Unmanned Effects: Taking the Human out of the Loop." They are suggesting that "as early as 2025, the presence of autonomous robots, networked or integrated, on the battlefield might not be the exception, but, in fact, the norm."

SWORDS, PackBots, Talons, Predators, Global Hawks, Big Dogs, and other unmanned weapons platforms just might be the starting point for an all new military, with exciting possibilities for civilian law enforcement agencies.

Final Thoughts

All of us who have built robots, feel a bit of pride that various levels of governments are now using robots for the betterment of society. Go to the Internet and use your favorite search engine to check out all the types of unique robots available to these agencies. **SV**

Tom Carroll can be reached via email at TWCarroll@aol.com.

APEC 2009 THE PREMIER
February 13–21, 2009 GLOBAL EVENT
Washington, DC IN POWER

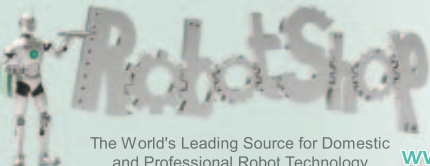
ELECTRONICS™
Visit the Apec 2009
web site for the latest
information!

www.apec-conf.org

SPONSORED BY



ROBO-LINKS



Robotics at your service!™
The World's Leading Source for Domestic and Professional Robot Technology
www.robotshop.com

For the finest in robots, parts, and services, go to www.servomagazine.com and click on **Robo-Links**.

CONTROLLERS & ROBOT KITS **SENSORS – DISPLAYS**

LOW COST! Free Expert Technical Support 1 (800) 985-AWIT
www.c-stamp.com

LOW COST! Professional SW Dev. Tools

Beta PCB-POOL

LAYOUT TOLL FREE 1877 3908541
WWW.PCBPOOL.COM
2 Days: \$92.80 8 Days: \$23.20
(Price Example: 16 Sq-Inches, Jds - gth)

HAGISONIC www.hagisonic.com
The Premier Robot-Sensor Company

Also, Visit Our Distributor www.robotshop.ca

AniBat™ : Ultrasonic Sensor For Intelligent Robots - Detects Wider Range!!!

StarGazer™ : Indoor Localization Sensor For Intelligent Robots - World Best Technology!!!

Pololu Robotics & Electronics
Motor & servo control
Robot controllers
Gearboxes & wheels
Robot kits
Custom laser cutting
Solder paste stencils

6000 S. Eastern Ave. 12D, Las Vegas, NV 89119
www.POLOLU.com 1-877-7-POLOLU

Loaded with brainpower!

www.NanoCore12.com

A new era of secure RF
www.CipherLinx.com

CipherLinx Technology

Bi-Directional Flex Sensors
passive style sensor changes resistance as it is flexed

www.imagesco.com/sensors/flex-sensor.html

ALL ELECTRONICS CORPORATION
Electronic Parts & Supplies Since 1967

\$29.95 MaxSonar-EZ1 (MSRP)

High Performance Ultrasonic Range Finder

- serial, analog voltage & pulse width outputs
- lowest power - 2mA
- narrow beam
- very easy to use!

www.maxbotix.com

GOT HAT?

www.nutsvolts.com

ADVERTISER INDEX

All Electronics Corp.37, 81	Hagisonic Co.37, 81	PCB Pool42, 81
AP Circuits/e-pcb.com77	Images Co.81	Pololu Robotics & Electronics ..12, 81
APEC80	Innovation First2	RoboteQ37
A-Wit/www.c-stamp.com37, 81	Jameco17	Robotis7
CrustCrawler13	Lynxmotion, Inc.3	Robot Power42
CypherLinx Technologies81	Maxbotix81	RobotShop, Inc.29, 81
DesignNotes77	Medonis Engineering37	Solarbotics/HVW9
DLP Design82	Net Media83	SolderByNumbers.com36
Electronics12337	Ortech Education Systems75	Technological Arts81
Firgelli Technologies7	Parallax, Inc.Back Cover	Vantec73

Enter the DLP Design

DLP-232PC DESIGN CONTEST

for your chance at winning a 65-inch Hi-Definition LCD TV!

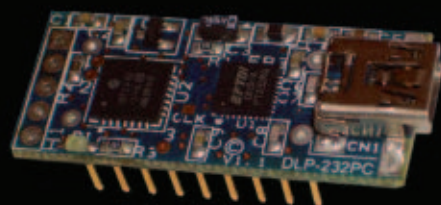
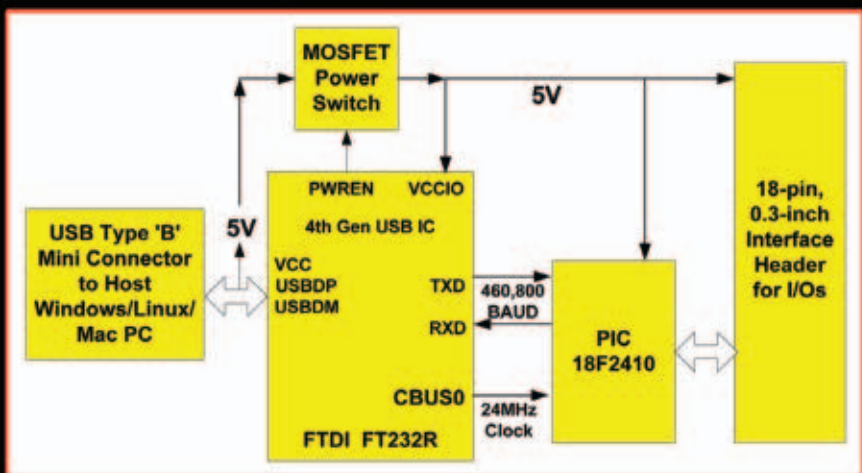
When small size and low cost are at the top of the design constraints list, look no further! The DLP-232PC offers 14 I/O channels (eight can be configured for analog input) and USB communication with a host PC at up to 460K baud. Pit your design skills against your colleagues and come up with the most innovative, smallest and lowest-cost design possible using this tiny USB-to-microcontroller module, and you just might walk away with a new Hi-Def LCD TV or one of five other cool prizes.

DLP-232PC FEATURES:

- FT232RL USB IC from FTDI
- 14 I/O (eight 10-bit analog channels)
- PIC18F2410 16K ROM, 768 RAM
- USB port powered
- Tiny Footprint: 1.375x.6 inches
- No in-depth knowledge of USB required!

PRIZES:

- 1st Place: 65-inch Hi-Def 1080P LCD TV or \$5,000 Cash
- 2nd Place: Dell XPS M1730 Laptop with 17-inch Hi-Def Screen or \$2,500 Cash
- 3rd Place: Dell Latitude Laptop or \$1,300 Cash
- 4th – 6th Place: 16GB iPod Touch or \$400 Cash



Sponsored by:



For more information and to download a contest entry form, visit www.dlpdesign.com/designcontest

Brought to you by DLP Design
and Nuts & Volts Magazine.



NUTS AND VOLTS

Serial-to-Ethernet Converter

Available Now!

New 2008 Lower Pricing

PoE SitePlayer Telnet System

- \$99.95

SitePlayer Telnet System

- \$79.95

SitePlayer Telnet Modules

- \$24.95

SitePlayerTM
TELNET



SitePlayer Telnet Module



Sample Browser Screenshot



Power Over Ethernet

- Configure and upgrade through web browser
- Access micro-controller or other RS232 serial device over The Internet
- Full RS232 handshaking
- 50 to 11500 baud data rates
- DHCP, Link Local, and static IP Addresses
- TCP/IP and UDP data transfers

siteplayer.com

NetMedia Inc. Tucson AZ 85737 t.520.544.4567 f.520.544.0800 netmedia.com

What would you like to learn today?

Parallax's Stamps in Class® offers **9 comprehensive tutorials** designed to introduce students and educators to **electronics, programming, robotics and sensor management** with the easy-to-use BASIC Stamp® microcontroller. Our **interchangeable** hardware platform and our **free** software, tutorials,

PowerPoint presentations, and sample code downloads make outfitting your classroom affordable. An excellent resource for **STEM programs**, our BASIC Stamp Educators Courses and broad support base help you get started and keep your program growing. **To learn more about Stamps In Class, visit www.parallax.com/sic.**

From BASIC Stamp programming to the Propeller chip, we've got you covered.

In the spirit of Stamps in Class, we have created our **Propeller™**

Education Kits and

Labs to provide an affordable platform for learning **cutting-edge multi-processing** microcontroller technology. With 8 processors that run simultaneously to perform independent or cooperative tasks, the Propeller chip is an ideal controller or integrated **embedded system design** projects. As always, our manual, labs, software and sample code are **free** downloads. **Visit www.parallax.com/propeller for details.**

Order 24-hours a day at www.parallax.com or call our Sales Department toll-free at 888-512-1024 (Monday - Friday, 7:00 a.m. - 5:00 p.m., PST).

BASIC Stamp and Stamp In Class are registered trademarks of Parallax Inc. Propeller, Parallax, and the Parallax logo are trademarks of Parallax Inc. Prices are subject to change without notice.

PARALLAX 
www.parallax.com